

Introduction to Machine Learning

Introduction

Machine Learning (ML) হলো Artificial Intelligence (AI)-এর একটি গুরুত্বপূর্ণ শাখা, যেখানে কম্পিউটারকে এমনভাবে তৈরি করা হয় যাতে এটি সরাসরি প্রতিটি নিয়ম (Rule) প্রোগ্রাম না করেও ডেটা থেকে শিখতে পারে, প্যাটার্ন খুঁজে বের করতে পারে এবং ভবিষ্যতের জন্য সিদ্ধান্ত বা পূর্বাভাস (Prediction) দিতে পারে।

সহজ ভাষায়, Machine Learning হলো কম্পিউটারকে অভিজ্ঞতা (Experience) থেকে শেখানোর একটি পদ্ধতি। এখানে হাজার হাজার বা লক্ষ লক্ষ ডেটা বিশ্লেষণ করে একটি Model তৈরি করা হয়। এরপর নতুন কোনো ডেটা দিলে সেই Model পূর্বে শেখা অভিজ্ঞতার ভিত্তিতে ফলাফল অনুমান করে।

Machine Learning মূলত Data Science-এর একটি গুরুত্বপূর্ণ অংশ। এটি Statistical, Mathematical এবং Computational Technique ব্যবহার করে ডেটা থেকে বিভিন্ন ধরনের জ্ঞান (Knowledge) আহরণ করে। এই জ্ঞান ব্যবহার করে ডেটার মধ্যে লুকিয়ে থাকা প্যাটার্ন শনাক্ত করা, ভবিষ্যৎ পূর্বাভাস দেওয়া, ডেটাকে বিভিন্ন দলে ভাগ করা, ডেটা বিশ্লেষণ করা এবং বিভিন্ন বাস্তব সমস্যার সমাধান করা সম্ভব হয়।

Machine Learning কেন প্রয়োজন?

আগে কম্পিউটারকে কোনো কাজ করাতে হলে প্রতিটি নিয়ম (Rule) আলাদাভাবে লিখে দিতে হতো। কিন্তু বাস্তব জীবনের অনেক সমস্যার ক্ষেত্রে নির্দিষ্ট নিয়ম তৈরি করা সম্ভব নয়। যেমন, একটি ছবিতে বিড়াল নাকি কুকুর রয়েছে, সেটি নির্ধারণ করার জন্য অসংখ্য নিয়ম লিখতে হবে, যা বাস্তবে খুবই কঠিন।

Machine Learning এই সমস্যার সমাধান করে। এটি প্রচুর পরিমাণ উদাহরণ (Training Data) থেকে নিজেই প্যাটার্ন শিখে নেয় এবং সেই শেখা জ্ঞান ব্যবহার করে নতুন ডেটার উপর সিদ্ধান্ত বা পূর্বাভাস প্রদান করে।

Machine Learning কীভাবে কাজ করে?

Machine Learning-এর কাজ সাধারণত কয়েকটি ধাপে সম্পন্ন হয়।

প্রথমে বিভিন্ন উৎস থেকে ডেটা সংগ্রহ করা হয়। এরপর ডেটা পরিষ্কার (Data Cleaning) ও প্রয়োজন অনুযায়ী প্রস্তুত (Preprocessing) করা হয়। তারপর সেই ডেটা ব্যবহার করে একটি Machine Learning Model প্রশিক্ষণ (Training) দেওয়া হয়। প্রশিক্ষণ শেষে Model-এর কার্যকারিতা পরীক্ষা (Testing) করা হয়। যদি Model সন্তোষজনক ফলাফল প্রদান করে, তাহলে সেটিকে নতুন ডেটার উপর Prediction বা Decision নেওয়ার জন্য ব্যবহার করা হয়।

সম্পূর্ণ প্রক্রিয়াটি সংক্ষেপে নিম্নরূপ:

Data Collection → Data Preprocessing → Model Training → Model Testing → Prediction

Example

ধরো, একটি Email Spam Detection System তৈরি করতে হবে।

Model-কে ১,০০,০০০টি Email দেওয়া হলো, যেখানে প্রতিটি Email-এর সাথে উল্লেখ করা আছে কোনটি Spam এবং কোনটি Not Spam।

Training-এর সময় Model লক্ষ্য করল যে "Congratulations", "Win Money", "Free Offer" ইত্যাদি শব্দ Spam Email-এ বেশি ব্যবহৃত হয়।

এখন নতুন একটি Email এলো:

Congratulations! You won \$1000.

Model পূর্বে শেখা তথ্য ব্যবহার করে অনুমান করবে যে এটি একটি Spam Email।

এখানে Model-কে কোনো নির্দিষ্ট Rule লিখে দেওয়া হয়নি; বরং এটি ডেটা থেকেই নিয়ম শিখেছে।

আরেকটি Example

ধরো, Netflix একটি Movie Recommendation System ব্যবহার করে।

Netflix লক্ষ লক্ষ ব্যবহারকারীর Movie Watch History বিশ্লেষণ করে দেখতে পেল যে যারা Avengers দেখেছে, তাদের অধিকাংশই Iron Man-ও দেখেছে।

ফলে, নতুন কোনো ব্যবহারকারী Avengers দেখলে System তাকে Iron Man দেখার পরামর্শ দেয়।

এটিও Machine Learning-এর একটি বাস্তব প্রয়োগ।

Machine Learning কী কী কাজ করতে পারে?

Machine Learning বিভিন্ন ধরনের বুদ্ধিমান কাজ সম্পাদন করতে সক্ষম।

- ভবিষ্যতের Prediction করা
 - Data Classification করা
 - একই ধরনের ডেটাকে Group করা
 - Hidden Pattern শনাক্ত করা
 - Recommendation প্রদান করা
 - Fraud Detection করা
 - Image ও Speech Recognition করা
-

Machine Learning-এর সীমাবদ্ধতা

Machine Learning একটি শক্তিশালী প্রযুক্তি হলেও এটি সব সমস্যার সমাধান করতে পারে না। এটি মানুষের সমালোচনামূলক চিন্তাভাবনা (Critical Thinking), যৌক্তিক বিশ্লেষণ (Logical Reasoning) কিংবা বৈজ্ঞানিক সমস্যা সমাধানের বিকল্প নয়। Machine Learning মূলত এমন একটি প্রযুক্তি, যা ডেটা থেকে শেখে এবং শেখা জ্ঞান ব্যবহার করে সিদ্ধান্ত গ্রহণে সহায়তা করে। এর কার্যকারিতা সম্পূর্ণভাবে নির্ভর করে ডেটার গুণগত মান, ব্যবহৃত অ্যালগরিদম এবং সঠিক সমস্যা বিশ্লেষণের উপর।

Machine Learning-এর ব্যবহার

- Email Spam Detection
 - Face Recognition
 - Self-driving Car
 - Medical Disease Prediction
 - Weather Forecasting
 - Stock Market Analysis
 - Customer Recommendation System
 - Loan Approval System
 - Fraud Detection
 - Voice Assistant (Google Assistant, Siri)
-

Advantages

- বড় পরিমাণ ডেটা দ্রুত বিশ্লেষণ করতে পারে।
 - মানুষের চোখে সহজে ধরা না পড়া প্যাটার্ন শনাক্ত করতে পারে।
 - অভিজ্ঞতার মাধ্যমে সময়ের সাথে সাথে আরও ভালো ফলাফল প্রদান করতে পারে।
 - Prediction-এর নির্ভুলতা বৃদ্ধি করতে পারে।
 - বিভিন্ন বাস্তব সমস্যার স্বয়ংক্রিয় সমাধান দিতে সক্ষম।
-

Disadvantages

- ভালো ফলাফলের জন্য উচ্চমানের এবং পর্যাপ্ত ডেটা প্রয়োজন।
- ভুল বা পক্ষপাতযুক্ত (Biased) ডেটা ব্যবহার করলে ভুল সিদ্ধান্ত শিখতে পারে।
- কিছু Machine Learning Model-এর সিদ্ধান্ত ব্যাখ্যা করা কঠিন।

- Model Training-এর জন্য অনেক সময় এবং উচ্চ Computational Power প্রয়োজন হতে পারে।
-

Key Points

- Machine Learning হলো Artificial Intelligence-এর একটি গুরুত্বপূর্ণ শাখা।
 - এটি ডেটা থেকে শেখে এবং নতুন ডেটার উপর Prediction বা Decision প্রদান করে।
 - প্রতিটি নিয়ম আলাদাভাবে Program করার প্রয়োজন হয় না।
 - এটি Statistics, Mathematics এবং Computational Technique-এর উপর ভিত্তি করে কাজ করে।
 - এটি Data Science-এর একটি গুরুত্বপূর্ণ অংশ।
 - Prediction, Classification, Clustering এবং Recommendation-এর মতো বিভিন্ন ক্ষেত্রে এর ব্যাপক ব্যবহার রয়েছে।
-

Exam Note

Machine Learning (ML) হলো Artificial Intelligence-এর একটি শাখা, যেখানে কম্পিউটার ডেটা থেকে শেখে এবং সেই শেখা জ্ঞান ব্যবহার করে নতুন ডেটার উপর Prediction বা Decision প্রদান করে। এটি মূলত Statistics, Mathematics এবং Computational Technique-এর সমন্বয়ে কাজ করে এবং Data Science-এর অন্যতম গুরুত্বপূর্ণ প্রযুক্তি। Prediction, Classification, Clustering, Recommendation System, Fraud Detection এবং Image Recognition-এর মতো বিভিন্ন ক্ষেত্রে Machine Learning ব্যাপকভাবে ব্যবহৃত হয়।

Supervised Learning

Introduction

Supervised Learning হলো Machine Learning-এর সবচেয়ে জনপ্রিয় এবং বহুল ব্যবহৃত Learning পদ্ধতি। এখানে Model-কে এমন একটি Dataset দিয়ে প্রশিক্ষণ (Training) দেওয়া হয়, যেখানে প্রতিটি Input Data-এর সঠিক Output বা Label আগে থেকেই জানা থাকে। অর্থাৎ, Model শেখার সময় শুধুমাত্র Input দেখে না, বরং তার সঠিক উত্তরও দেখে শেখে।

সহজভাবে বলা যায়, Supervised Learning হলো একজন শিক্ষকের তত্ত্বাবধানে শেখার মতো। যেমন একজন শিক্ষক শিক্ষার্থীকে প্রশ্নের সঙ্গে সঠিক উত্তর দেখিয়ে শেখান, ঠিক তেমনি Supervised Learning-এ Model-কে Input এবং তার সঠিক Output একসঙ্গে দেওয়া হয়। এর ফলে Model Input ও Output-এর মধ্যকার সম্পর্ক (Relationship) শিখে নেয় এবং পরবর্তীতে নতুন Input-এর জন্য সঠিক Output অনুমান করতে পারে।

Supervised Learning কেন প্রয়োজন?

অনেক বাস্তব সমস্যায় আমরা আগে থেকেই সঠিক ফলাফল জানি। যেমন, কোনো Email Spam নাকি Not Spam, কোনো রোগী Diabetes-এ আক্রান্ত কিনা, কোনো বাড়ির মূল্য কত হতে পারে অথবা কোনো গ্রাহক ভবিষ্যতে Churn করবে কিনা। এসব ক্ষেত্রে অতীতের Labelled Data ব্যবহার করে একটি Model তৈরি করা যায়, যা ভবিষ্যতের নতুন ডেটার জন্য সঠিক Prediction করতে সক্ষম হয়।

Supervised Learning মূলত তখনই ব্যবহৃত হয়, যখন Input এবং তার সঠিক Output উভয়ই উপলব্ধ থাকে এবং সেই সম্পর্ক শিখিয়ে ভবিষ্যতের জন্য একটি নির্ভরযোগ্য Prediction Model তৈরি করতে হয়।

Supervised Learning কীভাবে কাজ করে?

Supervised Learning-এর প্রথম ধাপে একটি Labelled Dataset সংগ্রহ করা হয়। এই Dataset-এর প্রতিটি রেকর্ডে একটি বা একাধিক Input Feature এবং একটি সঠিক Output থাকে। এরপর এই ডেটা ব্যবহার করে একটি Machine Learning Algorithm প্রশিক্ষণ গ্রহণ করে। Training-এর সময় Algorithm এমন একটি Pattern খুঁজে বের করার চেষ্টা করে, যা Input এবং Output-এর মধ্যে সম্পর্ক ব্যাখ্যা করতে পারে।

Training সম্পন্ন হলে Model-কে নতুন Data দেওয়া হয়। যেহেতু নতুন Data-এর Output আগে থেকে জানা থাকে না, তাই Model পূর্বে শেখা Pattern ব্যবহার করে সম্ভাব্য Output Prediction করে।

পুরো প্রক্রিয়াটি সংক্ষেপে নিম্নরূপ:

Labelled Data → Model Training → Pattern Learning → New Data → Prediction

Example 1: House Price Prediction

ধরো, একটি Dataset-এ নিচের তথ্য রয়েছে।

House Size (sq ft)	Price (Lakh Tk)
1000	45
1200	55
1500	68
1800	82

এখানে House Size হলো Input এবং Price হলো Output।

Model এই ডেটা দেখে শিখবে যে বাড়ির আকার বাড়ার সঙ্গে সঙ্গে সাধারণত দামও বাড়ে।

এখন যদি ১৬০০ sq ft-এর একটি নতুন বাড়ির তথ্য দেওয়া হয়, তাহলে Model পূর্বে শেখা সম্পর্ক ব্যবহার করে সম্ভাব্য মূল্য Prediction করবে।

Example 2: Email Spam Detection

ধরো, একটি Dataset-এ ৫০,০০০টি Email রয়েছে।

Email	Label
Win Money Now	Spam
Meeting at 10 AM	Not Spam
Claim Your Prize	Spam
Project Report Attached	Not Spam

Model Training-এর সময় শিখবে কোন ধরনের শব্দ বা বৈশিষ্ট্য Spam Email-এ বেশি দেখা যায়।

Supervised Learning-এর প্রধান কাজ

Supervised Learning সাধারণত দুই ধরনের সমস্যা সমাধান করে।

- Regression
- Classification

Regression-এর ক্ষেত্রে Output একটি Numerical Value হয়। যেমন বাড়ির মূল্য, তাপমাত্রা অথবা বিক্রয়ের পরিমাণ Prediction করা।

Classification-এর ক্ষেত্রে Output একটি Category বা Class হয়। যেমন Spam বা Not Spam, Positive বা Negative, Disease বা No Disease।

পরবর্তী অধ্যায়গুলোতে এই দুই ধরনের সমস্যাই বিস্তারিতভাবে আলোচনা করা হবে।

Supervised Learning-এর বৈশিষ্ট্য

- Training Data অবশ্যই Labelled হতে হবে।
 - প্রতিটি Input-এর একটি Correct Output থাকে।
 - Input এবং Output-এর মধ্যকার সম্পর্ক শেখা হয়।
 - নতুন Data-এর জন্য Prediction করা যায়।
 - Prediction-এর Accuracy Training Data-এর গুণগত মানের উপর নির্ভর করে।
-

Supervised Learning-এর ব্যবহার

- House Price Prediction
 - Student Result Prediction
 - Medical Disease Diagnosis
 - Email Spam Detection
 - Customer Churn Prediction
 - Credit Card Fraud Detection
 - Loan Approval System
 - Weather Forecasting
 - Sales Prediction
 - Face Recognition
-

Advantages

- Prediction-এর Accuracy সাধারণত বেশি হয়।
 - শেখার জন্য সঠিক উত্তর থাকায় Model সহজে Pattern শিখতে পারে।
 - Regression এবং Classification উভয় ধরনের সমস্যার সমাধান করতে পারে।
 - বাস্তব জীবনের অসংখ্য সমস্যায় সফলভাবে ব্যবহৃত হয়।
 - Model-এর Performance সহজে মূল্যায়ন করা যায়।
-

Disadvantages

- Training-এর জন্য প্রচুর Labelled Data প্রয়োজন।

- Label সংগ্রহ করা সময়সাপেক্ষ এবং ব্যয়বহুল হতে পারে।
 - ভুল Label থাকলে Model ভুল শিখবে।
 - নতুন ধরনের Pattern দেখা দিলে Model সবসময় সঠিক Prediction নাও করতে পারে।
 - বড় Dataset Training করতে বেশি সময় এবং Computational Power প্রয়োজন হয়।
-

Key Points

- Supervised Learning-এ Labelled Data ব্যবহার করা হয়।
 - প্রতিটি Input-এর Correct Output আগে থেকেই জানা থাকে।
 - Model Input এবং Output-এর সম্পর্ক শিখে।
 - প্রধান দুটি সমস্যা হলো Regression এবং Classification।
 - Training শেষে নতুন Data-এর জন্য Prediction করা হয়।
 - এটি Machine Learning-এর সবচেয়ে বেশি ব্যবহৃত Learning পদ্ধতি।
-

Exam Note

Supervised Learning হলো Machine Learning-এর এমন একটি Learning পদ্ধতি, যেখানে প্রতিটি Training Data-এর সঙ্গে সঠিক Output বা Label দেওয়া থাকে। Training-এর সময় Model Input এবং Output-এর মধ্যকার সম্পর্ক শিখে এবং সেই শেখা জ্ঞান ব্যবহার করে নতুন Data-এর জন্য Prediction বা Classification করে। Supervised Learning-এর প্রধান দুটি কাজ হলো **Regression**, যেখানে Numerical Value Prediction করা হয়, এবং **Classification**, যেখানে Data-কে বিভিন্ন Category-তে ভাগ করা হয়।

Supervised Learning-এর দুটি প্রধান ধরনের সমস্যা হলো **Regression** এবং **Classification**। এগুলোর জন্য বিভিন্ন Machine Learning Algorithm ব্যবহার করা হয়।

Regression Algorithms

Regression ব্যবহার করা হয় যখন Output একটি **Continuous Numerical Value** হয়।

উদাহরণ:

- Linear Regression
- Polynomial Regression
- Ridge Regression
- Lasso Regression
- Elastic Net Regression
- Decision Tree Regressor
- Random Forest Regressor
- Support Vector Regressor (SVR)
- K-Nearest Neighbors Regressor (KNN Regressor)
- Gradient Boosting Regressor
- XGBoost Regressor
- AdaBoost Regressor
- LightGBM Regressor
- CatBoost Regressor

Example

Problem	Output
House Price Prediction	85,00,000 Tk
Student GPA Prediction	3.82
Temperature Prediction	32.5°C
Monthly Sales Prediction	12,500 Units

Classification Algorithms

Classification ব্যবহার করা হয় যখন Output একটি **Category (Class)** হয়।

উদাহরণ:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Naive Bayes
- Support Vector Machine (SVM)
- Decision Tree Classifier
- Random Forest Classifier
- Gradient Boosting Classifier
- XGBoost Classifier
- AdaBoost Classifier
- LightGBM Classifier
- CatBoost Classifier
- Artificial Neural Network (ANN)

Example

Problem	Output
Email Detection	Spam / Not Spam
Disease Detection	Disease / No Disease
Customer Churn	Churn / Not Churn
Handwritten Digit Recognition	0–9
Animal Classification	Cat / Dog

সহজে মনে রাখার উপায়

Regression	Classification
Output একটি সংখ্যা (Numerical Value)	Output একটি শ্রেণি (Category/Class)
House Price Prediction	Spam Detection
Salary Prediction	Disease Detection
Temperature Prediction	Customer Churn Prediction
Sales Prediction	Face Recognition

Unsupervised Learning

Introduction

Unsupervised Learning হলো Machine Learning-এর এমন একটি Learning পদ্ধতি, যেখানে Model-কে Training দেওয়ার সময় কোনো Label বা Correct Output দেওয়া হয় না। অর্থাৎ, Model শুধুমাত্র Input Data দেখে নিজেই ডেটার মধ্যে থাকা লুকানো Pattern (Hidden Pattern), Structure এবং Relationship খুঁজে বের করার চেষ্টা করে।

Supervised Learning-এ যেমন প্রতিটি Input-এর সঠিক উত্তর আগে থেকেই জানা থাকে, Unsupervised Learning-এ তেমন কোনো উত্তর থাকে না। তাই এখানে Model-কে কেউ বলে দেয় না কোন Data কোন Category-তে পড়বে। Model নিজেই Similar Data-কে একসাথে Group করে এবং বিভিন্ন Pattern আবিষ্কার করে।

Unsupervised Learning কেন প্রয়োজন?

বাস্তব জীবনের অধিকাংশ Data-ই Unlabelled হয়। অনেক ক্ষেত্রে লক্ষ লক্ষ Data-এর জন্য Label তৈরি করা সময়সাপেক্ষ, ব্যয়বহুল এবং কখনও কখনও অসম্ভব। তাই এমন একটি পদ্ধতির প্রয়োজন হয়, যা কোনো Label ছাড়াই Data বিশ্লেষণ করে গুরুত্বপূর্ণ তথ্য বের করতে পারে।

Unsupervised Learning মূলত Data-এর মধ্যে লুকিয়ে থাকা মিল (Similarity), অমিল (Difference), সম্পর্ক (Relationship) এবং গঠন (Structure) আবিষ্কার করার জন্য ব্যবহৃত হয়। এটি Data Exploration, Customer Segmentation, Pattern Discovery এবং Feature Reduction-এর মতো কাজে অত্যন্ত কার্যকর।

Unsupervised Learning কীভাবে কাজ করে?

প্রথমে একটি Unlabelled Dataset Model-কে দেওয়া হয়। এরপর Model প্রতিটি Data Point-এর বৈশিষ্ট্য (Feature) বিশ্লেষণ করে। যেসব Data-এর বৈশিষ্ট্য একে অপরের সাথে বেশি মিল থাকে, সেগুলোকে একই Group বা Cluster-এ রাখে। আবার যেসব Data ভিন্ন, সেগুলোকে আলাদা Group-এ রাখে।

এই পুরো প্রক্রিয়ায় কোনো মানুষ Model-কে বলে দেয় না কোন Data কোন Group-এ থাকবে। সমস্ত সিদ্ধান্ত Model নিজেই Data-এর অভ্যন্তরীণ Pattern বিশ্লেষণ করে গ্রহণ করে।

পুরো প্রক্রিয়াটি সংক্ষেপে নিম্নরূপ:

Unlabelled Data → Pattern Discovery → Grouping / Relationship Finding → Knowledge Extraction

Example 1: Customer Segmentation

ধরো, একটি Shopping Mall-এর কাছে ৫০,০০০ জন Customer-এর তথ্য রয়েছে।

প্রতিটি Customer-এর জন্য নিচের তথ্য রয়েছে।

- Age

- Monthly Income
- Annual Spending

কিন্তু কোথাও লেখা নেই কে VIP Customer, কে Regular Customer এবং কে Budget Customer।

এখন Unsupervised Learning এই Data বিশ্লেষণ করে নিজেই তিনটি Group তৈরি করতে পারে।

- Cluster 1 → High Income, High Spending
- Cluster 2 → Medium Income, Medium Spending
- Cluster 3 → Low Income, Low Spending

অর্থাৎ, কোনো Label না থাকা সত্ত্বেও Model নিজেই Customer-দের বিভিন্ন Group-এ ভাগ করেছে।

Example 2: Image Grouping

ধরো, একটি Folder-এ ২০,০০০টি প্রাণীর ছবি রয়েছে।

ছবিগুলোর কোনো নাম নেই।

Model ছবিগুলো বিশ্লেষণ করে দেখতে পেল কিছু ছবির বৈশিষ্ট্য একই রকম।

ফলে এটি নিজেই আলাদা Group তৈরি করল।

- Group 1 → বিড়ালের মতো প্রাণী
- Group 2 → কুকুরের মতো প্রাণী
- Group 3 → ঘোড়ার মতো প্রাণী

Model জানে না এগুলোর নাম Cat বা Dog। এটি শুধু Similarity দেখে Group তৈরি করেছে।

Example 3: Market Basket Analysis

ধরো, একটি Super Shop প্রতিদিন হাজার হাজার Transaction সংরক্ষণ করে।

Data বিশ্লেষণ করে দেখা গেল,

- যারা Bread কেনে,
- তাদের অধিকাংশ Butter-ও কেনে।

আবার,

- যারা Laptop কেনে,
- তাদের অনেকেই Mouse কিনে।

এই ধরনের সম্পর্ক আগে থেকে লেখা ছিল না। Model নিজেই এই সম্পর্ক আবিষ্কার করেছে।

এই ধরনের কাজকে Association Rule Learning বলা হয়।

Unsupervised Learning-এর প্রধান কাজ

Unsupervised Learning সাধারণত তিন ধরনের কাজ করে।

- Clustering
- Association Rule Learning
- Dimensionality Reduction

Clustering-এর মাধ্যমে একই ধরনের Data-কে Group করা হয়।

Association Rule Learning-এর মাধ্যমে বিভিন্ন Data-এর মধ্যে সম্পর্ক (Relationship) খুঁজে বের করা হয়।

Dimensionality Reduction-এর মাধ্যমে অপ্রয়োজনীয় বা কম গুরুত্বপূর্ণ Feature কমিয়ে Data-কে সহজ করা হয়, যাতে বিশ্লেষণ ও Visualization সহজ হয়।

Unsupervised Learning-এর বৈশিষ্ট্য

- Training Data Unlabelled হয়।
 - কোনো Correct Output আগে থেকে দেওয়া থাকে না।
 - Model নিজেই Pattern আবিষ্কার করে।
 - Similar Data-কে একই Cluster-এ রাখে।
 - Hidden Structure এবং Relationship খুঁজে বের করে।
 - নতুন Knowledge Discover করার জন্য উপযোগী।
-

জনপ্রিয় Unsupervised Learning Algorithms

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN
- Gaussian Mixture Model (GMM)
- Fuzzy C-Means
- Principal Component Analysis (PCA)

- Apriori Algorithm

Supervised Learning এবং Unsupervised Learning-এর পার্থক্য

Supervised Learning	Unsupervised Learning
Labelled Data ব্যবহার করে	Unlabelled Data ব্যবহার করে
Correct Output জানা থাকে	Correct Output জানা থাকে না
Prediction করে	Pattern খুঁজে বের করে
Regression ও Classification সমস্যা সমাধান করে	Clustering, Association এবং Dimensionality Reduction সমস্যা সমাধান করে

Unsupervised Learning-এর ব্যবহার

- Customer Segmentation
 - Recommendation System
 - Market Basket Analysis
 - Image Grouping
 - Document Clustering
 - Social Network Analysis
 - Anomaly Detection
 - Gene Analysis
 - Data Compression
 - Feature Extraction
-

Advantages

- Labelled Data প্রয়োজন হয় না।
- Hidden Pattern সহজে আবিষ্কার করতে পারে।
- বড় Dataset বিশ্লেষণে কার্যকর।
- Data Exploration-এর জন্য অত্যন্ত উপযোগী।
- Feature Reduction ও Data Visualization সহজ করে।

Disadvantages

- Accuracy নির্ধারণ করা কঠিন।
- তৈরি হওয়া Cluster সবসময় অর্থবহ নাও হতে পারে।
- ফলাফল ব্যাখ্যা করতে Domain Knowledge প্রয়োজন হতে পারে।
- Algorithm-এর Parameter পরিবর্তনের ফলে ভিন্ন ফলাফল আসতে পারে।
- কিছু Algorithm Computationally ব্যয়বহুল।

Key Points

- Unsupervised Learning-এ কোনো Label থাকে না।
- Model নিজেই Pattern ও Structure আবিষ্কার করে।
- Similar Data-কে Cluster-এ ভাগ করা হয়।
- প্রধান কাজ হলো Clustering, Association Rule Learning এবং Dimensionality Reduction।
- K-Means, Hierarchical Clustering, DBSCAN, GMM এবং PCA জনপ্রিয় Algorithm।
- Data Exploration এবং Customer Segmentation-এ এটি ব্যাপকভাবে ব্যবহৃত হয়।

Exam Note

Unsupervised Learning হলো Machine Learning-এর এমন একটি পদ্ধতি, যেখানে Model-কে শুধুমাত্র Unlabelled Data দেওয়া হয় এবং কোনো Correct Output প্রদান করা হয় না। Model নিজেই Data-এর মধ্যে থাকা Hidden Pattern, Similarity এবং Relationship খুঁজে বের করে। এর প্রধান কাজ হলো Clustering, Association Rule Learning এবং Dimensionality Reduction। Customer Segmentation, Market Basket Analysis, Image Grouping এবং Feature Extraction-এর মতো বাস্তব সমস্যায় Unsupervised Learning ব্যাপকভাবে ব্যবহৃত হয়।

Reinforcement Learning

Introduction

Reinforcement Learning (RL) হলো Machine Learning-এর এমন একটি Learning পদ্ধতি, যেখানে একটি Agent তার চারপাশের Environment-এর সাথে বারবার Interaction করে শেখে। এখানে Agent-কে শুরুতে কোনো সঠিক উত্তর (Label) দেওয়া হয় না। বরং প্রতিটি কাজের (Action) জন্য তাকে Reward অথবা Penalty দেওয়া হয়। Agent-এর মূল লক্ষ্য হলো এমন Action নির্বাচন করা, যাতে দীর্ঘমেয়াদে সর্বোচ্চ মোট Reward (Cumulative Reward) অর্জন করা যায়।

সহজ ভাষায়, Reinforcement Learning হলো Trial and Error-এর মাধ্যমে শেখার একটি পদ্ধতি। Agent প্রথমে বিভিন্ন Action চেষ্টা করে, তারপর কোন Action ভালো ফল দেয় এবং কোনটি খারাপ ফল দেয় তা বুঝে ধীরে ধীরে নিজের সিদ্ধান্ত গ্রহণের ক্ষমতা উন্নত করে।

Reinforcement Learning কেন প্রয়োজন?

অনেক বাস্তব সমস্যায় সঠিক উত্তর আগে থেকে জানা থাকে না। সেখানে শুধুমাত্র একটি Goal থাকে এবং সেই Goal অর্জনের জন্য বারবার চেষ্টা করতে হয়। প্রতিটি সিদ্ধান্তের ফলাফল পর্যবেক্ষণ করে ধীরে ধীরে সবচেয়ে ভালো কৌশল (Strategy) শেখা হয়।

যেমন, একটি Robot-কে হাঁটা শেখানো, একটি Self-driving Car-কে নিরাপদে গাড়ি চালানো শেখানো অথবা একটি Game Playing AI-কে জয়ী হওয়া শেখানো। এসব ক্ষেত্রে প্রতিটি পরিস্থিতির জন্য আগে থেকে Rule লেখা সম্ভব নয়। তাই Reinforcement Learning ব্যবহার করা হয়।

Reinforcement Learning কীভাবে কাজ করে?

Reinforcement Learning-এ চারটি প্রধান উপাদান থাকে।

- Agent
- Environment
- Action
- Reward

প্রথমে Agent একটি নির্দিষ্ট Environment-এ অবস্থান করে। এরপর Agent একটি Action গ্রহণ করে। Environment সেই Action-এর ফলাফল অনুযায়ী একটি Reward অথবা Penalty প্রদান করে এবং একটি নতুন State-এ চলে যায়। এরপর Agent আবার নতুন State দেখে পরবর্তী Action নির্বাচন করে।

এই প্রক্রিয়া বারবার চলতে থাকে। সময়ের সাথে সাথে Agent বুঝতে শেখে কোন Action বেশি Reward দেয় এবং কোন Action এড়িয়ে চলা উচিত।

পুরো প্রক্রিয়াটি সংক্ষেপে নিম্নরূপ:

Agent → Action → Environment → Reward/Penalty → New State → Next Action

Reinforcement Learning-এর প্রধান উপাদান

Agent

Agent হলো শেখার মূল অংশ, যা সিদ্ধান্ত গ্রহণ করে। এটি একটি Robot, Game Player, Self-driving Car অথবা Software Program হতে পারে।

Environment

Environment হলো সেই পরিবেশ, যেখানে Agent কাজ করে। Agent-এর প্রতিটি Action-এর প্রতিক্রিয়া Environment প্রদান করে।

State

State হলো কোনো নির্দিষ্ট মুহূর্তে Environment-এর বর্তমান অবস্থা।

উদাহরণ হিসেবে Chess Game-এ Board-এর বর্তমান অবস্থাই একটি State।

Action

Action হলো Agent-এর নেওয়া সিদ্ধান্ত।

উদাহরণ হিসেবে Chess-এ একটি গুটি এক ঘর সামনে সরানো একটি Action।

Reward

Reward হলো Agent-এর কাজের মূল্যায়ন।

সঠিক Action নিলে Positive Reward এবং ভুল Action নিলে Negative Reward বা Penalty দেওয়া হয়।

Policy

Policy হলো এমন একটি নিয়ম বা কৌশল, যার মাধ্যমে Agent নির্ধারণ করে কোন State-এ কোন Action নেওয়া হবে।

Example 1: Maze Problem

ধরো, একটি Robot-কে একটি Maze-এর মধ্যে রাখা হয়েছে।

Robot-এর লক্ষ্য হলো Exit পর্যন্ত পৌঁছানো।

নিয়মগুলো হলো:

- সঠিক পথে এগোলে +10 Reward।
- দেয়ালে ধাক্কা খেলে -5 Penalty।
- Exit-এ পৌঁছালে +100 Reward।

প্রথমদিকে Robot অনেক ভুল করবে। কিন্তু বারবার চেষ্টা করার পর এটি বুঝে যাবে কোন পথটি সবচেয়ে ভালো এবং দ্রুত Exit-এ পৌঁছায়।

এভাবেই Robot Trial and Error-এর মাধ্যমে শেখে।

Example 2: Self-driving Car

ধরো, একটি Self-driving Car রাস্তা দিয়ে চলছে।

যদি Car সঠিক Lane-এ চলে এবং নিরাপদে গন্তব্যে পৌঁছায়, তাহলে Positive Reward পায়।

যদি Traffic Rule ভঙ্গ করে অথবা Accident ঘটায়, তাহলে Negative Reward বা Penalty পায়।

হাজার হাজার Simulation-এর মাধ্যমে Car ধীরে ধীরে নিরাপদভাবে গাড়ি চালানো শিখে।

Example 3: Chess Playing AI

ধরো, একটি AI Chess খেলছে।

শুরুতে এটি ভালো Move জানে না।

প্রতিটি Move-এর পর Game-এর ফলাফল দেখে শেখে।

- জিতলে Positive Reward।
- হারলে Negative Reward।
- ভুল Strategy নিলে কম Reward।

লক্ষ লক্ষ Game খেলার মাধ্যমে AI ধীরে ধীরে শক্তিশালী Player হয়ে ওঠে।

Reinforcement Learning-এর বৈশিষ্ট্য

- কোনো Labelled Data প্রয়োজন হয় না।
 - Trial and Error-এর মাধ্যমে শেখে।
 - Reward এবং Penalty-এর উপর ভিত্তি করে সিদ্ধান্ত উন্নত করে।
 - Long-term Reward সর্বাধিক করার চেষ্টা করে।
 - Environment-এর সাথে ধারাবাহিকভাবে Interaction করে শেখে।
-

Reinforcement Learning-এর ব্যবহার

- Self-driving Car
- Robotics
- Game Playing AI
- Drone Navigation
- Industrial Automation
- Traffic Signal Control
- Recommendation System
- Resource Management
- Financial Trading
- Smart Grid Optimization

Advantages

- নিজস্ব অভিজ্ঞতা থেকে শেখে।
- জটিল Decision-Making সমস্যার সমাধান করতে পারে।
- দীর্ঘমেয়াদি সর্বোত্তম Strategy শিখতে সক্ষম।
- Dynamic Environment-এ ভালোভাবে কাজ করতে পারে।
- মানুষের লেখা Rule-এর উপর নির্ভরশীল নয়।

Disadvantages

- Training করতে অনেক সময় লাগে।
- প্রচুর Trial এবং Simulation প্রয়োজন হয়।
- Computational Cost অনেক বেশি হতে পারে।
- Reward Function সঠিকভাবে ডিজাইন করা কঠিন।
- ভুল Reward দিলে ভুল Strategy শিখতে পারে।

Supervised Learning, Unsupervised Learning এবং Reinforcement Learning-এর পার্থক্য

Learning Type	Data	শেখার পদ্ধতি	প্রধান কাজ
---------------	------	--------------	------------

Supervised Learning	Labelled Data	সঠিক উত্তর দেখে শেখে	Prediction ও Classification
Unsupervised Learning	Unlabelled Data	Pattern নিজে খুঁজে বের করে	Clustering ও Pattern Discovery
Reinforcement Learning	Reward এবং Penalty	Trial and Error-এর মাধ্যমে শেখে	সর্বোচ্চ Reward অর্জন করা

Key Points

- Reinforcement Learning-এ একটি Agent এবং একটি Environment থাকে।
- Agent বিভিন্ন Action নিয়ে শেখে।
- সঠিক Action-এর জন্য Reward এবং ভুল Action-এর জন্য Penalty দেওয়া হয়।
- কোনো Labelled Data ব্যবহার করা হয় না।
- লক্ষ্য হলো দীর্ঘমেয়াদে সর্বোচ্চ Reward অর্জন করা।
- Robotics, Game AI এবং Self-driving Car-এ Reinforcement Learning ব্যাপকভাবে ব্যবহৃত হয়।

Exam Note

Reinforcement Learning হলো Machine Learning-এর একটি Learning পদ্ধতি, যেখানে একটি Agent Environment-এর সাথে Interaction করে Trial and Error-এর মাধ্যমে শেখে। এখানে কোনো Labelled Data ব্যবহার করা হয় না। বরং প্রতিটি Action-এর জন্য Reward অথবা Penalty প্রদান করা হয়। Agent-এর লক্ষ্য হলো এমন একটি Policy শেখা, যার মাধ্যমে দীর্ঘমেয়াদে সর্বোচ্চ Cumulative Reward অর্জন করা যায়। Self-driving Car, Robotics, Game Playing AI এবং Industrial Automation-এ Reinforcement Learning ব্যাপকভাবে ব্যবহৃত হয়।

Exploratory Data Analysis (EDA)

Introduction

Exploratory Data Analysis (EDA) হলো কোনো Dataset-এর উপর প্রাথমিক অনুসন্ধান (Initial Investigation) করার একটি গুরুত্বপূর্ণ প্রক্রিয়া। Machine Learning Model তৈরি করার আগে Dataset-কে ভালোভাবে বোঝার জন্য EDA করা হয়। এর মাধ্যমে Data-এর গঠন (Structure), বৈশিষ্ট্য (Characteristics), গুণগত মান (Quality), বিভিন্ন Variable-এর মধ্যে সম্পর্ক (Relationship) এবং Data-এর মধ্যে লুকিয়ে থাকা Pattern সম্পর্কে ধারণা পাওয়া যায়।

সহজ ভাষায়, EDA হলো Data-কে "বোঝার" একটি প্রক্রিয়া। Model তৈরি করার আগে Data কেমন, কোথায় সমস্যা আছে, কোথায় অস্বাভাবিক মান (Outlier) রয়েছে, কোনো তথ্য অনুপস্থিত (Missing Value) কিনা এবং কোন Feature গুরুত্বপূর্ণ হতে পারে, এসব বিষয় EDA-এর মাধ্যমে জানা যায়।

EDA কেন প্রয়োজন?

Machine Learning-এর একটি গুরুত্বপূর্ণ নীতি হলো,

"Garbage In, Garbage Out (GIGO)"

অর্থাৎ, যদি Model-এ নিম্নমানের বা ভুল Data দেওয়া হয়, তাহলে Model কখনোই ভালো ফলাফল দিতে পারবে না।

EDA করার মাধ্যমে Data-এর সমস্যা আগে থেকেই শনাক্ত করা যায়। ফলে Data পরিষ্কার (Cleaning), রূপান্তর (Transformation) এবং প্রস্তুত (Preprocessing) করা সহজ হয়। এতে Model-এর Accuracy এবং Performance উল্লেখযোগ্যভাবে বৃদ্ধি পায়।

EDA-এর উদ্দেশ্য

EDA-এর প্রধান উদ্দেশ্য হলো Dataset সম্পর্কে পরিষ্কার ধারণা অর্জন করা।

এর মাধ্যমে Data-এর Distribution, Missing Value, Duplicate Data, Outlier, Feature-এর সম্পর্ক এবং Target Variable-এর আচরণ বিশ্লেষণ করা হয়। পাশাপাশি কোন Feature Machine Learning Model-এর জন্য গুরুত্বপূর্ণ হতে পারে, সেটিও নির্ধারণ করা যায়।

EDA কীভাবে কাজ করে?

EDA সাধারণত কয়েকটি ধাপে সম্পন্ন করা হয়।

প্রথমে Dataset লোড করা হয় এবং Data-এর আকার (Shape), Column, Data Type এবং মৌলিক তথ্য দেখা হয়। এরপর Missing Value, Duplicate Record এবং Outlier আছে কিনা তা পরীক্ষা করা হয়। তারপর বিভিন্ন Statistical Summary এবং Visualization ব্যবহার করে Data বিশ্লেষণ করা হয়। সবশেষে Feature-এর মধ্যে সম্পর্ক বিশ্লেষণ করে Model তৈরির জন্য Data প্রস্তুত করা হয়।

পুরো প্রক্রিয়াটি সংক্ষেপে নিম্নরূপ:

Data Collection → Data Understanding → Data Cleaning → Data Visualization → Relationship Analysis → Feature Understanding

EDA-এর প্রধান ধাপসমূহ

- Data Understanding
 - Data Cleaning
 - Univariate Analysis
 - Bivariate Analysis
 - Multivariate Analysis
 - Data Visualization
 - Feature Engineering (প্রয়োজনে)
-

Data Understanding

এই ধাপে Dataset সম্পর্কে প্রাথমিক ধারণা নেওয়া হয়।

যেসব বিষয় সাধারণত দেখা হয়:

- Dataset-এর Row এবং Column সংখ্যা
- প্রতিটি Feature-এর Data Type
- Numerical ও Categorical Feature
- Target Variable
- Statistical Summary

Python-এ সাধারণত নিচের Function ব্যবহার করা হয়।

- `df.head()`
- `df.tail()`
- `df.shape`
- `df.columns`
- `df.info()`
- `df.describe()`

Example

ধরো, একটি Customer Churn Dataset রয়েছে।

CustomerID	Age	Balance	Churn
101	25	50000	No
102	48	120000	Yes
103	36	85000	No

Data Understanding করার সময় প্রথমেই দেখা হবে,

- মোট কতটি Record আছে।
- কতটি Feature আছে।
- কোন Feature Numerical।
- কোন Feature Categorical।
- Target Variable কোনটি।

Data Cleaning

বাস্তব Dataset প্রায়ই সম্পূর্ণ পরিষ্কার থাকে না।

এতে বিভিন্ন সমস্যা থাকতে পারে।

- Missing Value
- Duplicate Data
- Incorrect Data
- Outlier
- Inconsistent Format

Data Cleaning-এর মাধ্যমে এসব সমস্যা সমাধান করা হয়।

Example

ধরো,

Age	Salary
-----	--------

25	40000
NaN	50000
35	60000

এখানে দ্বিতীয় Row-তে Age অনুপস্থিত।

EDA-এর সময় এটি শনাক্ত করা হবে এবং পরে Mean, Median, Mode অথবা অন্য কোনো উপযুক্ত Method ব্যবহার করে পূরণ করা হবে।

Univariate Analysis

যখন শুধুমাত্র একটি Variable বিশ্লেষণ করা হয়, তখন তাকে Univariate Analysis বলা হয়।

এর মাধ্যমে জানা যায়,

- Distribution
- Mean
- Median
- Mode
- Minimum
- Maximum
- Standard Deviation

সাধারণত Histogram, Box Plot এবং Bar Chart ব্যবহার করা হয়।

Example

শুধুমাত্র Customer-এর Age বিশ্লেষণ করা হলে সেটি Univariate Analysis।

উদ্দেশ্য হবে,

- গড় বয়স কত।
 - সর্বোচ্চ ও সর্বনিম্ন বয়স কত।
 - কোনো Outlier আছে কিনা।
-

Bivariate Analysis

যখন দুটি Variable-এর মধ্যে সম্পর্ক বিশ্লেষণ করা হয়, তখন তাকে Bivariate Analysis বলা হয়।

এখানে সাধারণত Correlation, Scatter Plot এবং Crosstab ব্যবহার করা হয়।

Example

Customer-এর Age এবং Salary-এর মধ্যে সম্পর্ক বিশ্লেষণ করা।

অথবা,

Balance এবং Churn-এর মধ্যে কোনো সম্পর্ক আছে কিনা তা বিশ্লেষণ করা।

Multivariate Analysis

যখন দুইটির বেশি Variable একসাথে বিশ্লেষণ করা হয়, তখন তাকে Multivariate Analysis বলা হয়।

এই বিশ্লেষণের মাধ্যমে একাধিক Feature একসাথে Target Variable-কে কীভাবে প্রভাবিত করছে তা বোঝা যায়।

Example

House Price Prediction-এ নিচের তিনটি Feature একসাথে বিশ্লেষণ করা।

- House Size
- Number of Bedrooms
- Location

এগুলো একসাথে House Price-কে কীভাবে প্রভাবিত করছে, সেটিই Multivariate Analysis।

Data Visualization

EDA-এর অন্যতম গুরুত্বপূর্ণ অংশ হলো Data Visualization।

Visualization-এর মাধ্যমে Data সহজে বোঝা যায় এবং অনেক Hidden Pattern দ্রুত শনাক্ত করা সম্ভব হয়।

EDA-তে বহুল ব্যবহৃত Visualization হলো:

- Histogram
- Bar Chart
- Pie Chart
- Line Chart
- Scatter Plot

- Box Plot
 - Heatmap
 - Pair Plot
-

Example

ধরো, একটি Histogram আঁকা হলো।

দেখা গেল অধিকাংশ Customer-এর Age ২৫ থেকে ৪০ বছরের মধ্যে।

এই তথ্য শুধুমাত্র সংখ্যার Table দেখে বোঝা কঠিন হলেও Histogram দেখলে সহজেই বোঝা যায়।

EDA-এর সময় যেসব বিষয় অবশ্যই পরীক্ষা করা হয়

- Missing Value
 - Duplicate Data
 - Outlier
 - Data Type
 - Data Distribution
 - Feature Relationship
 - Class Imbalance
 - Feature Correlation
-

EDA-এর ব্যবহার

- Data সম্পর্কে ধারণা নেওয়া
- Missing Value শনাক্ত করা
- Outlier শনাক্ত করা
- Feature Selection
- Feature Engineering
- Machine Learning-এর জন্য Data প্রস্তুত করা
- Data Quality উন্নত করা
- Pattern এবং Trend খুঁজে বের করা

Advantages

- Dataset সম্পর্কে গভীর ধারণা পাওয়া যায়।
- Data-এর সমস্যা সহজে শনাক্ত করা যায়।
- Model-এর Accuracy বৃদ্ধি করতে সাহায্য করে।
- গুরুত্বপূর্ণ Feature নির্বাচন সহজ হয়।
- Visualization-এর মাধ্যমে Pattern সহজে বোঝা যায়।

Disadvantages

- বড় Dataset বিশ্লেষণে সময় বেশি লাগে।
- ভুল Interpretation করলে ভুল সিদ্ধান্ত হতে পারে।
- Domain Knowledge না থাকলে অনেক Pattern বোঝা কঠিন হতে পারে।
- সব ধরনের সমস্যা শুধুমাত্র Visualization দিয়ে ধরা যায় না।

Key Points

- EDA হলো Dataset বোঝার প্রথম ধাপ।
- Machine Learning Model তৈরির আগে EDA করা অত্যন্ত গুরুত্বপূর্ণ।
- EDA-এর মাধ্যমে Missing Value, Outlier, Duplicate Data এবং Pattern শনাক্ত করা হয়।
- Data Visualization EDA-এর একটি গুরুত্বপূর্ণ অংশ।
- Univariate, Bivariate এবং Multivariate Analysis হলো EDA-এর প্রধান বিশ্লেষণ পদ্ধতি।
- EDA-এর ফলাফলের উপর ভিত্তি করেই Data Preprocessing এবং Feature Engineering করা হয়।

Exam Note

Exploratory Data Analysis (EDA) হলো Machine Learning-এর একটি গুরুত্বপূর্ণ প্রাথমিক ধাপ, যার মাধ্যমে Dataset-এর গঠন, বৈশিষ্ট্য, গুণগত মান এবং বিভিন্ন Feature-এর মধ্যে সম্পর্ক বিশ্লেষণ করা হয়। EDA-এর মাধ্যমে Missing Value, Duplicate Data, Outlier, Data Distribution এবং Hidden Pattern শনাক্ত করা যায়। এছাড়া বিভিন্ন Statistical Analysis এবং Visualization ব্যবহার করে Dataset সম্পর্কে পরিষ্কার ধারণা অর্জন করা হয়, যা পরবর্তী Data Preprocessing এবং Machine Learning Model তৈরির ভিত্তি হিসেবে কাজ করে।

Linear Regression

Introduction

Linear Regression হলো Machine Learning-এর অন্যতম মৌলিক এবং বহুল ব্যবহৃত Supervised Learning Algorithm। এটি একটি Regression Algorithm, যার প্রধান কাজ হলো এক বা একাধিক Independent Variable (Feature) এবং একটি Dependent Variable (Target)-এর মধ্যে Linear Relationship নির্ণয় করা এবং সেই সম্পর্কের ভিত্তিতে ভবিষ্যতের Numerical Value Prediction করা।

সহজ ভাষায়, যদি একটি Variable-এর পরিবর্তনের সাথে অন্য একটি Variable প্রায় সরলরৈখিক (Linear)ভাবে পরিবর্তিত হয়, তাহলে সেই সম্পর্ক বোঝা এবং ভবিষ্যতের মান অনুমান করার জন্য Linear Regression ব্যবহার করা হয়।

উদাহরণস্বরূপ, বাড়ির আকার (House Size) বাড়লে সাধারণত তার মূল্যও বাড়ে। একজন শিক্ষার্থী যত বেশি সময় পড়াশোনা করে, অনেক ক্ষেত্রে তার পরীক্ষার নম্বরও তত বেশি হয়। এই ধরনের সম্পর্ক বিশ্লেষণ ও ভবিষ্যৎ Prediction করার জন্য Linear Regression ব্যবহৃত হয়।

Linear Regression কেন প্রয়োজন?

বাস্তব জীবনে অনেক সমস্যার Output একটি সংখ্যা (Numerical Value) হয়। যেমন, বাড়ির মূল্য, মাসিক বিক্রয়, বেতন, তাপমাত্রা অথবা কোনো কোম্পানির লাভ। এসব ক্ষেত্রে শুধুমাত্র অতীতের Data দেখে ভবিষ্যতের মান অনুমান করা প্রয়োজন হয়।

Linear Regression Data-এর মধ্যে বিদ্যমান Linear Relationship শিখে এমন একটি Mathematical Model তৈরি করে, যা নতুন Data-এর জন্য সম্ভাব্য Output Prediction করতে পারে।

Linear Regression কীভাবে কাজ করে?

Linear Regression মূলত এমন একটি Straight Line খুঁজে বের করার চেষ্টা করে, যা Data-এর অধিকাংশ Point-এর সবচেয়ে কাছ দিয়ে যায়। এই Line-কে Best Fit Line বলা হয়।

Training-এর সময় Algorithm বিভিন্ন সম্ভাব্য Line পরীক্ষা করে এবং এমন একটি Line নির্বাচন করে, যার Prediction Error সর্বনিম্ন হয়। এই Error পরিমাপ করার জন্য সাধারণত Cost Function ব্যবহার করা হয় এবং Cost কমানোর জন্য Gradient Descent-এর মতো Optimization Algorithm ব্যবহার করা হয়।

Training সম্পন্ন হওয়ার পর Model নতুন কোনো Input পেলে সেই Best Fit Line ব্যবহার করে Output Prediction করে।

Linear Regression-এর গাণিতিক সমীকরণ

গণিতে একটি সরলরেখার সমীকরণ সাধারণত লেখা হয়,

$$y = mx + c$$

Machine Learning-এ একই সমীকরণকে সাধারণত Weight এবং Bias ব্যবহার করে প্রকাশ করা হয়।

Simple Linear Regression-এর সমীকরণ হলো,

$$\hat{y} = wx + b$$

যেখানে,

- $\hat{y}$ = Predicted Value
- x = Input Feature
- w = Weight (Slope)
- b = Bias (Intercept)

যদি একাধিক Feature থাকে, তাহলে সমীকরণটি হবে,

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$$

এখানে,

- $w_1, w_2, \dots, w_n$ = প্রতিটি Feature-এর Weight
- $x_1, x_2, \dots, x_n$ = বিভিন্ন Input Feature
- b = Bias বা Intercept

Weight নির্ধারণ করে প্রতিটি Feature Prediction-কে কতটা প্রভাবিত করবে, আর Bias হলো সেই ধ্রুবক (Constant) মান, যা সব Prediction-এর সাথে যোগ হয়।

Example 1: House Price Prediction

ধরো নিচের Dataset রয়েছে।

House Size (sq ft)	Price (Lakh Tk)
1000	45
1200	55
1500	68
1800	82

Training-এর সময় Model বুঝতে শিখবে যে House Size বাড়ার সাথে সাথে Price-ও বাড়ছে।

ধরা যাক Training শেষে Model নিচের সমীকরণটি শিখেছে,

$$\hat{y} = 0.05x - 5$$

এখন যদি একটি বাড়ির আকার 1600 sq ft হয়, তাহলে

$$\hat{y} = 0.05 \times 1600 - 5$$

$$\hat{y} = 75$$

অর্থাৎ Model অনুমান করছে বাড়িটির মূল্য প্রায় 75 লক্ষ টাকা।

Example 2: Student Score Prediction

ধরো,

Study Hours	Exam Score
2	40
4	58
6	72
8	90

Training শেষে Model যদি শিখে,

$$\hat{y} = 8x + 25$$

তাহলে কোনো শিক্ষার্থী যদি 7 ঘন্টা পড়ে,

$$\hat{y} = 8 \times 7 + 25$$

$$\hat{y} = 81$$

অর্থাৎ সম্ভাব্য পরীক্ষার নম্বর হবে 81।

Linear Relationship কী?

যখন একটি Variable বাড়লে বা কমলে অন্য Variable-ও প্রায় একই অনুপাতে বাড়ে বা কমে, তখন তাকে Linear Relationship বলা হয়।

যদি Data Point-গুলো একটি Straight Line-এর কাছাকাছি অবস্থান করে, তাহলে Linear Regression ভালোভাবে কাজ করে।

Linear Regression-এর প্রধান উপাদান

- Independent Variable (Feature)
 - Dependent Variable (Target)
 - Weight
 - Bias
 - Best Fit Line
 - Prediction Error
-

Linear Regression-এর প্রকারভেদ

- Simple Linear Regression
- Multiple Linear Regression

Simple Linear Regression-এ একটি মাত্র Independent Variable থাকে।

Multiple Linear Regression-এ একাধিক Independent Variable থাকে।

এই দুটি বিষয় পরবর্তী অধ্যায়ে বিস্তারিত আলোচনা করা হবে।

Linear Regression-এর Assumptions

Linear Regression সঠিকভাবে কাজ করার জন্য সাধারণত নিচের বিষয়গুলো সত্য হওয়া উচিত।

- Input এবং Output-এর মধ্যে Linear Relationship থাকতে হবে।
 - Observation-গুলো একে অপরের থেকে স্বাধীন হতে হবে।
 - Error-এর Variance প্রায় সমান হতে হবে।
 - Error-এর Mean শূন্যের কাছাকাছি হওয়া উচিত।
 - Multiple Linear Regression-এর ক্ষেত্রে Feature-গুলোর মধ্যে অতিরিক্ত Multicollinearity থাকা উচিত নয়।
-

Linear Regression-এর ব্যবহার

- House Price Prediction
- Salary Prediction
- Sales Forecasting

- Temperature Prediction
 - Stock Price Trend Analysis
 - Medical Cost Prediction
 - Demand Forecasting
 - Energy Consumption Prediction
-

Advantages

- শেখা এবং বাস্তবায়ন করা সহজ।
 - Training দ্রুত সম্পন্ন হয়।
 - Prediction সহজে ব্যাখ্যা করা যায়।
 - ছোট Dataset-এর ক্ষেত্রেও কার্যকর।
 - Feature এবং Target-এর সম্পর্ক সহজে বোঝা যায়।
-

Disadvantages

- শুধুমাত্র Linear Relationship Model করতে পারে।
 - Outlier-এর কারণে ফলাফল প্রভাবিত হতে পারে।
 - Non-linear Data-এর ক্ষেত্রে ভালো কাজ করে না।
 - Assumption ভঙ্গ হলে Prediction-এর Accuracy কমে যেতে পারে।
-

Key Points

- Linear Regression একটি Supervised Learning-এর Regression Algorithm।
- এটি Numerical Value Prediction করার জন্য ব্যবহৃত হয়।
- Feature এবং Target-এর মধ্যে Linear Relationship নির্ণয় করে।
- Prediction-এর জন্য Best Fit Line ব্যবহার করা হয়।
- Model-এর মূল সমীকরণ হলো $\hat{y} = wx + b$ ।
- Multiple Linear Regression-এর ক্ষেত্রে সমীকরণটি হয় $\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ ।
- Weight Feature-এর প্রভাব নির্দেশ করে এবং Bias হলো ধ্রুবক মান।

Exam Note

Linear Regression হলো একটি Supervised Machine Learning Regression Algorithm, যা Feature এবং Target Variable-এর মধ্যে Linear Relationship শিখে Numerical Value Prediction করে। এটি একটি Best Fit Line তৈরি করে এবং Prediction-এর জন্য সাধারণ সমীকরণ $\hat{y} = wx + b$ ব্যবহার করে। এখানে w হলো Weight এবং b হলো Bias। একাধিক Feature থাকলে সমীকরণটি $\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ হয়। House Price Prediction, Salary Prediction, Sales Forecasting এবং Temperature Prediction-এর মতো সমস্যায় Linear Regression ব্যাপকভাবে ব্যবহৃত হয়।

Linear Models

আগের অধ্যায়ে আমরা Linear Regression সম্পর্কে জেনেছি। সেখানে একটি প্রশ্ন স্বাভাবিকভাবেই আসে, Linear Regression-এর নামের শুরুতে "Linear" কেন? এই "Linear" শব্দটি কোথা থেকে এসেছে?

এই প্রশ্নের উত্তরই হলো **Linear Model**।

Machine Learning-এ Linear Model বলতে এমন একটি Model-কে বোঝায়, যেখানে Prediction একটি Linear Equation-এর মাধ্যমে তৈরি হয়। এখানে "Linear" বলতে বোঝায় যে প্রতিটি Feature Prediction-এ একটি নির্দিষ্ট পরিমাণে অবদান রাখে এবং সেই অবদানগুলো যোগ করে চূড়ান্ত Prediction নির্ণয় করা হয়।

অর্থাৎ, যদি একটি Model-এর Prediction সমীকরণকে নিচের আকারে লেখা যায়,

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

তাহলে সেটি একটি Linear Model।

এখানে প্রতিটি Feature-এর জন্য একটি Weight থাকে। কোনো Feature Prediction-এর উপর বেশি প্রভাব ফেললে তার Weight-এর মান বড় হয়, আর কম প্রভাব ফেললে Weight-এর মান ছোট হয়। সবশেষে একটি Bias যোগ করা হয়, যা পুরো Prediction-কে উপরে বা নিচে সরিয়ে Data-এর সাথে আরও ভালোভাবে মিল করাতে সাহায্য করে।

এখানে একটি বিষয় লক্ষ্য করার মতো। অনেক শিক্ষার্থী মনে করে "Linear" মানে Graph অবশ্যই একটি সরলরেখা (Straight Line) হবে। এটি সম্পূর্ণ সঠিক নয়।

Machine Learning-এ "Linear" বলতে মূলত বোঝায় **Parameter-এর প্রতি Linear হওয়া**, অর্থাৎ Weight এবং Bias সমীকরণে প্রথম ঘাতে (Power 1) থাকবে এবং একে অপরের সাথে গুণ হবে না।

উদাহরণ হিসেবে নিচের সমীকরণটি একটি Linear Model।

$$\hat{y} = 3x + 5$$

আবার,

$$\hat{y} = 2x_1 + 4x_2 - 7x_3 + 10$$

এটিও একটি Linear Model।

কারণ এখানে প্রতিটি Weight শুধুমাত্র সংশ্লিষ্ট Feature-এর সাথে একবার করে গুণ হয়েছে এবং সবগুলো যোগ করা হয়েছে।

এখন একটি বাস্তব উদাহরণ দেখা যাক।

ধরো, একটি বাড়ির মূল্য তিনটি বিষয়ের উপর নির্ভর করে।

- House Size

- Number of Bedrooms
- House Age

Training শেষে Model নিচের সমীকরণটি শিখেছে,

$$\hat{y} = 0.05x_1 + 6x_2 - 0.3x_3 + 15$$

যদি,

- House Size = 1500 sq ft
- Bedrooms = 3
- House Age = 8 বছর

তাহলে,

$$\begin{aligned}\hat{y} &= 0.05(1500) + 6(3) - 0.3(8) + 15 \\ \hat{y} &= 75 + 18 - 2.4 + 15 \\ \hat{y} &= 105.6\end{aligned}$$

অর্থাৎ Model অনুমান করছে বাড়িটির মূল্য প্রায় 105.6 লক্ষ টাকা।

এখানে সমীকরণটি লক্ষ্য করলে দেখা যায়, প্রতিটি Feature আলাদাভাবে Prediction-এ অবদান রাখছে।

- House Size বাড়লে মূল্য বাড়ছে।
- Bedroom সংখ্যা বাড়লে মূল্য বাড়ছে।
- House Age বাড়লে মূল্য কিছুটা কমছে।

এই প্রভাবগুলো Weight-এর মাধ্যমে প্রকাশিত হয়েছে।

Machine Learning-এ অনেক জনপ্রিয় Algorithm Linear Model-এর ধারণা ব্যবহার করে।

- Linear Regression
- Logistic Regression
- Ridge Regression
- Lasso Regression
- Elastic Net

এদের প্রত্যেকের মূল ভিত্তি একই। সবাই প্রথমে একটি Linear Combination তৈরি করে। পরে Algorithm-এর ধরন অনুযায়ী সেই Linear Combination ব্যবহার করে Prediction তৈরি করা হয়।

এখানে একটি গুরুত্বপূর্ণ বিষয় রয়েছে।

অনেকেই মনে করেন Logistic Regression একটি Non-linear Algorithm, কারণ এটি Classification করে। কিন্তু বাস্তবে Logistic Regression-ও একটি Linear Model।

কারণ এটি প্রথমে একই Linear Equation ব্যবহার করে,

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

এরপর এই Linear Score-এর উপর একটি Sigmoid Function প্রয়োগ করে Probability বের করা হয়।

অর্থাৎ Classification করলেও এর ভিত্তি এখনো একটি Linear Model।

এ কারণেই Linear Model হলো Machine Learning-এর অন্যতম মৌলিক ধারণা। পরবর্তী অধ্যায়ে আমরা Hypothesis Function, Linear Regression Equation, Regression Coefficient, Cost Function এবং Gradient Descent সম্পর্কে জানব। তখন দেখা যাবে, এই প্রতিটি বিষয়ই মূলত Linear Model-এর সমীকরণকে আরও ভালোভাবে বোঝা এবং শেখানোর জন্য ব্যবহৃত হয়।

Key Points

- Linear Model একটি Linear Equation ব্যবহার করে Prediction তৈরি করে।
- এর সাধারণ সমীকরণ হলো $\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ ।
- প্রতিটি Feature-এর জন্য একটি পৃথক Weight থাকে।
- Bias Prediction-কে সামঞ্জস্য (Adjust) করতে সাহায্য করে।
- Linear Regression, Logistic Regression, Ridge Regression, Lasso Regression এবং Elastic Net সবই Linear Model-এর উদাহরণ।
- Linear Model হলো Machine Learning-এর বহু Algorithm-এর ভিত্তি।

Hypothesis Function

আগের অধ্যায়ে আমরা দেখেছি যে একটি Linear Model Prediction করার জন্য নিচের সমীকরণটি ব্যবহার করে।

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

কিন্তু Machine Learning-এ এই সমীকরণটিকে শুধু একটি Equation বলা হয় না। এটিকে একটি বিশেষ নাম দেওয়া হয়েছে, সেটি হলো **Hypothesis Function**।

Hypothesis Function হলো এমন একটি Mathematical Function, যা Input Feature গ্রহণ করে একটি Predicted Output প্রদান করে। অর্থাৎ, এটি Model-এর "অনুমান করার নিয়ম" (Prediction Rule)।

সহজভাবে বলতে গেলে, যখন কোনো নতুন Data Model-এর কাছে আসে, তখন Model প্রথমে Hypothesis Function ব্যবহার করে একটি সম্ভাব্য Output গণনা করে। এই Output-ই হলো Prediction।

Machine Learning-এর প্রায় প্রতিটি Supervised Learning Algorithm-এর একটি Hypothesis Function থাকে। Algorithm ভেদে এই Function-এর রূপ পরিবর্তিত হতে পারে। Linear Regression-এ এটি একটি Linear Function, আবার Logistic Regression-এ প্রথমে একটি Linear Function ব্যবহার করা হলেও পরে তার উপর Sigmoid Function প্রয়োগ করা হয়।

Linear Regression-এর Hypothesis Function

Linear Regression-এ Hypothesis Function লেখা হয়,

$$h(x) = wx + b$$

অথবা একাধিক Feature থাকলে,

$$h(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

অনেক বই এবং গবেষণাপত্রে একই সমীকরণ নিচেরভাবেও লেখা হয়,

$$h_{\theta}(x) = \theta^T x$$

এখানে,

- $h(x)$ = Hypothesis Function
- x = Input Feature
- w = Weight
- b = Bias

আমাদের নোটে আমরা Weight এবং Bias ব্যবহার করব, কারণ আধুনিক Machine Learning Library যেমন TensorFlow, PyTorch এবং Scikit-learn-এ এই নোটেশনই বেশি ব্যবহৃত হয়।

Hypothesis Function কী করে?

Hypothesis Function-এর একমাত্র কাজ হলো Input Feature ব্যবহার করে একটি Prediction তৈরি করা।

ধরো, একটি House Price Prediction Model-এর Hypothesis Function হলো,

$$h(x) = 0.05x - 5$$

যদি,

House Size = 1600 sq ft

তাহলে,

$$h(1600) = 0.05 \times 1600 - 5$$

$$h(1600) = 75$$

অর্থাৎ Model-এর Prediction হলো 75 লক্ষ টাকা।

এখানে 75 হলো প্রকৃত মূল্য (Actual Value) নয়, বরং Model-এর অনুমানকৃত মূল্য (Predicted Value)।

আরেকটি Example

ধরো, একজন শিক্ষার্থীর পড়ার সময়ের উপর ভিত্তি করে পরীক্ষার নম্বরের Prediction করা হবে।

Hypothesis Function,

$$h(x) = 8x + 20$$

যদি,

Study Hour = 7

তাহলে,

$$h(7) = 8 \times 7 + 20$$

$$h(7) = 76$$

অর্থাৎ Model অনুমান করছে শিক্ষার্থী 76 নম্বরের পেতে পারে।

Hypothesis Function কেন "Hypothesis" বলা হয়?

"Hypothesis" শব্দের অর্থ হলো একটি সম্ভাব্য ধারণা বা অনুমান।

Training-এর শুরুতে Model Weight এবং Bias-এর কিছু প্রাথমিক মান (Initial Value) নির্বাচন করে। তখন সেই সমীকরণটি কেবল একটি অনুমানমাত্র।

উদাহরণস্বরূপ,

প্রথমে Model ধরে নিল,

$$h(x) = 2x + 10$$

কিন্তু Training-এর সময় দেখা গেল এই সমীকরণটি সঠিকভাবে Prediction করতে পারছে না।

তখন Algorithm ধীরে ধীরে Weight এবং Bias পরিবর্তন করতে থাকে।

শেষ পর্যন্ত হয়তো Model শিখল,

$$h(x) = 5x + 2$$

এখন এই নতুন Hypothesis আগেরটির তুলনায় অনেক ভালো Prediction দেয়।

অর্থাৎ Training-এর পুরো সময়জুড়ে Model বারবার নিজের Hypothesis পরিবর্তন করে, যতক্ষণ না Prediction Error সর্বনিম্ন হয়।

Hypothesis Function এবং Prediction-এর সম্পর্ক

অনেক শিক্ষার্থী Hypothesis Function এবং Prediction-কে একই জিনিস মনে করে। কিন্তু এদের মধ্যে সামান্য পার্থক্য রয়েছে।

Hypothesis Function হলো Prediction করার Mathematical Rule।

Prediction হলো সেই Rule ব্যবহার করে পাওয়া Final Output।

উদাহরণ হিসেবে,

যদি,

$$h(x) = 4x + 6$$

এবং,

$$x = 5$$

তাহলে,

$$h(5) = 4 \times 5 + 6 = 26$$

এখানে,

- $h(x) = 4x + 6$ হলো Hypothesis Function।
- 26 হলো Prediction।

Hypothesis Function কীভাবে শেখে?

শুরুতে Weight এবং Bias এলোমেলো (Random) মান দিয়ে শুরু হয়।

এরপর প্রতিটি Prediction-এর Error গণনা করা হয়।

যদি Error বেশি হয়, তাহলে Algorithm Weight এবং Bias পরিবর্তন করে।

এই পরিবর্তনের কাজ সাধারণত Gradient Descent Algorithm-এর মাধ্যমে করা হয়।

বারবার এই প্রক্রিয়া চলতে থাকে যতক্ষণ না একটি ভালো Hypothesis পাওয়া যায়, যা Data-এর সাথে সবচেয়ে ভালোভাবে মিলে যায়।

এই কারণেই Hypothesis Function, Cost Function এবং Gradient Descent একে অপরের সাথে সরাসরি সম্পর্কিত। পরবর্তী অধ্যায়গুলোতে এই সম্পর্ক আরও পরিষ্কার হবে।

বাস্তব জীবনের উদাহরণ

ধরো, একজন শিক্ষক ভাবলেন,

"যে শিক্ষার্থী প্রতিদিন ১ ঘণ্টা বেশি পড়বে, সে ৮ নম্বর বেশি পাবে।"

এটি একটি অনুমান।

গাণিতিকভাবে,

$$h(x) = 8x + 20$$

কিন্তু পরীক্ষার ফলাফল দেখার পরে শিক্ষক বুঝলেন, আসলে প্রতি ঘণ্টায় ৬ নম্বর করে বাড়ছে।

তখন নতুন Hypothesis হবে,

$$h(x) = 6x + 22$$

অর্থাৎ বাস্তব Data দেখে Hypothesis পরিবর্তন করা হয়েছে।

Machine Learning-এও ঠিক একই কাজ হয়।

Key Points

- Hypothesis Function হলো Model-এর Prediction করার Mathematical Function।
- Linear Regression-এ Hypothesis Function হলো $h(x) = wx + b$ ।
- Multiple Feature-এর ক্ষেত্রে $h(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ ।
- Hypothesis Function Input গ্রহণ করে Predicted Output তৈরি করে।
- Training-এর সময় Weight এবং Bias পরিবর্তন করে আরও ভালো Hypothesis শেখা হয়।
- Cost Function Hypothesis-এর Error পরিমাপ করে এবং Gradient Descent সেই Error কমানোর জন্য Hypothesis-এর Weight ও Bias আপডেট করে।

Linear Regression Equation

Linear Regression-এ একটি সরলরেখার মাধ্যমে Input Feature এবং Target Variable-এর মধ্যে সম্পর্ক প্রকাশ করা হয়। এই গাণিতিক সমীকরণকে **Linear Regression Equation** বলা হয়।

Simple Linear Regression-এর Equation হলো,

$$\hat{y} = wx + b$$

Multiple Linear Regression-এর Equation হলো,

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Terms

- $\hat{y}$ = Predicted Value
- x = Input Feature
- w = Weight বা Regression Coefficient
- b = Bias (Intercept)
- $x_1, x_2, \dots, x_n$ = একাধিক Feature
- $w_1, w_2, \dots, w_n$ = প্রতিটি Feature-এর Weight

Linear Regression-এর Training-এর মূল উদ্দেশ্য হলো এমন একটি w এবং b খুঁজে বের করা, যাতে Prediction বাস্তব মানের (Actual Value) যতটা সম্ভব কাছাকাছি হয়।

ধরো Model-এর Equation,

$$\hat{y} = 4x + 10$$

যদি,

$$x = 8$$

তাহলে,

$$\hat{y} = 4(8) + 10 = 42$$

অর্থাৎ Prediction হবে 42।

Regression Coefficient

Regression Coefficient হলো Linear Regression Equation-এর সবচেয়ে গুরুত্বপূর্ণ Parameter। এটি নির্দেশ করে একটি Feature এক ইউনিট পরিবর্তিত হলে Prediction গড়ে কত ইউনিট পরিবর্তিত হবে।

Simple Linear Regression-এ Regression Coefficient হলো,

$$w = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

এখানে,

- w = Regression Coefficient (Weight)
- x_i = প্রতিটি Input Feature
- y_i = প্রতিটি Actual Target Value
- $\bar{x}$ = সকল x -এর Mean
- $\bar{y}$ = সকল y -এর Mean
- n = মোট Observation সংখ্যা

Regression Coefficient নির্ণয় করার পর Bias নির্ণয় করা হয়।

Bias-এর Formula হলো,

$$b = \bar{y} - w\bar{x}$$

এখানে,

- b = Bias (Intercept)
- $\bar{x}$ = Input-এর Mean
- $\bar{y}$ = Target-এর Mean
- w = Regression Coefficient

অর্থাৎ, প্রথমে Regression Coefficient (w) নির্ণয় করা হয়, তারপর সেই মান ব্যবহার করে Bias (b) বের করা হয়।

সবশেষে এই দুটি মান মূল Equation-এ বসানো হয়,

$$\hat{y} = wx + b$$

এবং এটিই Final Linear Regression Model।

Example

ধরো,

$$w = 3$$

এবং,

$$b = 5$$

তাহলে Model-এর Equation হবে,

$$\hat{y} = 3x + 5$$

যদি,

$$x = 10$$

তাহলে,

$$\hat{y} = 3(10) + 5 = 35$$

অর্থাৎ Model-এর Prediction হবে 35।

নোট: এই w এবং b -এর Formula-গুলো **Ordinary Least Squares (OLS)** Method থেকে আসে। তাই যখন **OLS** টপিকে যাব, তখন আমরা ধাপে ধাপে প্রমাণ (Derivation) করব কেন

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

এবং

$$b = \bar{y} - w\bar{x}$$

Properties of Linear Regression

Linear Regression একটি Mathematical Model, যা Input Feature এবং Target Variable-এর মধ্যে একটি Linear Relationship স্থাপন করে। এই Model-এর কিছু গুরুত্বপূর্ণ বৈশিষ্ট্য (Properties) রয়েছে, যা এর কার্যপদ্ধতি এবং ব্যবহার বোঝার জন্য অত্যন্ত গুরুত্বপূর্ণ।

প্রথমত, Linear Regression ধরে নেয় যে Input Feature এবং Target Variable-এর মধ্যে একটি **Linear Relationship** রয়েছে। অর্থাৎ, Feature পরিবর্তিত হলে Target-ও একটি নির্দিষ্ট হারে পরিবর্তিত হবে।

দ্বিতীয়ত, এটি একটি **Supervised Learning Algorithm**। অর্থাৎ, Training-এর সময় Model-কে Input এবং তার সঠিক Output (Label) উভয়ই দেওয়া হয়।

তৃতীয়ত, এটি একটি **Regression Algorithm**, তাই এর Output সবসময় একটি Continuous Numerical Value হয়। যেমন, House Price, Salary, Temperature, Sales ইত্যাদি।

Linear Regression-এর Prediction একটি **Linear Equation**-এর মাধ্যমে করা হয়।

$$\hat{y} = wx + b$$

অথবা,

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Training-এর সময় Model-এর প্রধান কাজ হলো এমন একটি **Best Fit Line** খুঁজে বের করা, যা Data Point-গুলোর সবচেয়ে কাছ দিয়ে যায় এবং Prediction Error সর্বনিম্ন করে।

Linear Regression-এর আরেকটি গুরুত্বপূর্ণ বৈশিষ্ট্য হলো এটি **Ordinary Least Squares (OLS)** নীতি অনুসরণ করে। অর্থাৎ, এটি এমন একটি Equation নির্বাচন করে, যার Squared Error-এর যোগফল সর্বনিম্ন হয়। পরবর্তী অধ্যায়ে OLS বিস্তারিত আলোচনা করা হবে।

এছাড়া Model-এর Weight এবং Bias স্থির থাকে না। Training-এর সময় **Gradient Descent**-এর মাধ্যমে এগুলো বারবার Update করা হয় যতক্ষণ না Cost Function সর্বনিম্ন হয়।

Linear Regression-এর ফলাফল সহজে ব্যাখ্যা করা যায়। প্রতিটি Weight দেখে বোঝা যায়, কোনো Feature Prediction-এর উপর কতটা প্রভাব ফেলেছে। এজন্য এটিকে একটি **Interpretable Model** বলা হয়।

Properties of Linear Regression

- এটি একটি Supervised Learning Algorithm।
- এটি একটি Regression Algorithm।
- Continuous Numerical Value Prediction করে।
- Linear Relationship ধরে কাজ করে।

- Prediction-এর জন্য Linear Equation ব্যবহার করে।
 - Best Fit Line তৈরি করে।
 - OLS Principle অনুসরণ করে।
 - Cost Function কমিয়ে শেখে।
 - Gradient Descent ব্যবহার করে Weight ও Bias Update করা যায়।
 - Model-এর ফলাফল সহজে ব্যাখ্যা করা যায় (Highly Interpretable)।
-

Simple Linear Regression

যখন একটি মাত্র Independent Variable ব্যবহার করে একটি Dependent Variable Prediction করা হয়, তখন তাকে **Simple Linear Regression** বলা হয়।

অর্থাৎ, এখানে শুধুমাত্র একটি Input Feature এবং একটি Output থাকে।

এর Mathematical Equation হলো,

$$\hat{y} = wx + b$$

যেখানে,

- $\hat{y}$ = Predicted Value
- x = Independent Variable (Input Feature)
- w = Regression Coefficient (Weight)
- b = Bias (Intercept)

Simple Linear Regression-এর Graph সাধারণত একটি সরলরেখা (Straight Line) হয়।

ধরো, একজন শিক্ষার্থীর পড়ার সময় (Study Hour) এবং পরীক্ষার নম্বর (Exam Score)-এর মধ্যে সম্পর্ক নির্ণয় করতে হবে।

Training শেষে Model নিচের Equation শিখেছে,

$$\hat{y} = 8x + 20$$

যদি,

Study Hour = 6 ঘন্টা

তাহলে,

$$\hat{y} = 8(6) + 20 = 68$$

অর্থাৎ Model অনুমান করছে শিক্ষার্থী 68 নম্বর পাবে।

Simple Linear Regression সাধারণত তখন ব্যবহার করা হয়, যখন একটি মাত্র Feature দিয়েই Target ভালোভাবে Prediction করা সম্ভব।

Example

- Study Hour → Exam Score
- House Size → House Price
- Experience → Salary
- Temperature → Ice Cream Sales

Key Points

- একটি মাত্র Independent Variable থাকে।
- একটি Dependent Variable থাকে।
- Equation হলো $\hat{y} = wx + b$ ।
- Graph একটি Straight Line হয়।
- বোঝা ও বাস্তবায়ন করা সহজ।

Multiple Linear Regression

যখন দুই বা ততোধিক Independent Variable ব্যবহার করে একটি Dependent Variable Prediction করা হয়, তখন তাকে **Multiple Linear Regression** বলা হয়।

বাস্তব জীবনের অধিকাংশ সমস্যা একাধিক Feature-এর উপর নির্ভর করে। তাই বাস্তবে Multiple Linear Regression-এর ব্যবহার Simple Linear Regression-এর তুলনায় অনেক বেশি।

এর Mathematical Equation হলো,

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

যেখানে,

- $\hat{y}$ = Predicted Value
- $x_1, x_2, \dots, x_n$ = বিভিন্ন Independent Variable

- $w_1, w_2, \dots, w_n$ = প্রতিটি Feature-এর Regression Coefficient
- b = Bias (Intercept)

ধরো, একটি House Price Prediction Model তিনটি Feature ব্যবহার করছে।

- House Size
- Number of Bedrooms
- House Age

Training শেষে Model-এর Equation হলো,

$$\hat{y} = 0.05x_1 + 6x_2 - 0.4x_3 + 12$$

যদি,

- House Size = 1600 sq ft
- Bedrooms = 3
- House Age = 5 বছর

তাহলে,

$$\begin{aligned}\hat{y} &= 0.05(1600) + 6(3) - 0.4(5) + 12 \\ \hat{y} &= 80 + 18 - 2 + 12 \\ \hat{y} &= 108\end{aligned}$$

অর্থাৎ Model বাড়িটির সম্ভাব্য মূল্য 108 লক্ষ টাকা Prediction করছে।

Multiple Linear Regression-এ প্রতিটি Feature আলাদাভাবে Prediction-এ অবদান রাখে। কোনো Feature-এর Weight Positive হলে Prediction বাড়ায়, আর Negative হলে Prediction কমায়।

Example

- House Size + Bedrooms + Age $\rightarrow$ House Price
- Age + Experience + Education $\rightarrow$ Salary
- Advertising Cost + Product Price + Season $\rightarrow$ Sales
- Rainfall + Temperature + Fertilizer $\rightarrow$ Crop Yield

Key Points

- দুই বা ততোধিক Independent Variable থাকে।

- একটি Dependent Variable থাকে।
- Equation হলো $\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ ।
- বাস্তব জীবনের অধিকাংশ Prediction Problem-এ ব্যবহৃত হয়।
- প্রতিটি Feature-এর জন্য পৃথক Regression Coefficient থাকে।

Best Fit Line

Linear Regression-এর মূল লক্ষ্য হলো এমন একটি সরলরেখা (Straight Line) খুঁজে বের করা, যা Training Data-এর সাথে সবচেয়ে ভালোভাবে মিল খায়। এই রেখাকেই **Best Fit Line** বলা হয়।

বাস্তবে কোনো Dataset-এর সমস্ত Data Point কখনোই একটি সরলরেখার উপর অবস্থান করে না। কিছু Point রেখার উপরে থাকে, আবার কিছু নিচে থাকে। তাই এমন একটি রেখা নির্বাচন করতে হয়, যার থেকে Data Point-গুলোর মোট দূরত্ব বা Error যতটা সম্ভব কম হয়।

Best Fit Line-ই হলো সেই রেখা, যা Data-এর Overall Trend সবচেয়ে ভালোভাবে প্রকাশ করে এবং নতুন Data-এর জন্য সবচেয়ে ভালো Prediction দিতে সক্ষম হয়।

Best Fit Line-এর Equation হলো,

$$\hat{y} = wx + b$$

যেখানে,

- $\hat{y}$ = Predicted Value
- x = Input Feature
- w = Slope (Regression Coefficient)
- b = Bias (Intercept)

Training-এর সময় Model বিভিন্ন সম্ভাব্য Line পরীক্ষা করে। প্রতিটি Line-এর জন্য Prediction Error গণনা করা হয়। যেই Line-এর মোট Error সবচেয়ে কম হয়, সেটিকেই Best Fit Line হিসেবে নির্বাচন করা হয়।

ধরো নিচের Dataset রয়েছে।

Study Hour Exam Score

2	40
4	55
6	68
8	85

এই Point-গুলোর উপর অনেকগুলো Line আঁকা সম্ভব। কিন্তু সবগুলো সমান ভালো নয়। যে Line-এর Prediction প্রতিটি Point-এর সবচেয়ে কাছাকাছি হবে, সেটিই Best Fit Line।

Best Fit Line নির্বাচন করার ক্ষেত্রে সবচেয়ে গুরুত্বপূর্ণ ভূমিকা পালন করে **Ordinary Least Squares (OLS) Method**। OLS এমন একটি Line নির্বাচন করে, যার Squared Error-এর যোগফল সর্বনিম্ন হয়।

Key Points

- Best Fit Line হলো Data-এর সাথে সবচেয়ে ভালোভাবে মিল থাকা সরলরেখা।
 - এটি Prediction Error সর্বনিম্ন করে।
 - Linear Regression-এর Final Model-ই হলো Best Fit Line।
 - OLS Method ব্যবহার করে Best Fit Line নির্ণয় করা হয়।
-

Ordinary Least Squares (OLS)

Best Fit Line কীভাবে নির্বাচন করা হয়?

এই প্রশ্নের উত্তর হলো **Ordinary Least Squares (OLS)**।

OLS হলো Linear Regression-এর সবচেয়ে জনপ্রিয় Estimation Method। এর কাজ হলো এমন একটি Weight (w) এবং Bias (b) নির্ণয় করা, যাতে Prediction Error-এর **Squared Sum** সর্বনিম্ন হয়।

প্রথমে প্রতিটি Data Point-এর জন্য Error নির্ণয় করা হয়।

$$Error = y - \hat{y}$$

যেখানে,

- y = Actual Value
- $\hat{y}$ = Predicted Value

কিছু Error Positive এবং কিছু Negative হতে পারে। যদি এগুলো সরাসরি যোগ করা হয়, তাহলে অনেক Error একে অপরকে বাতিল করে দিতে পারে। তাই প্রতিটি Error-এর Square করা হয়।

$$(y - \hat{y})^2$$

সবশেষে সবগুলো Squared Error যোগ করা হয়।

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

OLS-এর মূল লক্ষ্য হলো,

$$\min \sum (y_i - \hat{y}_i)^2$$

অর্থাৎ এমন একটি Line খুঁজে বের করা, যার Squared Error-এর যোগফল সর্বনিম্ন।

OLS Method ব্যবহার করলে Simple Linear Regression-এর Weight এবং Bias-এর Closed-form Formula পাওয়া যায়।

Regression Coefficient,

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

Bias,

$$b = \bar{y} - w\bar{x}$$

যেখানে,

- x_i = প্রতিটি Input
- y_i = প্রতিটি Actual Output
- $\bar{x}$ = Mean of x
- $\bar{y}$ = Mean of y

ধরো,

Actual Marks = 70

Prediction = 65

তাহলে,

$$Error = 70 - 65 = 5$$

Squared Error,

$$5^2 = 25$$

এভাবেই প্রতিটি Data Point-এর Squared Error যোগ করে OLS সর্বনিম্ন করার চেষ্টা করে।

Key Points

- OLS = Ordinary Least Squares।
- এটি Best Fit Line নির্ণয়ের Method।
- Error-এর Square ব্যবহার করা হয়।

- Squared Error-এর যোগফল সর্বনিম্ন করা হয়।
 - OLS থেকে w এবং b -এর Closed-form Formula পাওয়া যায়।
-

Cost Function

Best Fit Line নির্ণয় করার জন্য শুধু Error জানা যথেষ্ট নয়। এমন একটি Function প্রয়োজন, যা পুরো Model-এর মোট Error একটি মাত্র সংখ্যার মাধ্যমে প্রকাশ করতে পারে। এই Function-কে **Cost Function** বলা হয়।

Cost Function হলো এমন একটি Mathematical Function, যা Model-এর Prediction কতটা ভুল হচ্ছে তা পরিমাপ করে।

Cost Function-এর মান যত কম হবে, Model-এর Prediction তত ভালো হবে।

Linear Regression-এ সবচেয়ে বেশি ব্যবহৃত Cost Function হলো **Mean Squared Error (MSE)**।

এর Formula হলো,

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

এখানে,

- $J(w, b)$ = Cost Function
- m = মোট Training Example-এর সংখ্যা
- y_i = Actual Value
- $\hat{y}_i$ = Predicted Value
- $(y_i - \hat{y}_i)^2$ = Squared Error

অনেক বইয়ে Formula-টি নিচেরভাবেও লেখা হয়।

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Machine Learning-এ Gradient Descent ব্যবহার করার সুবিধার জন্য Cost Function-এ সাধারণত $\frac{1}{2m}$ ব্যবহার করা হয়। এখানে 2 ব্যবহারের কারণ হলো Derivative নেওয়ার সময় Square-এর 2 কেটে যায়, ফলে হিসাব সহজ হয়।

ধরো,

Actual Prediction

40 42

60 58

80 83

প্রথমে Error,

- $40 - 42 = -2$
- $60 - 58 = 2$
- $80 - 83 = -3$

Squared Error,

- 4
- 4
- 9

মোট Squared Error,

$$SSE = 4 + 4 + 9 = 17$$

Mean Squared Error,

$$MSE = \frac{17}{3} = 5.67$$

Gradient Descent-এর কাজ হলো Weight এবং Bias বারবার পরিবর্তন করে এই Cost Function-এর মান যতটা সম্ভব কমিয়ে আনা।

Training শেষ হলে Cost Function সর্বনিম্ন হয় এবং তখনই Model Best Fit Line পেয়ে যায়।

Key Points

- Cost Function পুরো Model-এর Error পরিমাপ করে।
- Linear Regression-এ সাধারণত Mean Squared Error (MSE) ব্যবহার করা হয়।
- Cost Function-এর Formula,

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

- Cost যত কম, Model তত ভালো।
- Gradient Descent Cost Function সর্বনিম্ন করার জন্য Weight এবং Bias Update করে।

Example 1: Simple Linear Regression (Using OLS Formula)

ধরা যাক, একজন শিক্ষার্থীর পড়ার সময় (Study Hours) এবং পরীক্ষার নম্বর (Marks) নিচের মতো।

Study Hours (x)	Marks (y)
2	50
4	60
6	70
8	80

প্রথমে Mean বের করি।

$$\bar{x} = \frac{2 + 4 + 6 + 8}{4} = 5$$
$$\bar{y} = \frac{50 + 60 + 70 + 80}{4} = 65$$

Formula 1 (Mean-based Formula)

Regression Coefficient,

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

এখন টেবিল তৈরি করি।

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
2	50	-3	-15	45	9
4	60	-1	-5	5	1
6	70	1	5	5	1
8	80	3	15	45	9

$$\sum (x_i - \bar{x})(y_i - \bar{y}) = 100$$
$$\sum (x_i - \bar{x})^2 = 20$$

অতএব,

$$w = \frac{100}{20} = 5$$

Formula 2 (Shortcut Formula)

একই w নিচের Formula দিয়েও বের করা যায়।

$$w = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2}$$

এখন,

$$\begin{aligned} n &= 4 \\ \sum x &= 20 \\ \sum y &= 260 \\ \sum xy &= 2(50) + 4(60) + 6(70) + 8(80) = 1400 \\ \sum x^2 &= 2^2 + 4^2 + 6^2 + 8^2 = 120 \end{aligned}$$

তাহলে,

$$\begin{aligned} w &= \frac{4(1400) - 20(260)}{4(120) - 20^2} \\ &= \frac{5600 - 5200}{480 - 400} \\ &= \frac{400}{80} \\ \boxed{w} &= \boxed{5} \end{aligned}$$

দেখা যাচ্ছে, দুইটি Formula ব্যবহার করেই একই Weight পাওয়া গেছে।

Bias (Intercept)

Bias-এর Formula,

$$\begin{aligned} \boxed{b} &= \boxed{\bar{y} - w\bar{x}} \\ &= 65 - (5 \times 5) \\ &= 65 - 25 \\ \boxed{b} &= \boxed{40} \end{aligned}$$

Final Linear Regression Equation

$$\hat{y} = 5x + 40$$

যদি,

$$x = 5$$

তাহলে,

$$\hat{y} = 5(5) + 40 = 65$$

অর্থাৎ ৫ ঘণ্টা পড়লে Model ৬৫ নম্বর Prediction করছে।

Example 2: Multiple Linear Regression

ধরা যাক, একটি House Price Dataset রয়েছে।

House Size (x_1)	Bedrooms (x_2)	Price (y)
1200	2	60
1500	3	78
1800	3	90
2000	4	105

Multiple Linear Regression-এর Equation হলো,

$$\hat{y} = w_1x_1 + w_2x_2 + b$$

এখানে,

- w_1 = House Size-এর Weight
- w_2 = Bedroom-এর Weight
- b = Bias

গুরুত্বপূর্ণ বিষয়:

Simple Linear Regression-এর মতো Multiple Linear Regression-এ একটি মাত্র Formula ব্যবহার করে w_1 এবং w_2 বের করা যায় না।

কারণ এখানে একাধিক Unknown Parameter থাকে।

তাই Multiple Linear Regression-এ Matrix Equation ব্যবহার করা হয়।

$$w = (X^T X)^{-1} X^T y$$

যেখানে,

- X = Feature Matrix
- X^T = Transpose of Feature Matrix
- $(X^T X)^{-1}$ = Inverse Matrix
- y = Target Vector
- w = সকল Weight-এর Vector

Weight নির্ণয় করার পর Final Equation পাওয়া যায়। ধরো Training শেষে Model শিখেছে,

$$\hat{y} = 0.04x_1 + 8x_2 - 4$$

যদি,

- House Size = 1600
- Bedrooms = 3

তাহলে,

$$\begin{aligned}\hat{y} &= 0.04(1600) + 8(3) - 4 \\ &= 64 + 24 - 4 \\ \hat{y} &= 84\end{aligned}$$

অর্থাৎ Model বাড়িটির সম্ভাব্য মূল্য ৮৪ লক্ষ টাকা Prediction করছে।

Gradient Descent

আগের অধ্যায়ে আমরা দেখেছি, Linear Regression-এর লক্ষ্য হলো এমন একটি Best Fit Line খুঁজে বের করা যার Cost Function সর্বনিম্ন হয়। কিন্তু প্রশ্ন হলো, Model কীভাবে সঠিক Weight (w) এবং Bias (b) খুঁজে বের করে?

এই কাজটি করে **Gradient Descent**।

Gradient Descent হলো একটি **Optimization Algorithm**, যার কাজ হলো বারবার Weight এবং Bias পরিবর্তন করে Cost Function-এর মান কমানো। অর্থাৎ, এটি এমন Parameter খুঁজে বের করে যার জন্য Prediction Error সর্বনিম্ন হয়।

ধরো, একটি পাহাড়ের চূড়ায় একটি বল রাখা হয়েছে। বলটি সবসময় নিচের দিকে গড়িয়ে এমন স্থানে যায় যেখানে উচ্চতা সবচেয়ে কম। Gradient Descent-ও ঠিক একইভাবে Cost Function-এর সর্বনিম্ন মান (Minimum Point) খুঁজে বের করে।

ধরা যাক, শুরুতে Model-এর Equation হলো,

$$\hat{y} = 2x + 15$$

কিন্তু এই Equation দিয়ে Prediction করলে Cost Function-এর মান অনেক বেশি আসে।

Gradient Descent তখন w এবং b -এর মান একটু পরিবর্তন করে।

$$\hat{y} = 3x + 12$$

আবার Cost গণনা করা হয়।

যদি Cost কমে যায়, তাহলে আবার Parameter পরিবর্তন করা হয়।

$$\hat{y} = 4.2x + 9$$

এভাবে বারবার Update হতে হতে একসময় এমন একটি Equation পাওয়া যায়, যার Cost সবচেয়ে কম।

Gradient Descent-এর Update Rule হলো,

Weight Update,

$$w := w - \alpha \frac{\partial J(w, b)}{\partial w}$$

Bias Update,

$$b := b - \alpha \frac{\partial J(w, b)}{\partial b}$$

Terms

- w = Weight
- b = Bias
- α = Learning Rate
- $J(w, b)$ = Cost Function
- $\frac{\partial J}{\partial w}$ = Cost-এর Weight অনুযায়ী Gradient
- $\frac{\partial J}{\partial b}$ = Cost-এর Bias অনুযায়ী Gradient

Gradient নির্দেশ করে কোন দিকে এবং কত দ্রুত Cost বৃদ্ধি পাচ্ছে।

Gradient যদি Positive হয়, তাহলে Weight কমানো হয়।

Gradient যদি Negative হয়, তাহলে Weight বাড়ানো হয়।

এইভাবেই প্রতিবার Update-এর মাধ্যমে Model Minimum Cost-এর দিকে এগিয়ে যায়।

Example

ধরো,

প্রথম Iteration-এ,

- $w = 2$
- $b = 10$
- Learning Rate = 0.1

এবং,

$$\frac{\partial J}{\partial w} = 8$$

তাহলে,

$$w = 2 - 0.1(8)$$
$$w = 1.2$$

আবার যদি,

$$\frac{\partial J}{\partial b} = 5$$

তাহলে,

$$b = 10 - 0.1(5)$$
$$b = 9.5$$

এভাবে প্রতিটি Iteration-এ নতুন Weight এবং Bias পাওয়া যায়।

Training তখনই শেষ হয়, যখন Cost আর উল্লেখযোগ্যভাবে কমে না।

Key Points

- Gradient Descent একটি Optimization Algorithm।
- এটি Cost Function কমানোর জন্য ব্যবহৃত হয়।
- প্রতিটি Iteration-এ Weight ও Bias Update হয়।
- Learning Rate Update-এর Step Size নির্ধারণ করে।
- লক্ষ্য হলো Global Minimum-এর যতটা সম্ভব কাছাকাছি পৌঁছানো।

Gradient Descent for Multiple Variables

Simple Linear Regression-এ একটি মাত্র Weight ছিল। কিন্তু Multiple Linear Regression-এ প্রতিটি Feature-এর জন্য আলাদা Weight থাকে।

Equation,

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

এখন শুধুমাত্র একটি Weight Update করলেই হবে না। প্রতিটি Weight এবং Bias আলাদাভাবে Update করতে হবে।

ধরো House Price Prediction-এর তিনটি Feature রয়েছে।

- House Size
- Number of Bedrooms
- House Age

তাহলে Model হবে,

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Gradient Descent এখন চারটি Parameter Update করবে।

Weight Update,

$$w_j := w_j - \alpha \frac{\partial J}{\partial w_j}$$

যেখানে,

$$j = 1, 2, \dots, n$$

Bias Update,

$$b := b - \alpha \frac{\partial J}{\partial b}$$

Terms

- w_j = j -th Feature-এর Weight
 - x_j = j -th Feature
 - n = মোট Feature-এর সংখ্যা
 - α = Learning Rate
 - J = Cost Function
-

ধরো,

বর্তমান Parameter,

- $w_1 = 0.5$
- $w_2 = 2$
- $b = 5$

Gradient,

$$\frac{\partial J}{\partial w_1} = 4$$

$$\frac{\partial J}{\partial w_2} = 1$$
$$\frac{\partial J}{\partial b} = 3$$

Learning Rate,

$$\alpha = 0.1$$

তাহলে,

$$w_1 = 0.5 - 0.1(4) = 0.1$$
$$w_2 = 2 - 0.1(1) = 1.9$$
$$b = 5 - 0.1(3) = 4.7$$

এইভাবে প্রতিটি Iteration-এ সবগুলো Weight এবং Bias একসাথে Update হয়।

Key Points

- Multiple Linear Regression-এ প্রতিটি Feature-এর জন্য আলাদা Weight থাকে।
 - Gradient Descent প্রতিটি Weight পৃথকভাবে Update করে।
 - Bias-ও প্রতিবার Update হয়।
 - সব Parameter Update হওয়ার পর নতুন Cost গণনা করা হয়।
-

Types of Gradient Descent

Gradient Descent-এর প্রধান পার্থক্য হলো, **একবারে কতগুলো Training Sample ব্যবহার করে Gradient গণনা করা হয়।** এই ভিত্তিতে Gradient Descent তিন ভাগে বিভক্ত।

1. Batch Gradient Descent

সম্পূর্ণ Training Dataset একবারে ব্যবহার করে Gradient গণনা করা হয়।

যদি Dataset-এ 10,000 Sample থাকে, তাহলে প্রতিটি Update-এর আগে 10,000 Sample-ই ব্যবহার করা হবে।

Advantages

- Stable Update।
- Smooth Convergence।
- Accurate Gradient।

Disadvantages

- বড় Dataset-এ ধীরগতির।
 - Memory বেশি লাগে।
-

2. Stochastic Gradient Descent (SGD)

প্রতিবার মাত্র **একটি Training Sample** ব্যবহার করে Gradient গণনা করা হয়।

অর্থাৎ, প্রতিটি Sample-এর পরে Weight Update হয়।

ধরো Dataset-এ 1000 Sample আছে।

তাহলে 1000 বার Update হবে।

Advantages

- খুব দ্রুত Update হয়।
- বড় Dataset-এর জন্য উপযোগী।
- Local Minimum থেকে বের হওয়ার সম্ভাবনা বেশি।

Disadvantages

- Update অনেক Noise তৈরি করে।
 - Convergence স্থিতিশীল নয়।
-

3. Mini Batch Gradient Descent

এটি Batch এবং SGD-এর সমন্বয়।

এখানে পুরো Dataset-কে ছোট ছোট Batch-এ ভাগ করা হয়।

যেমন,

Batch Size = 32

তাহলে প্রতিবার 32টি Sample ব্যবহার করে Gradient গণনা করা হবে।

বর্তমানে Deep Learning-এ সবচেয়ে বেশি ব্যবহৃত Gradient Descent হলো Mini Batch Gradient Descent।

Advantages

- দ্রুত Training।
- Stable Convergence।

- Memory কম লাগে।
- GPU-এর সাথে ভালোভাবে কাজ করে।

Disadvantages

- Batch Size সঠিকভাবে নির্বাচন করতে হয়।

Comparison

Feature	Batch GD	SGD	Mini Batch GD
প্রতি Update-এ Data পুরো Dataset ১টি Sample ছোট Batch			
Training Speed	ধীর	দ্রুত	মাঝারি থেকে দ্রুত
Memory Usage	বেশি	কম	মাঝারি
Update Stability	খুব স্থিতিশীল	অস্থিতিশীল	স্থিতিশীল
বর্তমান ব্যবহার	কম	মাঝারি	সর্বাধিক

Key Points

- Gradient Descent তিন ধরনের।
- Batch Gradient Descent পুরো Dataset ব্যবহার করে।
- SGD একবারে একটি Sample ব্যবহার করে।
- Mini Batch Gradient Descent ছোট Batch ব্যবহার করে।
- আধুনিক Machine Learning এবং Deep Learning-এ Mini Batch Gradient Descent সবচেয়ে বেশি ব্যবহৃত হয়।

Gradient Descent Step-by-Step Example

ধরো আমাদের Dataset হলো,

Study Hour (x)	Marks (y)
1	3
2	5
3	7

এখানে আমরা একটি Linear Regression Model তৈরি করব।

Model Equation,

$$\hat{y} = wx + b$$

Step 1: Initialize Parameters

প্রথমে Model কিছু Random Value দিয়ে শুরু করে।

$$w = 0$$

$$b = 0$$

Learning Rate,

$$\alpha = 0.1$$

Number of Samples,

$$m = 3$$

Step 2: Cost Function

$$J(w, b) = \frac{1}{2m} \sum (y - \hat{y})^2$$

Step 3: Gradient Formula

Weight Gradient,

$$\frac{\partial J}{\partial w} = -\frac{1}{m} \sum x (y - \hat{y})$$

Bias Gradient,

$$\frac{\partial J}{\partial b} = -\frac{1}{m} \sum (y - \hat{y})$$

Iteration 1

বর্তমান Parameter,

$$w = 0, b = 0$$

Prediction

$$\hat{y} = wx + b$$

x	y	Prediction
1	3	0
2	5	0
3	7	0

Error

$$Error = y - \hat{y}$$

x	y	Prediction	Error
1	3	0	3
2	5	0	5
3	7	0	7

Squared Error

Error	Error ²
3	9
5	25
7	49

$$SSE = 9 + 25 + 49 = 83$$

Cost,

$$J = \frac{83}{2 \times 3}$$
$$\boxed{J = 13.83}$$

Calculate Gradient of w

$$\boxed{\frac{\partial J}{\partial w} = -\frac{1}{m} \sum x_i (y_i - \hat{y}_i)}$$

$$-\frac{1}{3} [(1)(3) + (2)(5) + (3)(7)]$$
$$= -\frac{1}{3} (3 + 10 + 21)$$
$$= -\frac{34}{3}$$

$$\boxed{\frac{\partial J}{\partial w} = -11.33}$$

একইভাবে Bias-এর জন্য,

$$\boxed{\frac{\partial J}{\partial b} = -\frac{1}{m} \sum (y_i - \hat{y}_i)}$$

$$-\frac{1}{3} (3 + 5 + 7)$$
$$= -\frac{15}{3}$$

$$\frac{\partial J}{\partial b} = -5$$

Update Weight

$$\begin{aligned}w &= w - \alpha \frac{\partial J}{\partial w} \\&= 0 - 0.1(-11.33) \\w &= 1.133\end{aligned}$$

Update Bias

$$\begin{aligned}b &= b - \alpha \frac{\partial J}{\partial b} \\&= 0 - 0.1(-5) \\b &= 0.5\end{aligned}$$

After Iteration 1

$$w = 1.133, b = 0.5$$

Iteration 2

এখন নতুন Parameter ব্যবহার করব।

$$\begin{aligned}w &= 1.133 \\b &= 0.5\end{aligned}$$

Prediction

$$\hat{y} = wx + b$$

x	y	Prediction
1	3	1.633

2	5	2.766
3	7	3.899

Error

x	y	Prediction	Error
1	3	1.633	1.367
2	5	2.766	2.234
3	7	3.899	3.101

Squared Error

Error	Error ²
1.367	1.87
2.234	4.99
3.101	9.62

$$SSE = 1.87 + 4.99 + 9.62$$

$$SSE = 16.48$$

Cost,

$$J = \frac{16.48}{6}$$

$$J = 2.75$$

দেখো,

প্রথম Iteration-এ Cost ছিল

13.83

এখন হয়েছে

2.75

অর্থাৎ Cost অনেক কমে গেছে।

Gradient of w

$$\begin{aligned} & -\frac{1}{3}[(1)(1.367) + (2)(2.234) + (3)(3.101)] \\ &= -\frac{1}{3}(15.138) \\ &= \boxed{-5.046} \end{aligned}$$

Gradient of b

$$\begin{aligned} & -\frac{1}{3}(1.367 + 2.234 + 3.101) \\ &= -\frac{6.702}{3} \\ &= \boxed{-2.234} \end{aligned}$$

Update Weight

$$\begin{aligned} w &= 1.133 - 0.1(-5.046) \\ &= \boxed{1.638} \end{aligned}$$

Update Bias

$$\begin{aligned} b &= 0.5 - 0.1(-2.234) \\ &= \boxed{0.723} \end{aligned}$$

After Many Iterations...

Gradient Descent একই কাজ শত শত Iteration ধরে করতে থাকে।

ধরো ২০০ Iteration শেষে Model শিখল,

$$\boxed{w = 2, b = 1}$$

তখন Final Linear Regression Equation হবে,

$$\boxed{\hat{y} = 2x + 1}$$

Final Prediction

Study Hour	Actual Marks	Predicted Marks
1	3	3
2	5	5
3	7	7

এখানে দেখা যাচ্ছে Model-এর Prediction এবং Actual Value সম্পূর্ণ মিলে গেছে।

Learning Summary

- শুরুতে Model-এর Weight ও Bias ছিল Random।
- প্রথম Iteration-এ Cost ছিল **13.83**।
- দ্বিতীয় Iteration-এ Cost কমে **2.75** হয়েছে।
- প্রতিটি Iteration-এ Gradient ব্যবহার করে Weight ও Bias Update হয়েছে।
- Cost ধীরে ধীরে কমতে কমতে Model $w = 2$ এবং $b = 1$ শিখেছে।
- Final Equation হয়েছে,

$$\hat{y} = 2x + 1$$

Batch Gradient Descent

Gradient Descent-এর সবচেয়ে মৌলিক রূপ হলো **Batch Gradient Descent (BGD)**। এখানে প্রতিবার Weight এবং Bias Update করার আগে **সম্পূর্ণ Training Dataset** ব্যবহার করে Gradient গণনা করা হয়।

অর্থাৎ, Dataset-এ যদি ১০,০০০টি Training Example থাকে, তাহলে প্রতিটি Iteration-এ Model প্রথমে সব ১০,০০০টি Example-এর Prediction করবে, তারপর সবগুলোর Error গণনা করবে, এরপর Gradient নির্ণয় করবে এবং শেষে একবার Weight ও Bias Update করবে।

Weight Update Formula,

$$w := w - \alpha \frac{\partial J}{\partial w}$$

Bias Update Formula,

$$b := b - \alpha \frac{\partial J}{\partial b}$$

যেখানে,

- w = Weight
- b = Bias
- α = Learning Rate
- J = Cost Function

ধরো একটি Dataset-এ ৫টি Sample রয়েছে।

Sample	x	y
1	1	3
2	2	5
3	3	7
4	4	9
5	5	11

Batch Gradient Descent-এ প্রথমে এই পাঁচটি Sample-এর Prediction করা হবে। এরপর পাঁচটি Error গণনা করা হবে। তারপর পাঁচটি Sample ব্যবহার করে Gradient নির্ণয় করা হবে এবং সবশেষে Weight ও Bias **একবার** Update হবে।

অর্থাৎ,

Complete Dataset



Prediction



Error Calculation



Gradient Calculation



One Update of w and b

Dataset যত বড় হবে, প্রতিটি Update করতে তত বেশি সময় লাগবে। তবে Gradient অনেক স্থিতিশীল (Stable) হয় এবং Cost Function সাধারণত Smoothভাবে কমে।

Advantages

- সম্পূর্ণ Dataset ব্যবহার করায় Gradient নির্ভুল হয়।
- Cost Function Smoothভাবে কমে।
- Convergence তুলনামূলকভাবে স্থিতিশীল।

Disadvantages

- বড় Dataset-এ Training ধীর হয়।
- Memory বেশি লাগে।
- প্রতিটি Update-এর আগে পুরো Dataset Process করতে হয়।

Key Points

- পুরো Dataset ব্যবহার করে Gradient গণনা করা হয়।
- প্রতিটি Iteration-এ মাত্র একবার Weight Update হয়।
- Stable হলেও বড় Dataset-এর জন্য ধীর।

Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD)-এ পুরো Dataset ব্যবহার করা হয় না। এখানে একবারে মাত্র একটি **Training Example** ব্যবহার করে Gradient গণনা করা হয় এবং সঙ্গে সঙ্গে Weight Update করা হয়।

অর্থাৎ, যদি Dataset-এ ১০,০০০টি Sample থাকে, তাহলে প্রতিটি Sample-এর পরে Weight Update হবে।

Weight Update Formula,

$$w := w - \alpha \frac{\partial J_i}{\partial w}$$

Bias Update Formula,

$$b := b - \alpha \frac{\partial J_i}{\partial b}$$

এখানে,

- J_i = একটি মাত্র Training Example-এর Cost

ধরো নিচের Dataset রয়েছে।

Sample	x	y
1	1	3
2	2	5
3	3	7
4	4	9

SGD-এর কাজ হবে নিচের মতো।

Sample 1

↓

Update

Sample 2

↓

Update

Sample 3

↓

Update

Sample 4

↓

Update

অর্থাৎ ৪টি Sample থাকলে ৪ বার Update হবে।

SGD অনেক দ্রুত শেখে, কারণ প্রতিটি Sample-এর পরই নতুন Parameter পাওয়া যায়। তবে Gradient শুধুমাত্র একটি Sample-এর উপর নির্ভর করায় Update-এ Noise থাকে এবং Cost Function অনেক ওঠানামা করতে পারে।

Advantages

- Training দ্রুত শুরু হয়।
 - Memory কম লাগে।
 - বড় Dataset-এর জন্য কার্যকর।
 - Local Minimum থেকে বের হওয়ার সম্ভাবনা বেশি।
-

Disadvantages

- Gradient অনেক Noise তৈরি করে।
 - Cost Function Smoothভাবে কমে না।
 - Convergence অস্থিতিশীল হতে পারে।
-

Key Points

- একবারে একটি Sample ব্যবহার করা হয়।
 - প্রতিটি Sample-এর পরে Weight Update হয়।
 - Fast কিন্তু Noisy।
-

Mini Batch Gradient Descent

Mini Batch Gradient Descent হলো Batch Gradient Descent এবং SGD-এর সমন্বয়।

এখানে পুরো Dataset-কে ছোট ছোট Batch-এ ভাগ করা হয়। প্রতিটি Batch ব্যবহার করে Gradient গণনা করা হয় এবং এরপর Weight Update করা হয়।

ধরো Dataset-এ ১২টি Sample রয়েছে এবং Batch Size = 8।

তাহলে Dataset তিনটি Batch-এ ভাগ হবে।

Batch 1
Sample 1 - 4
↓
Update

Batch 2
Sample 5 - 8
↓
Update

Batch 3

Sample 9 - 12

↓

Update

প্রতিটি Batch-এর জন্য একই Update Formula ব্যবহৃত হয়।

Weight Update,

$$w := w - \alpha \frac{\partial J_{batch}}{\partial w}$$

Bias Update,

$$b := b - \alpha \frac{\partial J_{batch}}{\partial b}$$

এখানে,

- J_{batch} = একটি Batch-এর Average Cost

বর্তমানে TensorFlow, PyTorch এবং অধিকাংশ Deep Learning Framework-এ Mini Batch Gradient Descent-ই Default Training Method হিসেবে ব্যবহৃত হয়।

সাধারণ Batch Size হলো,

- 16
- 32
- 64
- 128
- 256

Advantages

- Batch Gradient Descent-এর তুলনায় দ্রুত।
 - SGD-এর তুলনায় অনেক বেশি Stable।
 - Memory কম লাগে।
 - GPU-তে খুব ভালো কাজ করে।
-

Disadvantages

- সঠিক Batch Size নির্বাচন করতে হয়।
 - খুব ছোট Batch হলে Noise বাড়ে।
 - খুব বড় Batch হলে Batch Gradient Descent-এর মতো ধীর হয়ে যায়।
-

Key Points

- Dataset ছোট ছোট Batch-এ ভাগ করা হয়।
 - প্রতিটি Batch-এর পরে Weight Update হয়।
 - বর্তমানে সবচেয়ে বেশি ব্যবহৃত Gradient Descent।
-

Learning Rate

Gradient Descent-এ একটি অত্যন্ত গুরুত্বপূর্ণ Hyperparameter হলো **Learning Rate**।

Learning Rate নির্ধারণ করে প্রতিবার Gradient অনুসরণ করে Weight এবং Bias কতটুকু পরিবর্তন হবে। অর্থাৎ, এটি প্রতিটি Update-এর **Step Size** নির্ধারণ করে।

Weight Update Formula,

$$w := w - \alpha \frac{\partial J}{\partial w}$$

Bias Update Formula,

$$b := b - \alpha \frac{\partial J}{\partial b}$$

এখানে,

- α = Learning Rate

Learning Rate-এর মান খুব ছোট, খুব বড় অথবা উপযুক্ত হতে পারে।

Case 1: Learning Rate খুব ছোট

ধরো,

$$\alpha = 0.0001$$

তাহলে প্রতিবার খুব অল্প পরিবর্তন হবে। ফলে Model সঠিক Solution পেলেও সেখানে পৌঁছাতে অনেক Iteration লাগবে।

Minimum

● → → → → → → → → →

Case 2: Learning Rate খুব বড়

ধরো,

$$\alpha = 1$$

তাহলে প্রতিবার অনেক বড় Jump হবে। ফলে Model Minimum Point অতিক্রম করে বারবার এদিক-ওদিক লাফাবে এবং Converge নাও করতে পারে।

Minimum

← — ● — →

↑

বারবার লাফাচ্ছে

Case 3: উপযুক্ত Learning Rate

ধরো,

$$\alpha = 0.01$$

তাহলে প্রতিটি Step সঠিক আকারের হবে এবং Model ধীরে ধীরে Minimum-এর দিকে এগিয়ে যাবে।

Minimum

● → → → ✓

Example

ধরো,

বর্তমান Weight,

$$w = 2$$

Gradient,

$$\frac{\partial J}{\partial w} = 5$$

যদি,

$$\alpha = 0.1$$

তাহলে,

$$\begin{aligned} w &= 2 - 0.1(5) \\ w &= 1.5 \end{aligned}$$

আবার যদি,

$$\alpha = 0.01$$

হয়,

$$\begin{aligned} w &= 2 - 0.01(5) \\ w &= 1.95 \end{aligned}$$

দেখা যাচ্ছে, Learning Rate কম হলে Update-ও ছোট হয়।

Key Points

- Learning Rate একটি Hyperparameter।
- এটি প্রতিটি Update-এর Step Size নির্ধারণ করে।
- খুব ছোট Learning Rate হলে Training ধীর হয়।
- খুব বড় Learning Rate হলে Model Converge নাও করতে পারে।
- উপযুক্ত Learning Rate দ্রুত এবং স্থিতিশীলভাবে Minimum-এর দিকে নিয়ে যায়।

Polynomial Regression

অনেক সময় বাস্তব জীবনের Data-এর মধ্যে সম্পর্ক সরলরেখা (Linear) অনুসরণ করে না। অর্থাৎ, Input বাড়ার সাথে সাথে Output একই হারে বাড়ে বা কমে না। এই ধরনের **Non-linear Relationship** মডেল করার জন্য **Polynomial Regression** ব্যবহার করা হয়।

নাম Polynomial Regression হলেও এটি মূলত **Linear Regression-এরই একটি Extension**। কারণ Model এখনও Weight-এর প্রতি Linear থাকে। পার্থক্য হলো, এখানে মূল Feature-এর সাথে তার বিভিন্ন Power (যেমন x^2 , x^3 , x^4) যুক্ত করা হয়।

Simple Linear Regression-এর Equation হলো,

$$\hat{y} = wx + b$$

কিন্তু Polynomial Regression-এ একই Feature-এর বিভিন্ন Power ব্যবহার করা হয়।

Second Degree Polynomial,

$$\hat{y} = b + w_1x + w_2x^2$$

Third Degree Polynomial,

$$\hat{y} = b + w_1x + w_2x^2 + w_3x^3$$

General Form,

$$\hat{y} = b + w_1x + w_2x^2 + w_3x^3 + \dots + w_nx^n$$

Terms

- $\hat{y}$ = Predicted Value
- x = Input Feature
- $x^2, x^3, \dots, x^n$ = Polynomial Features
- $w_1, w_2, \dots, w_n$ = Regression Coefficients
- b = Bias (Intercept)

ধরো একজন শিক্ষার্থী প্রতিদিন কত ঘণ্টা পড়ে এবং তার পরীক্ষার নম্বর নিচের মতো।

Study Hours	Marks
-------------	-------

1	15
2	32
3	55
4	80
5	90

এখানে লক্ষ্য করলে দেখা যায়, Study Hour বাড়ার সাথে Marks সমান হারে বাড়ছে না। শুরুতে ধীরে বাড়ছে, পরে দ্রুত বাড়ছে। তাই একটি Straight Line এই Data-কে ভালোভাবে Fit করতে পারবে না।

এই ক্ষেত্রে একটি Polynomial Equation ব্যবহার করা যেতে পারে।

ধরো Training শেষে Model শিখেছে,

$$\hat{y} = 5 + 4x + 2x^2$$

যদি,

$$x = 3$$

তাহলে,

$$\begin{aligned}\hat{y} &= 5 + 4(3) + 2(3)^2 \\ &= 5 + 12 + 18 \\ \boxed{\hat{y} = 35}\end{aligned}$$

অর্থাৎ ৩ ঘন্টা পড়লে Model ৩৫ নম্বর Prediction করছে।

আরেকটি উদাহরণ হলো House Price Prediction। অনেক সময় বাড়ির আয়তন বাড়ার সাথে সাথে দাম সরলরেখার মতো বাড়ে না। একইভাবে Population Growth, Temperature Change, Crop Yield, Fuel Consumption ইত্যাদি সমস্যাও Polynomial Regression ব্যবহার করা হয়।

তবে Degree খুব বেশি হলে Model Training Data খুব ভালোভাবে শিখে ফেলে এবং নতুন Data-এর ক্ষেত্রে খারাপ Prediction দেয়। একে **Overfitting** বলা হয়। তাই সাধারণত Degree 2 বা Degree 3 বেশি ব্যবহৃত হয়।

Advantages

- Non-linear Relationship শিখতে পারে।
- Linear Regression-এর তুলনায় বেশি Flexible।
- Curved Data ভালোভাবে Fit করতে পারে।
- বাস্তব জীবনের অনেক সমস্যাও কার্যকর।

Disadvantages

- Degree বেশি হলে Overfitting হতে পারে।
- Model জটিল হয়ে যায়।
- খুব বড় Degree Prediction অস্থিতিশীল করতে পারে।

Key Points

- Polynomial Regression হলো Linear Regression-এর Extension।
- এটি Non-linear Data Model করতে ব্যবহৃত হয়।
- এখানে Feature-এর বিভিন্ন Power ব্যবহার করা হয়।
- সাধারণ Equation হলো $\hat{y} = b + w_1x + w_2x^2 + \dots + w_nx^n$ ।
- Degree বেশি হলে Overfitting-এর ঝুঁকি থাকে।

Model Performance Metrics

Machine Learning Model Training শেষ হওয়ার পর সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন হলো, **Model কতটা ভালো কাজ করছে?**

এই প্রশ্নের উত্তর পাওয়ার জন্য বিভিন্ন **Model Performance Metrics** ব্যবহার করা হয়। এগুলোর মাধ্যমে বোঝা যায় Model-এর Prediction বাস্তব মানের কতটা কাছাকাছি, Error কত, এবং Model Data-এর Variation কতটা ব্যাখ্যা করতে পারছে।

Regression Model-এর ক্ষেত্রে সবচেয়ে বেশি ব্যবহৃত Performance Metrics হলো:

- SSR (Regression Sum of Squares)
- SSE (Sum of Squared Errors)
- SST (Total Sum of Squares)
- MSE (Mean Squared Error)
- RMSE (Root Mean Squared Error)
- MAE (Mean Absolute Error)
- R^2 Score (Coefficient of Determination)

এই Metrics-গুলোর বেশিরভাগই **Actual Value (y)**, **Predicted Value ($\hat{y}$)**, এবং **Mean of Actual Values ($\bar{y}$)**-এর উপর ভিত্তি করে গণনা করা হয়।

ধরো একটি Dataset রয়েছে।

Sample	Actual (y)	Prediction ($\hat{y}$)
1	40	42
2	50	48
3	60	61
4	70	69

প্রথমে Actual Value-এর Mean বের করি।

$$\bar{y} = \frac{40 + 50 + 60 + 70}{4} = 55$$

এখন প্রতিটি Metric এই Dataset ব্যবহার করে গণনা করা যাবে।

Performance Metrics-এর মূল উদ্দেশ্য তিনটি।

প্রথমত, Model-এর Error কত তা নির্ণয় করা।

দ্বিতীয়ত, Model Data-এর Variation কতটা ব্যাখ্যা করতে পারছে তা জানা।

তৃতীয়ত, একাধিক Model-এর মধ্যে কোনটি ভালো তা তুলনা করা।

যেমন, যদি দুটি Linear Regression Model থাকে এবং একটির RMSE কম হয়, তাহলে সাধারণত সেটিকেই ভালো Model ধরা হয়। আবার R^2 Score যত 1-এর কাছাকাছি হবে, Model তত ভালোভাবে Data ব্যাখ্যা করতে পারবে।

Regression-এর প্রতিটি Metric-এর নিজস্ব গুরুত্ব রয়েছে। কোনো একটি Metric সব পরিস্থিতিতে সেরা নয়। তাই বাস্তবে একাধিক Performance Metric একসাথে ব্যবহার করা হয়।

পরবর্তী অংশে আমরা SSR, SSE, SST, MSE, RMSE, MAE এবং R^2 Score প্রতিটি Metric-এর Formula, Terms এবং Step-by-Step Calculation শিখব।

Key Points

- Model Performance Metrics Model-এর কার্যকারিতা মূল্যায়নের জন্য ব্যবহৃত হয়।
- Regression-এর জন্য একাধিক Metric ব্যবহার করা হয়।
- এগুলো Prediction Error এবং Model Accuracy পরিমাপ করে।
- একাধিক Model তুলনা করার জন্যও এই Metrics ব্যবহৃত হয়।
- পরবর্তী অধ্যায়ে প্রতিটি Metric আলাদাভাবে বিস্তারিত আলোচনা করা হবে।

SSR (Regression Sum of Squares)

SSR পরিমাপ করে Model Data-এর মোট Variation-এর কতটুকু ব্যাখ্যা করতে পেরেছে।

Formula

$$SSR = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

Terms

- SSR = Regression Sum of Squares
- $\hat{y}_i$ = Predicted Value
- $\bar{y}$ = Mean of Actual Values
- n = Number of Samples

Interpretation

- SSR যত বেশি, Model তত বেশি Variation ব্যাখ্যা করতে পারে।
-

SSE (Sum of Squared Errors)

SSE পরিমাপ করে Prediction এবং Actual Value-এর মধ্যে মোট Squared Error।

Formula

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Terms

- SSE = Sum of Squared Errors
- y_i = Actual Value
- $\hat{y}_i$ = Predicted Value

Interpretation

- SSE যত কম, Model তত ভালো।
-

SST (Total Sum of Squares)

SST Data-এর মোট Variation পরিমাপ করে।

Formula

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2$$

Terms

- SST = Total Sum of Squares
- y_i = Actual Value
- $\bar{y}$ = Mean of Actual Values

Relationship

$$SST = SSR + SSE$$

MSE (Mean Squared Error)

MSE হলো প্রতি Sample-এর গড় Squared Error।

Formula

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Terms

- MSE = Mean Squared Error
- n = Number of Samples
- y_i = Actual Value
- $\hat{y}_i$ = Predicted Value

Interpretation

- MSE যত কম, Prediction তত ভালো।
-

RMSE (Root Mean Squared Error)

RMSE হলো MSE-এর Square Root।

Formula

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

অথবা,

$$RMSE = \sqrt{MSE}$$

Terms

- $RMSE$ = Root Mean Squared Error
- MSE = Mean Squared Error

Interpretation

- $RMSE$ -এর Unit Target Variable-এর Unit-এর সমান হয়।
- $RMSE$ যত কম, Model তত ভালো।

MAE (Mean Absolute Error)

MAE হলো Prediction Error-এর গড় Absolute Value।

Formula

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Terms

- MAE = Mean Absolute Error
- y_i = Actual Value
- $\hat{y}_i$ = Predicted Value
- n = Number of Samples

Interpretation

- MAE যত কম, Prediction তত সঠিক।

R² Score (Coefficient of Determination)

R² Score নির্দেশ করে Model Data-এর মোট Variation-এর কত শতাংশ ব্যাখ্যা করতে পেরেছে।

Formula

$$R^2 = \frac{SSR}{SST}$$

অথবা,

$$R^2 = 1 - \frac{SSE}{SST}$$

Terms

- R^2 = Coefficient of Determination
- SSR = Regression Sum of Squares
- SSE = Sum of Squared Errors
- SST = Total Sum of Squares

Interpretation

- $R^2 = 1 \rightarrow$ Perfect Fit
- $R^2 = 0 \rightarrow$ Model কোনো Variation ব্যাখ্যা করতে পারেনি।
- $R^2 < 0 \rightarrow$ Model Mean Prediction-এর থেকেও খারাপ Performance করেছে।
- R^2 যত 1-এর কাছাকাছি হবে, Model তত ভালো।

Metric	Formula
SSR (Regression Sum of Squares)	$SSR = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$
SSE (Sum of Squared Errors)	$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$
SST (Total Sum of Squares)	$SST = \sum_{i=1}^n (y_i - \bar{y})^2$
Relationship	$SST = SSR + SSE$
MSE (Mean Squared Error)	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
RMSE (Root Mean Squared Error)	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} = \sqrt{MSE}$
MAE (Mean Absolute Error)	$MAE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $
R ² Score (Coefficient of Determination)	$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$

Classification

Machine Learning-এ **Classification** হলো এমন একটি Supervised Learning Task, যেখানে Model-এর কাজ হলো কোনো Data-কে পূর্বনির্ধারিত একটি বা একাধিক Category (Class)-এর মধ্যে শ্রেণিবদ্ধ করা। অর্থাৎ, এখানে Output একটি **Continuous Number** নয়, বরং একটি **Discrete Category বা Label** হয়।

Classification-এর সময় Model Training Dataset থেকে বিভিন্ন Feature এবং তাদের Corresponding Label শিখে। এরপর নতুন কোনো Data Input দিলে Model নির্ধারণ করে সেটি কোন Class-এর অন্তর্ভুক্ত হবে।

গাণিতিকভাবে একটি Classification Model-কে নিচেরভাবে প্রকাশ করা যায়।

$$\hat{y} = f(x)$$

যেখানে,

- $\hat{y}$ = Predicted Class
- x = Input Features
- $f(\cdot)$ = Classification Model

অনেক Classification Algorithm প্রথমে প্রতিটি Class-এর Probability নির্ণয় করে, তারপর সর্বোচ্চ Probability-যুক্ত Class-কে Final Prediction হিসেবে নির্বাচন করে।

ধরো, একটি Email Spam Detection System তৈরি করা হয়েছে। এখানে প্রতিটি Email-এর দুটি সম্ভাব্য Class রয়েছে।

- Spam
- Not Spam

যদি Model কোনো নতুন Email-এর জন্য নিচের Probability Prediction করে,

- Spam = 0.92
- Not Spam = 0.08

তাহলে Final Prediction হবে,

Spam

একইভাবে একটি Medical Diagnosis Model রোগীর বিভিন্ন তথ্য বিশ্লেষণ করে নির্ধারণ করতে পারে রোগী **Disease** অথবা **No Disease** Category-তে পড়ে কিনা।

Classification-এর সবচেয়ে সাধারণ ধরন হলো **Binary Classification**, যেখানে মাত্র দুটি Class থাকে।

উদাহরণ:

- Yes / No

- Pass / Fail
- Fraud / Not Fraud
- Churn / Not Churn
- Cancer / No Cancer
- Spam / Not Spam

Classification Problem সমাধানের জন্য বিভিন্ন Machine Learning Algorithm ব্যবহৃত হয়।

Common Classification Algorithms

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Decision Tree
- Random Forest
- Naive Bayes
- Gradient Boosting
- XGBoost
- Neural Networks

Classification Model-এর Performance সাধারণত Accuracy, Precision, Recall, F1-Score এবং ROC-AUC-এর মাধ্যমে মূল্যায়ন করা হয়।

Examples

Problem	Class
Email Detection	Spam / Not Spam
Student Result	Pass / Fail
Loan Approval	Approved / Rejected
Customer Churn	Churn / Not Churn
Disease Detection	Positive / Negative

Key Points

- Classification একটি Supervised Learning Task।

- Output একটি Category বা Label হয়।
 - Continuous Value Prediction করে না।
 - Binary এবং Multiclass উভয় ধরনের Problem সমাধান করতে পারে।
 - বিভিন্ন Classification Algorithm ব্যবহার করা হয়।
-

Multiclass Classification

যখন একটি Classification Problem-এ দুইটির বেশি Class থাকে, তখন তাকে **Multiclass Classification** বলা হয়।

অর্থাৎ, Model-কে একাধিক সম্ভাব্য Category-এর মধ্যে শুধুমাত্র একটি সঠিক Class নির্বাচন করতে হয়।

ধরো, একটি Handwritten Digit Recognition System তৈরি করা হয়েছে। এখানে প্রতিটি ছবি নিচের যেকোনো একটি সংখ্যার হতে পারে।

0,1,2,3,4,5,6,7,8,9

মোট ১০টি Class রয়েছে। তাই এটি একটি Multiclass Classification Problem।

একইভাবে একটি Animal Classification System বিভিন্ন প্রাণীর ছবি দেখে নিচের Class-এর একটি Prediction করতে পারে।

- Cat
- Dog
- Horse
- Cow
- Bird

এখানে পাঁচটি সম্ভাব্য Class রয়েছে, তাই এটিও একটি Multiclass Classification Problem।

গাণিতিকভাবে,

$$\hat{y} \in \{C_1, C_2, \dots, C_k\}$$

যেখানে,

- $\hat{y}$ = Predicted Class
- $C_1, C_2, \dots, C_k$ = সম্ভাব্য সব Class
- k = মোট Class-এর সংখ্যা

ধরো একটি Model একটি ছবির জন্য নিচের Probability Prediction করেছে।

Class	Probability
Cat	0.10
Dog	0.72
Horse	0.08
Cow	0.06
Bird	0.04

সর্বোচ্চ Probability হলো **Dog (0.72)**।

অতএব,

Predicted Class = Dog

অনেক জনপ্রিয় Machine Learning Algorithm সরাসরি Multiclass Classification সমর্থন করে। কিছু Algorithm আবার Binary Classification Algorithm হওয়ায় One-vs-Rest (OvR) বা One-vs-One (OvO) কৌশল ব্যবহার করে Multiclass Problem সমাধান করে।

Algorithms Supporting Multiclass Classification

- Logistic Regression (OvR / Multinomial)
- K-Nearest Neighbors (KNN)
- Decision Tree
- Random Forest
- Naive Bayes
- Support Vector Machine (OvR / OvO)
- Neural Networks

Examples

Problem	Classes
Handwritten Digit Recognition	0–9
Iris Flower Classification	Setosa, Versicolor, Virginica
Animal Classification	Cat, Dog, Horse, Cow, Bird
Fruit Classification	Apple, Mango, Orange, Banana
Language Detection	English, Bangla, Arabic, Hindi

Key Points

- Multiclass Classification-এ দুইটির বেশি Class থাকে।
- প্রতিটি Sample শুধুমাত্র একটি Class-এর অন্তর্ভুক্ত হয়।
- Model সর্বোচ্চ Probability-যুক্ত Class নির্বাচন করে।
- Digit Recognition, Iris Classification এবং Animal Classification হলো Multiclass Classification-এর জনপ্রিয় উদাহরণ।
- KNN, Decision Tree, Random Forest, Naive Bayes এবং Neural Networks সরাসরি Multiclass Classification সমর্থন করে।

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) হলো একটি **Supervised Machine Learning Algorithm**, যা Classification এবং Regression উভয় ধরনের সমস্যার সমাধানে ব্যবহার করা যায়। তবে বাস্তবে KNN সবচেয়ে বেশি ব্যবহৃত হয় **Classification Problem** সমাধানের জন্য।

KNN-এর মূল ধারণা হলো, "**একই ধরনের Data সাধারণত একে অপরের কাছাকাছি অবস্থান করে।**" তাই নতুন কোনো Data Point-এর Class নির্ধারণ করতে হলে তার সবচেয়ে কাছাকাছি থাকা Training Data-গুলোর Class পর্যবেক্ষণ করলেই ভালো Prediction পাওয়া যায়।

অন্যান্য অনেক Machine Learning Algorithm-এর মতো KNN Training-এর সময় কোনো Mathematical Model তৈরি করে না। এটি শুধুমাত্র Training Dataset সংরক্ষণ করে রাখে। Prediction করার সময় নতুন Data Point-এর সাথে Training Data-এর Distance গণনা করে এবং সবচেয়ে কাছের Kটি Neighbor-এর উপর ভিত্তি করে সিদ্ধান্ত নেয়।

এই বৈশিষ্ট্যের কারণে KNN-কে **Lazy Learning Algorithm**, **Instance-Based Learning Algorithm** এবং **Memory-Based Learning Algorithm** বলা হয়।

Why is it called K-Nearest Neighbors?

- **K** = কতজন Neighbor বিবেচনা করা হবে।
- **Nearest** = Distance অনুযায়ী সবচেয়ে কাছাকাছি।
- **Neighbors** = Training Dataset-এর কাছাকাছি থাকা Data Points।

উদাহরণস্বরূপ, যদি $K = 5$ হয়, তাহলে নতুন Data Point-এর সবচেয়ে কাছের ৫টি Neighbor নির্বাচন করা হবে। এরপর এই ৫টি Neighbor-এর Class বিশ্লেষণ করে Final Prediction দেওয়া হবে।

How KNN Works

KNN Prediction করার সময় নিচের ধাপগুলো অনুসরণ করে।

1. একটি K-এর মান নির্বাচন করা।
2. নতুন Data Point-এর সাথে Training Data-এর প্রতিটি Sample-এর Distance গণনা করা।
3. Distance অনুযায়ী ছোট থেকে বড় ক্রমে সাজানো।
4. সবচেয়ে কাছের Kটি Neighbor নির্বাচন করা।
5. Classification হলে Majority Voting করা।
6. Regression হলে Neighbor-গুলোর Average নেওয়া।
7. Final Prediction প্রদান করা।

KNN-এর সম্পূর্ণ কাজ শুধুমাত্র Prediction-এর সময় হয়। তাই Training Phase প্রায় থাকে না, কিন্তু Prediction তুলনামূলক ধীর হয়।

KNN Algorithm

Input

- Training Dataset
- Test Data
- Value of K

Algorithm

Step 1: একটি উপযুক্ত K নির্বাচন করো।

Step 2: নতুন Data Point-এর সাথে Training Dataset-এর প্রতিটি Sample-এর Distance গণনা করো।

Step 3: Distance অনুযায়ী সব Sample-কে Ascending Order-এ সাজাও।

Step 4: সবচেয়ে কাছের K টি Neighbor নির্বাচন করো।

Step 5: যদি এটি Classification Problem হয়, তাহলে Neighbor-গুলোর মধ্যে Majority Vote গণনা করো।

Step 6: যদি এটি Regression Problem হয়, তাহলে Neighbor-গুলোর Target Value-এর Average নির্ণয় করো।

Step 7: Final Prediction প্রদান করো।

KNN for Classification and Regression

KNN দুটি ভিন্ন ধরনের Problem সমাধান করতে পারে।

Classification

Classification-এ KNN সবচেয়ে কাছের Neighbor-গুলোর মধ্যে যে Class সবচেয়ে বেশি থাকে, সেটিকে Prediction হিসেবে নির্বাচন করে।

Formula,

$$\hat{y} = \text{Mode of } K \text{ Nearest Neighbors}$$

Terms

- $\hat{y}$ = Predicted Class
 - Mode = সবচেয়ে বেশি সংখ্যক Neighbor-এর Class
-

Example

ধরো,

Nearest Neighbor-এর Class হলো,

- Blue
- Blue
- Red
- Blue
- Red

Voting,

- Blue = 3
- Red = 2

সুতরাং,

Prediction = Blue

Regression

Regression-এর ক্ষেত্রে Majority Voting ব্যবহার করা হয় না।

বরং সবচেয়ে কাছের Neighbor-গুলোর Target Value-এর Average নেওয়া হয়।

Formula,

$$\hat{y} = \frac{1}{K} \sum_{i=1}^K y_i$$

Terms

- $\hat{y}$ = Predicted Value
- K = Number of Neighbors
- y_i = Neighbor-এর Target Value

Example

ধরো,

K = 3

Neighbor-এর House Price,

- 52 Lakh
- 55 Lakh
- 56 Lakh

তাহলে,

$$\begin{aligned}\hat{y} &= \frac{52 + 55 + 56}{3} \\ &= \frac{163}{3} \\ \boxed{\hat{y} = 54.33 \text{ Lakh}}\end{aligned}$$

Distance Metrics in KNN

KNN-এর সবচেয়ে গুরুত্বপূর্ণ ধাপ হলো Distance Measurement।

Distance নির্ণয়ের জন্য বিভিন্ন Metric ব্যবহার করা যায়। তবে Machine Learning-এ সবচেয়ে বেশি ব্যবহৃত দুটি Distance হলো,

- Euclidean Distance
- Manhattan Distance

Euclidean Distance

Euclidean Distance হলো দুইটি Point-এর মধ্যে **Straight-Line Distance**।

এটি KNN-এ সবচেয়ে বেশি ব্যবহৃত Distance Metric।

দুইটি Feature-এর জন্য Formula,

$$\boxed{d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}$$

General Formula,

$$\boxed{d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}}$$

Terms

- d = Euclidean Distance
 - x_i, y_i = দুইটি Data Point-এর Feature Value
 - n = মোট Feature-এর সংখ্যা
-

Example

ধরো,

Point A = (2,3)

Point B = (5,7)

তাহলে,

$$\begin{aligned}d &= \sqrt{(5-2)^2 + (7-3)^2} \\&= \sqrt{3^2 + 4^2} \\&= \sqrt{9 + 16} \\&= \sqrt{25} \\&\boxed{d = 5}\end{aligned}$$

Manhattan Distance

Manhattan Distance-কে **City Block Distance** অথবা **Taxicab Distance**-ও বলা হয়।

এখানে সরাসরি Line ধরা হয় না। বরং অনুভূমিক (Horizontal) এবং উল্লম্ব (Vertical) Distance যোগ করা হয়।

দুইটি Feature-এর জন্য Formula,

$$\boxed{d = |x_2 - x_1| + |y_2 - y_1|}$$

General Formula,

$$\boxed{d = \sum_{i=1}^n |x_i - y_i|}$$

Terms

- d = Manhattan Distance

- $|x_i - y_i|$ = Absolute Difference
- n = মোট Feature-এর সংখ্যা

Example

Point A = (2,3)

Point B = (5,7)

তাহলে,

$$d = |5 - 2| + |7 - 3|$$

$$= 3 + 4$$

$$\boxed{d = 7}$$

Euclidean Distance vs Manhattan Distance

Feature	Euclidean Distance	Manhattan Distance
Path	Straight Line	Horizontal + Vertical Path
Formula	$\sqrt{\sum (x_i - y_i)^2}$	$(\sum$
Also Known As	L2 Distance	L1 Distance
Best For	Continuous Space	Grid-like Path

Key Points

- KNN একটি Supervised Learning Algorithm।
- এটি Classification এবং Regression উভয় সমস্যার জন্য ব্যবহার করা যায়।
- Training-এর সময় Model তৈরি করে না; Training Data সংরক্ষণ করে।
- Prediction করার সময় Distance গণনা করা হয়।
- সবচেয়ে বেশি ব্যবহৃত দুটি Distance Metric হলো Euclidean Distance এবং Manhattan Distance।
- Classification-এ Majority Voting এবং Regression-এ Average ব্যবহার করা হয়।

Choosing the Value of K

KNN-এর সবচেয়ে গুরুত্বপূর্ণ Hyperparameter হলো **K**, যা নির্দেশ করে Prediction করার সময় কতজন Nearest Neighbor বিবেচনা করা হবে।

K-এর মান সরাসরি Model-এর Accuracy এবং Generalization-এর উপর প্রভাব ফেলে। যদি K-এর মান খুব ছোট বা খুব বড় নির্বাচন করা হয়, তাহলে Model সঠিকভাবে শিখতে পারে না।

সাধারণভাবে K-এর মান এমনভাবে নির্বাচন করা উচিত যাতে Model Training Data-ও ভালোভাবে শিখতে পারে এবং নতুন Data-এর ক্ষেত্রেও ভালো Prediction দিতে পারে।

Case 1: K = 1 (Very Small K)

যখন $K = 1$, তখন Model শুধুমাত্র সবচেয়ে কাছের একটি Neighbor ব্যবহার করে Prediction করে।

ধরো, একটি নতুন Data Point-এর সবচেয়ে কাছের Neighbor হলো **Blue Class**।

তাহলে,

$$\text{Prediction} = \text{Blue}$$

যদিও আশেপাশের বাকি Data-এর বেশিরভাগই Red হতে পারে, তবুও শুধুমাত্র একটি Neighbor-এর উপর ভিত্তি করে সিদ্ধান্ত নেওয়া হবে।

ফলে Noise বা Outlier থাকলে Prediction সহজেই ভুল হতে পারে।

Advantages

- Prediction খুব দ্রুত হয়।
- Local Pattern ভালোভাবে শিখতে পারে।

Disadvantages

- Noise-এর প্রতি অত্যন্ত সংবেদনশীল।
- Overfitting হওয়ার সম্ভাবনা বেশি।

Case 2: K খুব বড়

যদি K-এর মান অনেক বড় হয়, তাহলে দূরের Neighbor-গুলোকেও Prediction-এ অন্তর্ভুক্ত করা হয়।

ধরো,

মোট Sample = 100

এবং,

$$K = 90$$

এখন Model ৯০টি Neighbor ব্যবহার করবে।

ফলে কাছের Data-এর পরিবর্তে দূরের অনেক Data-ও Decision-এ অংশ নেবে।

এর ফলে Model অত্যন্ত Smooth হয়ে যায় এবং অনেক সময় আসল Pattern শিখতে পারে না।

Advantages

- Noise-এর প্রভাব কমে যায়।
- Prediction তুলনামূলকভাবে Stable হয়।

Disadvantages

- Underfitting হতে পারে।
- Local Pattern হারিয়ে যায়।

Choosing an Appropriate Value of K

K নির্বাচন করার জন্য একটি জনপ্রিয় Rule of Thumb হলো,

$$K \approx \sqrt{N}$$

যেখানে,

- K = Number of Neighbors
- N = মোট Training Sample

Example

যদি,

$$N = 100$$

তাহলে,

$$K = \sqrt{100} = 10$$

বাস্তবে সাধারণত ৯ বা ১১-এর মতো কাছাকাছি একটি **Odd Number** নির্বাচন করা হয়।

তবে সবচেয়ে ভালো K সাধারণত **Cross Validation** ব্যবহার করে নির্ধারণ করা হয়।

Odd vs Even Value of K

সাধারণত K-এর জন্য **Odd Number** নির্বাচন করা হয়।

যেমন,

- $K = 3$
- $K = 5$
- $K = 7$
- $K = 9$

এর কারণ হলো **Tie এড়ানো**।

Example (Even K)

ধরো,

$$K = 4$$

Nearest Neighbor-এর Class,

- Blue
- Blue
- Red
- Red

Voting,

Class	Vote
Blue	2
Red	2

এখানে Tie হয়েছে।

এখন Model সহজে সিদ্ধান্ত নিতে পারবে না।

Example (Odd K)

ধরো,

$$K = 5$$

Neighbor-এর Class,

- Blue
- Blue
- Blue
- Red
- Red

Voting,

Class Vote

Blue 3

Red 2

এখন,

<i>Prediction = Blue</i>

কোনো Tie হয়নি।

Feature Scaling in KNN

KNN সম্পূর্ণভাবে **Distance Calculation**-এর উপর নির্ভর করে।

যদি বিভিন্ন Feature-এর Scale ভিন্ন হয়, তাহলে বড় Scale-এর Feature Distance-এর উপর বেশি প্রভাব ফেলবে এবং ছোট Scale-এর Feature প্রায় উপেক্ষিত হবে।

তাই KNN ব্যবহার করার আগে Feature Scaling করা অত্যন্ত গুরুত্বপূর্ণ।

Example (Without Scaling)

ধরো,

Feature	Value
Age	25
Salary	80000

এখানে Age-এর Range খুব ছোট, কিন্তু Salary-এর Range অনেক বড়।

Distance গণনার সময় Salary-এর পার্থক্য Age-এর তুলনায় অনেক বেশি প্রভাব ফেলবে।
ফলে Prediction ভুল হতে পারে।

Example (After Scaling)

Scaling করার পরে,

Feature	Scaled Value
Age	0.42
Salary	0.48

এখন উভয় Feature সমান গুরুত্ব পাবে।

Common Scaling Methods

- Min-Max Scaling
- Standard Scaling
- Robust Scaling

KNN-এর ক্ষেত্রে সাধারণত **StandardScaler** অথবা **MinMaxScaler** বেশি ব্যবহৃত হয়।

Complete Example of KNN

ধরো নিচের Dataset রয়েছে।

Point	Height (cm)	Weight (kg)	Class
A	160	55	Blue
B	165	60	Blue
C	170	65	Blue
D	180	80	Red
E	185	85	Red

এখন নতুন একটি Data Point এসেছে।

Height Weight

172 68

ধরি,

$$K = 3$$

Euclidean Distance গণনার পরে Distance পাওয়া গেল,

Neighbor	Distance	Class
C	3.61	Blue
B	10.63	Blue
D	14.42	Red
A	17.69	Blue
E	21.40	Red

সবচেয়ে কাছের তিনটি Neighbor হলো,

- C (Blue)
- B (Blue)
- D (Red)

Voting,

Class	Vote
Blue	2
Red	1

যেহেতু Blue-এর Vote বেশি,

Prediction = Blue

এটিই KNN-এর সম্পূর্ণ Prediction Process।

Advantages

- Algorithm সহজ এবং বোঝা সহজ।
- Training Phase প্রায় নেই।
- Classification এবং Regression উভয়ের জন্য ব্যবহার করা যায়।
- Non-linear Decision Boundary শিখতে পারে।
- ছোট Dataset-এ ভালো Performance দেয়।

Disadvantages

- বড় Dataset-এ Prediction ধীর হয়।
- Memory বেশি ব্যবহার করে।
- Feature Scaling না করলে Accuracy কমে যেতে পারে।
- Noise এবং Outlier-এর প্রতি সংবেদনশীল।
- সঠিক K নির্বাচন করা গুরুত্বপূর্ণ।

Applications

- Image Classification
- Handwritten Digit Recognition
- Face Recognition
- Medical Diagnosis
- Recommendation System
- Customer Segmentation
- Pattern Recognition
- Credit Risk Analysis
- Fraud Detection

Key Points

- KNN একটি Distance-Based Supervised Learning Algorithm।
- Prediction করার সময় Training Data-এর Kটি Nearest Neighbor ব্যবহার করা হয়।
- Classification-এ Majority Voting এবং Regression-এ Average ব্যবহার করা হয়।
- Euclidean এবং Manhattan হলো সবচেয়ে বেশি ব্যবহৃত Distance Metrics।
- Feature Scaling KNN-এর জন্য অত্যন্ত গুরুত্বপূর্ণ।
- সাধারণত Odd Value-এর K ব্যবহার করা হয় যাতে Tie না হয়।
- K খুব ছোট হলে Overfitting এবং খুব বড় হলে Underfitting হওয়ার সম্ভাবনা থাকে।

Support Vector Machine (SVM)

Support Vector Machine (SVM) হলো একটি শক্তিশালী **Supervised Machine Learning Algorithm**, যা মূলত **Classification** সমস্যার জন্য ব্যবহৃত হয়। তবে এটি **Regression** সমস্যার (Support Vector Regression বা SVR) সমাধানেও ব্যবহার করা যায়।

SVM-এর মূল উদ্দেশ্য হলো এমন একটি **Decision Boundary** খুঁজে বের করা, যা বিভিন্ন Class-কে সর্বোচ্চ দূরত্ব (Maximum Margin) বজায় রেখে আলাদা করতে পারে। এই Decision Boundary-কে **Hyperplane** বলা হয়।

SVM-এর মূল ধারণা হলো, যদি দুটি Class-কে একাধিক Line বা Plane দিয়ে আলাদা করা সম্ভব হয়, তাহলে এমন Line বা Plane নির্বাচন করা হবে যার দুই পাশের Class-এর নিকটতম Data Point-এর দূরত্ব সর্বাধিক হয়। এই Maximum Distance Model-কে নতুন Data-এর উপর আরও ভালোভাবে কাজ করতে সাহায্য করে।

Working Principle of SVM

SVM-এর কাজকে নিচের ধাপে ব্যাখ্যা করা যায়।

1. Training Dataset গ্রহণ করা।
2. বিভিন্ন সম্ভাব্য Hyperplane তৈরি করা।
3. প্রতিটি Hyperplane-এর Margin গণনা করা।
4. যে Hyperplane-এর Margin সবচেয়ে বেশি, সেটি নির্বাচন করা।
5. নতুন Data Point Hyperplane-এর কোন পাশে রয়েছে তা দেখে তার Class নির্ধারণ করা।

Hyperplane

Hyperplane হলো এমন একটি Decision Boundary যা বিভিন্ন Class-কে আলাদা করে।

- 2D Data-এর জন্য Hyperplane একটি **Straight Line**।
- 3D Data-এর জন্য Hyperplane একটি **Plane**।
- আরও বেশি Feature থাকলে এটি একটি **High-Dimensional Hyperplane**।

Linear SVM-এর Hyperplane-এর Equation হলো,

$$w^T x + b = 0$$

Terms

- w = Weight Vector

- x = Feature Vector
 - b = Bias (Intercept)
 - $w^T x$ = Dot Product of Weight and Feature
 - $w^T x + b = 0$ = Decision Boundary (Hyperplane)
-

Support Vectors

Support Vector হলো সেই Data Point-গুলো, যেগুলো Hyperplane-এর সবচেয়ে কাছাকাছি থাকে।

এই Data Point-গুলোর উপর ভিত্তি করেই SVM Hyperplane নির্ধারণ করে। অন্য দূরের Data Point-গুলোর তুলনায় Support Vector-ই Model-এর Decision-এ সবচেয়ে বেশি প্রভাব ফেলে।

যদি Support Vector পরিবর্তন হয়, তাহলে Hyperplane-ও পরিবর্তিত হতে পারে।

Margin

Margin হলো Hyperplane থেকে দুই Class-এর নিকটতম Support Vector পর্যন্ত দূরত্ব।

SVM-এর লক্ষ্য হলো এই Margin-কে সর্বাধিক করা।

Margin-এর মান,

$$\text{Margin} = \frac{2}{\|w\|}$$

Terms

- $\|w\|$ = Weight Vector-এর Magnitude
 - Margin = দুই Support Vector-এর মধ্যকার সর্বনিম্ন দূরত্ব
-

Types of SVM

SVM প্রধানত দুই ধরনের।

1. Linear SVM

যখন Data-কে একটি সরলরেখা (Line) বা Hyperplane দ্বারা আলাদা করা যায়, তখন Linear SVM ব্যবহার করা হয়।

উদাহরণ,

- Spam Detection
 - Customer Churn Prediction
 - Document Classification
-

2. Non-Linear SVM

যখন Data সরলরেখা দ্বারা আলাদা করা সম্ভব হয় না, তখন Non-Linear SVM ব্যবহার করা হয়।

এই ক্ষেত্রে **Kernel Trick** ব্যবহার করে Data-কে উচ্চমাত্রিক (Higher-Dimensional) Space-এ রূপান্তর করা হয়, যেখানে Linear Hyperplane দিয়ে Data আলাদা করা সম্ভব হয়।

Kernel Function

Kernel Function এমন একটি গাণিতিক পদ্ধতি যা Data-কে সরাসরি Transform না করেও Higher-Dimensional Space-এ Mapping করার কাজ করে।

সবচেয়ে বেশি ব্যবহৃত Kernel হলো,

- Linear Kernel
 - Polynomial Kernel
 - Radial Basis Function (RBF) Kernel
 - Sigmoid Kernel
-

Linear Kernel

Formula,

$$K(x_i, x_j) = x_i^T x_j$$

Polynomial Kernel

Formula,

$$K(x_i, x_j) = (x_i^T x_j + c)^d$$

Terms

- c = Constant
 - d = Polynomial Degree
-

RBF (Gaussian) Kernel

Formula,

$$K(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2}$$

Terms

- γ = Kernel Parameter
 - $\|x_i - x_j\|^2$ = Squared Euclidean Distance
-

Sigmoid Kernel

Formula,

$$K(x_i, x_j) = \tanh(\alpha x_i^T x_j + c)$$

Terms

- α = Scaling Parameter
 - c = Constant
-

Example

ধরো একটি Email Classification Dataset রয়েছে।

Email	Class
Offer Today	Spam
Win Prize	Spam
Project Report	Not Spam

Email **Class**

Meeting Schedule Not Spam

SVM Training-এর পর একটি Hyperplane তৈরি করবে যা Spam এবং Not Spam Email-কে আলাদা করবে।

এখন যদি নতুন Email হয়,

"Congratulations! You won a free gift."

Model সেটিকে Hyperplane-এর Spam Side-এ পেলো Prediction হবে,

Spam

Advantages

- High-Dimensional Data-এ ভালো কাজ করে।
- Overfitting-এর সম্ভাবনা তুলনামূলক কম।
- Maximum Margin-এর কারণে ভালো Generalization করে।
- ছোট ও মাঝারি আকারের Dataset-এ উচ্চ Accuracy দেয়।
- Non-Linear Problem-এর জন্য Kernel ব্যবহার করা যায়।

Disadvantages

- বড় Dataset-এ Training ধীর হয়।
- Kernel নির্বাচন করা কঠিন হতে পারে।
- Hyperparameter Tuning প্রয়োজন।
- Training Time তুলনামূলক বেশি।
- ফলাফল ব্যাখ্যা করা অন্যান্য Model-এর তুলনায় কঠিন।

Applications

- Face Recognition
- Image Classification
- Handwritten Digit Recognition
- Spam Email Detection

- Text Classification
 - Medical Diagnosis
 - Fraud Detection
 - Bioinformatics
 - Customer Churn Prediction
-

Key Points

- SVM একটি Supervised Learning Algorithm।
- এটি প্রধানত Classification Problem সমাধানের জন্য ব্যবহৃত হয়।
- Decision Boundary-কে **Hyperplane** বলা হয়।
- Hyperplane-এর সবচেয়ে কাছের Data Point-গুলোকে **Support Vector** বলা হয়।
- SVM-এর লক্ষ্য হলো **Maximum Margin Hyperplane** খুঁজে বের করা।
- Non-Linear Data-এর জন্য Kernel Function ব্যবহার করা হয়।
- সবচেয়ে জনপ্রিয় Kernel হলো **Linear, Polynomial, RBF এবং Sigmoid**।

Decision Tree

Decision Tree হলো একটি জনপ্রিয় **Supervised Machine Learning Algorithm**, যা Classification এবং Regression উভয় ধরনের সমস্যার সমাধানে ব্যবহার করা যায়। এটি মানুষের সিদ্ধান্ত গ্রহণের পদ্ধতির মতো ধাপে ধাপে বিভিন্ন প্রশ্ন (Decision) করে চূড়ান্ত ফলাফল নির্ধারণ করে।

Decision Tree-এর গঠন অনেকটা একটি উল্টো গাছের মতো। এটি একটি Root Node থেকে শুরু হয় এবং বিভিন্ন শর্ত (Condition) অনুযায়ী Branch তৈরি করে। সর্বশেষ Leaf Node-এ গিয়ে Model একটি Final Prediction প্রদান করে।

Decision Tree-এর মূল ধারণা হলো, প্রতিটি ধাপে এমন একটি Feature নির্বাচন করা যা Data-কে সবচেয়ে ভালোভাবে বিভিন্ন Class-এ বিভক্ত (Split) করতে পারে। এই প্রক্রিয়া চলতে থাকে যতক্ষণ পর্যন্ত Leaf Node-এ পৌঁছানো না যায়।

Structure of a Decision Tree

Decision Tree প্রধানত চারটি অংশ নিয়ে গঠিত।

- Root Node
- Decision Node
- Branch
- Leaf Node

Root Node

Root Node হলো Tree-এর প্রথম Node, যেখানে সম্পূর্ণ Dataset অবস্থান করে। এখান থেকেই প্রথম Split শুরু হয়।

Decision Node

যে Node-এ কোনো Feature-এর উপর ভিত্তি করে Data বিভক্ত হয়, তাকে Decision Node বলা হয়।

Branch

Decision Node থেকে বিভিন্ন সম্ভাব্য Outcome-এর দিকে যে সংযোগ তৈরি হয় তাকে Branch বলা হয়।

Leaf Node

Leaf Node হলো Tree-এর শেষ Node, যেখানে আর কোনো Split হয় না এবং Final Prediction পাওয়া যায়।

Working Principle of Decision Tree

Decision Tree নিচের ধাপগুলো অনুসরণ করে কাজ করে।

1. সম্পূর্ণ Dataset দিয়ে Root Node তৈরি করা।

2. সবচেয়ে গুরুত্বপূর্ণ Feature নির্বাচন করা।
 3. Feature-এর ভিত্তিতে Dataset Split করা।
 4. প্রতিটি Subset-এর জন্য একই প্রক্রিয়া পুনরাবৃত্তি করা।
 5. Stop Condition পূরণ হলে Leaf Node তৈরি করা।
 6. Leaf Node থেকে Final Prediction প্রদান করা।
-

Decision Tree Algorithm

Input

- Training Dataset
- Target Variable

Algorithm

Step 1: Root Node-এ সম্পূর্ণ Dataset রাখো।

Step 2: প্রতিটি Feature-এর জন্য Split Quality নির্ণয় করো।

Step 3: যে Feature সবচেয়ে ভালো Split দেয় সেটি নির্বাচন করো।

Step 4: Dataset-কে বিভিন্ন Branch-এ ভাগ করো।

Step 5: প্রতিটি Branch-এর জন্য একই ধাপ পুনরাবৃত্তি করো।

Step 6: Stop Condition পূরণ হলে Leaf Node তৈরি করো।

Step 7: নতুন Data-এর জন্য Root থেকে Leaf পর্যন্ত Traverse করে Prediction করো।

How Does Decision Tree Choose the Best Feature?

Decision Tree প্রতিটি ধাপে এমন Feature নির্বাচন করে যা Data-কে সবচেয়ে ভালোভাবে আলাদা করতে পারে।

এই Feature নির্বাচন করার জন্য বিভিন্ন Splitting Criteria ব্যবহার করা হয়।

সবচেয়ে জনপ্রিয় দুটি হলো,

- Entropy
- Gini Index

Regression Tree-এর ক্ষেত্রে Variance Reduction বা Mean Squared Error (MSE) ব্যবহার করা হয়।

Entropy

Entropy একটি Node-এর **Impurity** বা **Uncertainty** পরিমাপ করে।

যদি একটি Node-এর সব Sample একই Class-এর হয়, তাহলে Entropy হবে 0।

যদি বিভিন্ন Class সমান সংখ্যায় থাকে, তাহলে Entropy সর্বাধিক হবে।

Formula,

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

Terms

- S = Dataset
 - p_i = Class i -এর Probability
 - c = মোট Class-এর সংখ্যা
-

Information Gain

Information Gain নির্দেশ করে কোনো Feature ব্যবহার করার পরে Entropy কতটা কমেছে।

Decision Tree সাধারণত সর্বোচ্চ Information Gain-যুক্ত Feature নির্বাচন করে।

Formula,

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Terms

- IG = Information Gain
 - S = Original Dataset
 - A = Selected Feature
 - S_v = Split হওয়ার পর Subset
-

Gini Index

Gini Index-ও একটি Impurity Measure।

CART (Classification and Regression Tree) Algorithm সাধারণত Gini Index ব্যবহার করে।

Formula,

$$Gini = 1 - \sum_{i=1}^c p_i^2$$

Terms

- p_i = Probability of Class i
- c = মোট Class-এর সংখ্যা

Gini Index যত কম হবে, Node তত বেশি Pure হবে।

Example

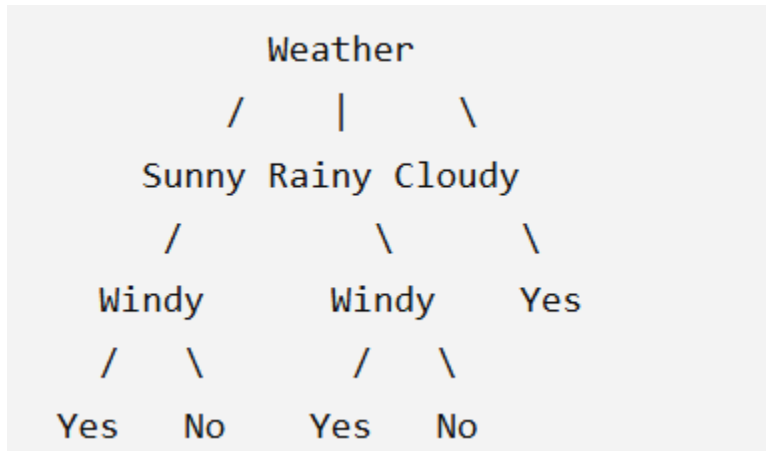
ধরো একটি Dataset রয়েছে।

Weather	Windy	Play Cricket
Sunny	No	Yes
Sunny	Yes	No
Rainy	No	Yes
Rainy	Yes	No
Cloudy	No	Yes

Decision Tree প্রথমে Weather Feature ব্যবহার করে Dataset Split করতে পারে।

তারপর প্রতিটি Branch-এ Windy Feature ব্যবহার করে আরও Split করবে।

শেষে Leaf Node-এ গিয়ে Prediction হবে।



যদি নতুন Data হয়,

- Weather = Sunny
- Windy = No

তাহলে Tree অনুসরণ করলে Prediction হবে,

Play Cricket = Yes

Stopping Criteria

Decision Tree অনন্তকাল Split করতে পারে না। তাই কিছু Stop Condition নির্ধারণ করা হয়।

- সব Sample একই Class-এর হলে।
- Maximum Depth পৌঁছে গেলে।
- Minimum Sample Limit পূরণ না হলে।
- Information Gain খুব কম হলে।

Advantages

- বোঝা এবং ব্যাখ্যা করা সহজ।
- Data Scaling প্রয়োজন হয় না।
- Classification এবং Regression উভয়ের জন্য ব্যবহার করা যায়।
- Numerical এবং Categorical Data উভয়ই পরিচালনা করতে পারে।
- Feature Importance নির্ণয় করা যায়।

Disadvantages

- Overfitting হওয়ার সম্ভাবনা বেশি।
- Noise-এর প্রতি সংবেদনশীল।
- ছোট পরিবর্তনে সম্পূর্ণ Tree পরিবর্তিত হতে পারে।
- Deep Tree-এর Generalization কম হতে পারে।

Applications

- Loan Approval
- Medical Diagnosis
- Fraud Detection
- Customer Churn Prediction
- Spam Detection
- Credit Risk Analysis
- Recommendation System

Key Points

- Decision Tree একটি Supervised Learning Algorithm।
- এটি Classification এবং Regression উভয়ের জন্য ব্যবহৃত হয়।
- Tree-এর প্রধান অংশ হলো Root Node, Decision Node, Branch এবং Leaf Node।
- Feature নির্বাচন করার জন্য সাধারণত **Entropy**, **Information Gain** এবং **Gini Index** ব্যবহার করা হয়।
- CART Algorithm সাধারণত Gini Index ব্যবহার করে।
- Decision Tree সহজে ব্যাখ্যা করা যায়, তবে Overfitting-এর ঝুঁকি বেশি থাকে।

Random Forest

Random Forest হলো একটি শক্তিশালী **Supervised Machine Learning Algorithm**, যা Classification এবং Regression উভয় ধরনের সমস্যার সমাধানে ব্যবহার করা হয়। এটি একাধিক **Decision Tree**-এর সমন্বয়ে গঠিত একটি **Ensemble Learning Algorithm**।

একটি মাত্র Decision Tree ব্যবহার করলে অনেক সময় Model Training Data খুব ভালোভাবে শিখে ফেলে, ফলে নতুন Data-এর উপর খারাপ Performance করে। এই সমস্যাটিকে **Overfitting** বলা হয়। Random Forest এই সমস্যা কমানোর জন্য অনেকগুলো আলাদা Decision Tree তৈরি করে এবং তাদের সম্মিলিত সিদ্ধান্তের ভিত্তিতে Final Prediction প্রদান করে।

Random Forest-এ প্রতিটি Decision Tree ভিন্ন ভিন্ন Training Data এবং ভিন্ন Feature ব্যবহার করে তৈরি করা হয়। ফলে প্রতিটি Tree কিছুটা আলাদা সিদ্ধান্ত দেয়। সবশেষে সব Tree-এর Prediction একত্র করে একটি Final Prediction তৈরি করা হয়।

Why is it called Random Forest?

- **Random** → প্রতিটি Tree Training-এর সময় Random Data এবং Random Feature নির্বাচন করা হয়।
- **Forest** → অনেকগুলো Decision Tree একসাথে একটি Forest তৈরি করে।

Working Principle of Random Forest

Random Forest নিচের ধাপগুলো অনুসরণ করে কাজ করে।

1. মূল Dataset থেকে Random Sampling করে একাধিক Dataset তৈরি করা।
2. প্রতিটি Dataset দিয়ে একটি করে Decision Tree তৈরি করা।
3. প্রতিটি Tree-তে Random Feature নির্বাচন করে Split করা।
4. সব Tree স্বাধীনভাবে Prediction প্রদান করে।
5. সব Tree-এর Prediction একত্র করে Final Prediction নির্ধারণ করা।

Random Forest Algorithm

Input

- Training Dataset
- Number of Trees (N)

Algorithm

Step 1: মূল Dataset থেকে Bootstrap Sampling ব্যবহার করে একাধিক Random Dataset তৈরি করা।

Step 2: প্রতিটি Dataset ব্যবহার করে একটি Decision Tree তৈরি করো।

Step 3: প্রতিটি Split-এর সময় সব Feature নয়, বরং Random কিছু Feature নির্বাচন করো।

Step 4: সব Decision Tree সম্পূর্ণভাবে Train করো।

Step 5: নতুন Data-এর জন্য প্রতিটি Tree আলাদাভাবে Prediction করবে।

Step 6: সব Prediction একত্র করে Final Prediction প্রদান করো।

Bootstrap Sampling

Random Forest-এ প্রতিটি Decision Tree একই Dataset দিয়ে Train হয় না।

বরং মূল Dataset থেকে **Replacement-এর মাধ্যমে Random Sampling** করা হয়। একে **Bootstrap Sampling** বলা হয়।

ধরো মূল Dataset,

$$[A, B, C, D, E]$$

একটি Bootstrap Sample হতে পারে,

$$[A, C, C, D, E]$$

এখানে একই Sample একাধিকবার আসতে পারে এবং কিছু Sample বাদও পড়তে পারে।

Random Feature Selection

Decision Tree-এর মতো সব Feature ব্যবহার না করে Random Forest প্রতিটি Split-এর সময় কিছু Random Feature নির্বাচন করে।

উদাহরণ,

ধরো মোট Feature,

- Age
- Salary
- Balance
- Credit Score
- Gender
- Location

একটি Split-এর সময় Random Forest হয়তো শুধুমাত্র,

- Salary
- Balance
- Credit Score

এই তিনটি Feature বিবেচনা করবে।

এর ফলে সব Tree একরকম হয় না এবং Diversity বৃদ্ধি পায়।

Prediction in Random Forest

Classification

Classification Problem-এ প্রতিটি Decision Tree একটি করে Class Prediction করে।

সবচেয়ে বেশি Vote পাওয়া Class Final Prediction হয়।

Formula,

$$\hat{y} = \text{Majority Vote of All Trees}$$

Example

ধরো ৫টি Decision Tree-এর Prediction হলো,

Tree Prediction

Tree 1 Spam

Tree 2 Spam

Tree 3 Not Spam

Tree 4 Spam

Tree 5 Not Spam

Voting,

- Spam = 3
- Not Spam = 2

অতএব,

$$\boxed{Prediction = Spam}$$

Regression

Regression-এর ক্ষেত্রে Majority Voting ব্যবহার করা হয় না।

সব Tree-এর Prediction-এর Average নেওয়া হয়।

Formula,

$$\hat{y} = \frac{1}{N} \sum_{i=1}^N T_i$$

Terms

- $\hat{y}$ = Predicted Value
 - N = Number of Trees
 - T_i = Prediction from Tree i
-

Example

ধরো,

৫টি Tree-এর Prediction,

- 52
- 54
- 53
- 55
- 56

তাহলে,

$$\begin{aligned}\hat{y} &= \frac{52 + 54 + 53 + 55 + 56}{5} \\ &= \frac{270}{5} \\ &\boxed{\hat{y} = 54}\end{aligned}$$

Difference Between Decision Tree and Random Forest

Decision Tree

একটি Tree ব্যবহার করে

Overfitting-এর ঝুঁকি বেশি

Prediction দ্রুত

Accuracy কম হতে পারে

Noise-এর প্রতি সংবেদনশীল

Random Forest

অনেকগুলো Decision Tree ব্যবহার করে

Overfitting কম

Prediction তুলনামূলক ধীর

সাধারণত Accuracy বেশি

Noise-এর প্রতি বেশি Robust

Advantages

- Overfitting কম হয়।
- Accuracy সাধারণত বেশি।
- Classification এবং Regression উভয়ের জন্য ব্যবহার করা যায়।
- Noise এবং Outlier-এর প্রতি তুলনামূলকভাবে Robust।
- Feature Importance নির্ণয় করা যায়।
- Missing Value-এর ক্ষেত্রেও ভালো Performance দিতে পারে।

Disadvantages

- Training Time বেশি।
- Prediction তুলনামূলক ধীর।
- Memory বেশি প্রয়োজন।
- Model ব্যাখ্যা করা একটি Decision Tree-এর তুলনায় কঠিন।

Applications

- Customer Churn Prediction
- Credit Risk Analysis
- Fraud Detection

- Disease Prediction
 - Stock Market Analysis
 - Recommendation System
 - Image Classification
 - Spam Detection
-

Key Points

- Random Forest একটি **Ensemble Learning Algorithm**।
- এটি একাধিক **Decision Tree**-এর সমন্বয়ে গঠিত।
- **Bootstrap Sampling** ব্যবহার করে বিভিন্ন Training Dataset তৈরি করা হয়।
- প্রতিটি Split-এ **Random Feature Selection** করা হয়।
- Classification-এ **Majority Voting** এবং Regression-এ **Average Prediction** ব্যবহার করা হয়।
- এটি সাধারণত একটি একক Decision Tree-এর তুলনায় বেশি Accurate এবং Overfitting-এর ঝুঁকি কম।

Gradient Boosted Trees (GBT)

Gradient Boosted Trees (GBT) হলো একটি **Supervised Ensemble Learning Algorithm**, যা একাধিক ছোট Decision Tree-কে ধারাবাহিকভাবে (Sequentially) তৈরি করে একটি শক্তিশালী Model গঠন করে। প্রতিটি নতুন Tree আগের Tree-এর Prediction Error বা Residual কমানোর চেষ্টা করে। অর্থাৎ, প্রতিটি Tree পূর্ববর্তী Tree-এর ভুল থেকে শিখে আরও ভালো Prediction প্রদান করে।

Decision Tree-এর মতো সব Tree স্বাধীনভাবে কাজ করে না। বরং প্রতিটি নতুন Tree আগের Model-এর ভুল সংশোধন করে। এই ধারাবাহিক শেখার (Sequential Learning) প্রক্রিয়াই Gradient Boosting-এর মূল বৈশিষ্ট্য।

Gradient Boosting-এ Loss Function কমানোর জন্য Gradient Descent-এর ধারণা ব্যবহার করা হয়। তাই এর নাম **Gradient Boosted Trees**।

Working Steps

1. একটি ছোট Decision Tree তৈরি করা।
2. প্রথম Tree-এর Error (Residual) গণনা করা।
3. সেই Error শেখার জন্য নতুন Tree তৈরি করা।
4. নতুন Tree-কে আগের Model-এর সাথে যুক্ত করা।
5. এই প্রক্রিয়া নির্দিষ্ট সংখ্যক Tree পর্যন্ত চলতে থাকে।

Gradient Boosting-এর জনপ্রিয় Implementation হলো,

- XGBoost
- LightGBM
- CatBoost

Advantages

- উচ্চ Accuracy প্রদান করে।
- Complex Pattern শিখতে পারে।
- Overfitting নিয়ন্ত্রণের বিভিন্ন ব্যবস্থা রয়েছে।

Disadvantages

- Training তুলনামূলক ধীর।
- Hyperparameter Tuning প্রয়োজন।
- Noise-এর প্রতি সংবেদনশীল হতে পারে।

Neural Networks

Neural Network হলো এমন একটি **Machine Learning Model**, যা মানুষের মস্তিষ্কের Neuron-এর কার্যপ্রণালী দ্বারা অনুপ্রাণিত। এটি Input Data থেকে Pattern শিখে Prediction করতে পারে এবং Deep Learning-এর ভিত্তি হিসেবে কাজ করে।

একটি Neural Network সাধারণত তিনটি Layer নিয়ে গঠিত।

- Input Layer
- Hidden Layer
- Output Layer

প্রতিটি Neuron Input গ্রহণ করে, Weight ও Bias ব্যবহার করে গণনা করে এবং একটি Activation Function-এর মাধ্যমে Output প্রদান করে।

একটি Neuron-এর গাণিতিক সমীকরণ,

$$z = w^T x + b$$

Activation Function প্রয়োগের পরে,

$$\hat{y} = f(z)$$

Terms

- x = Input Feature
- w = Weight
- b = Bias
- z = Weighted Sum
- $f(z)$ = Activation Function
- $\hat{y}$ = Output

Common Activation Functions

- Sigmoid
- ReLU
- Tanh
- Softmax

Applications

- Image Recognition
- Speech Recognition

- Natural Language Processing
 - Face Recognition
 - Medical Diagnosis
-

Ensemble Learning

Ensemble Learning হলো এমন একটি Machine Learning Technique যেখানে একাধিক Model-এর Prediction একত্র করে একটি শক্তিশালী Final Model তৈরি করা হয়।

একটি Model-এর পরিবর্তে অনেকগুলো Model একসাথে ব্যবহার করলে সাধারণত Accuracy বৃদ্ধি পায় এবং Overfitting কমে যায়।

Ensemble Learning-এর প্রধান তিনটি পদ্ধতি হলো,

- Bagging
- Boosting
- Stacking

Bagging

সব Model স্বাধীনভাবে (Parallel) Train হয় এবং শেষে Voting বা Averaging-এর মাধ্যমে Final Prediction দেয়।

Example: Random Forest

Boosting

সব Model ধারাবাহিকভাবে (Sequentially) Train হয় এবং প্রতিটি নতুন Model আগের Model-এর ভুল সংশোধন করে।

Example: Gradient Boosting, XGBoost, AdaBoost

Stacking

একাধিক Base Model-এর Prediction একটি Meta Model-এর Input হিসেবে ব্যবহার করা হয়।

Advantages

- Accuracy বৃদ্ধি করে।
- Overfitting কমায়ে।
- Prediction আরও Robust হয়।

Disadvantages

- Training Time বেশি।

- Computational Cost বেশি।
 - Model ব্যাখ্যা করা কঠিন।
-

Clustering Fundamentals

Clustering হলো একটি **Unsupervised Machine Learning Technique**, যেখানে Label ছাড়া Data-কে তাদের Similarity অনুযায়ী বিভিন্ন Group বা Cluster-এ ভাগ করা হয়।

একই Cluster-এর Data-গুলোর মধ্যে Similarity বেশি থাকে এবং ভিন্ন Cluster-এর Data-গুলোর মধ্যে Similarity কম থাকে।

Clustering-এর মূল উদ্দেশ্য হলো Dataset-এর Hidden Pattern বা Structure খুঁজে বের করা।

Clustering-এর সাধারণ ধাপগুলো হলো,

1. Dataset সংগ্রহ করা।
2. Similarity বা Distance Measure নির্বাচন করা।
3. Clustering Algorithm প্রয়োগ করা।
4. Similar Data-কে একই Cluster-এ Group করা।
5. Cluster বিশ্লেষণ করা।

Common Clustering Algorithms

- K-Means
- Hierarchical Clustering
- DBSCAN
- Gaussian Mixture Model (GMM)
- Fuzzy K-Means

Applications

- Customer Segmentation
- Image Segmentation
- Market Basket Analysis
- Document Clustering
- Social Network Analysis
- Anomaly Detection

Key Points

- Clustering একটি Unsupervised Learning Technique।
- পূর্বনির্ধারিত Label প্রয়োজন হয় না।
- Similar Data একই Cluster-এ থাকে।
- Dissimilar Data ভিন্ন Cluster-এ থাকে।
- K-Means, Hierarchical Clustering এবং DBSCAN সবচেয়ে জনপ্রিয় Clustering Algorithm।

Hard Clustering

Hard Clustering হলো এমন একটি Clustering Technique যেখানে প্রতিটি Data Point **শুধুমাত্র একটি Cluster-এর সদস্য** হতে পারে। অর্থাৎ, একটি Data একই সময়ে একাধিক Cluster-এ থাকতে পারে না।

Hard Clustering-এ Membership সম্পূর্ণ নির্দিষ্ট (Crisp Membership) হয়। একটি Data হয় Cluster-এর অন্তর্ভুক্ত হবে, নতুবা হবে না।

Example

ধরো, Customer-দের তিনটি Cluster-এ ভাগ করা হয়েছে।

- Cluster 1 = Students
- Cluster 2 = Businessman
- Cluster 3 = Retired People

যদি একজন Customer Cluster 2-তে থাকে, তাহলে সে Cluster 1 বা Cluster 3-এর সদস্য হতে পারবে না।

Common Algorithm

- K-Means Clustering

Advantages

- সহজ ও দ্রুত।
- Computational Cost কম।
- ব্যাখ্যা করা সহজ।

Disadvantages

- Overlapping Data-এর জন্য উপযুক্ত নয়।
- একটি Data একাধিক Cluster-এর বৈশিষ্ট্য থাকলেও একটি Cluster-এ সীমাবদ্ধ থাকে।

Soft Clustering

Soft Clustering (Fuzzy Clustering) হলো এমন একটি Clustering Technique যেখানে একটি Data Point **একাধিক Cluster-এর সদস্য হতে পারে**, তবে প্রতিটি Cluster-এর জন্য তার Membership Degree ভিন্ন হয়।

অর্থাৎ, একটি Data একই সঙ্গে বিভিন্ন Cluster-এর অন্তর্ভুক্ত হতে পারে।

Example

ধরো, একটি Customer-এর Membership হলো,

- Student = 0.20
- Businessman = 0.70

- Retired = 0.10

এখানে Customer মূলত Businessman Cluster-এর অন্তর্ভুক্ত, তবে Student Cluster-এর সাথেও কিছুটা মিল রয়েছে।

Common Algorithm

- Fuzzy C-Means (FCM)

Advantages

- Overlapping Data-এর জন্য উপযুক্ত।
- বাস্তব সমস্যায় বেশি কার্যকর।

Disadvantages

- Computational Cost বেশি।
- Hard Clustering-এর তুলনায় ধীর।

Partitioning Clustering

Partitioning Clustering হলো এমন একটি Clustering Method যেখানে Dataset-কে পূর্বনির্ধারিত **K সংখ্যক Cluster**-এ ভাগ করা হয়।

এই পদ্ধতিতে প্রতিটি Data Point শুধুমাত্র একটি Cluster-এর সদস্য হয় এবং সাধারণত Cluster-এর কেন্দ্র (Centroid)-এর ভিত্তিতে Grouping করা হয়।

সবচেয়ে জনপ্রিয় Partitioning Algorithm হলো **K-Means Clustering**।

Working Steps

1. K-এর মান নির্বাচন করা।
2. Kটি Initial Centroid নির্বাচন করা।
3. প্রতিটি Data-কে নিকটতম Centroid-এর Cluster-এ Assign করা।
4. নতুন Centroid গণনা করা।
5. Cluster পরিবর্তন না হওয়া পর্যন্ত প্রক্রিয়া পুনরাবৃত্তি করা।

Advantages

- সহজ এবং দ্রুত।
- বড় Dataset-এর জন্য কার্যকর।
- Implementation সহজ।

Disadvantages

- K আগে থেকে নির্ধারণ করতে হয়।
- Outlier-এর প্রতি সংবেদনশীল।
- গোলাকার (Spherical) Cluster-এর জন্য বেশি উপযোগী।

Common Algorithms

- K-Means
 - K-Medoids
-

Density-Based Clustering

Density-Based Clustering হলো এমন একটি Clustering Method যেখানে **High-Density Region**-কে একটি Cluster হিসেবে ধরা হয় এবং Low-Density Region-কে Cluster-এর সীমানা হিসেবে বিবেচনা করা হয়।

এই পদ্ধতিতে Cluster-এর Shape পূর্বনির্ধারিত নয়। তাই এটি অনিয়মিত (Irregular) আকৃতির Cluster-ও সনাক্ত করতে পারে।

সবচেয়ে জনপ্রিয় Density-Based Algorithm হলো **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**।

DBSCAN দুটি গুরুত্বপূর্ণ Parameter ব্যবহার করে।

- **Epsilon (ϵ)** = প্রতিবেশী (Neighborhood) নির্ধারণের Radius।
- **MinPts** = একটি Region-কে Dense Area হিসেবে গণ্য করার জন্য ন্যূনতম Data Point-এর সংখ্যা।

Working Steps

1. একটি Data Point নির্বাচন করা।
2. ϵ Radius-এর মধ্যে কতটি Neighbor আছে তা গণনা করা।
3. যদি Neighbor সংখ্যা $\geq$ MinPts হয়, তাহলে নতুন Cluster তৈরি করা।
4. Dense Region-এর সব Point-কে একই Cluster-এ যুক্ত করা।
5. Density না থাকলে Point-টিকে Noise বা Outlier হিসেবে চিহ্নিত করা।

Advantages

- K-এর মান আগে থেকে নির্ধারণ করতে হয় না।
- Outlier সনাক্ত করতে পারে।
- Irregular Shape-এর Cluster ভালোভাবে শনাক্ত করতে পারে।

Disadvantages

- এএবং MinPts নির্বাচন করা কঠিন।
- বিভিন্ন Density-এর Dataset-এ Performance কমতে পারে।

Difference Among the Four Methods

Method	Main Idea	One Data in Multiple Clusters?	Popular Algorithm
Hard Clustering	একটি Data শুধুমাত্র একটি Cluster-এ থাকে	No	K-Means
Soft Clustering	একটি Data একাধিক Cluster-এ থাকতে পারে	Yes	Fuzzy C-Means
Partitioning Clustering	Dataset-কে Kটি Cluster-এ ভাগ করে	No	K-Means, K-Medoids
Density-Based Clustering	Density-এর ভিত্তিতে Cluster তৈরি করে	No	DBSCAN

K-Means Clustering

K-Means Clustering হলো একটি জনপ্রিয় **Unsupervised Machine Learning Algorithm**, যা Data-কে তাদের Similarity-এর ভিত্তিতে **K সংখ্যক Cluster**-এ ভাগ করে। এটি **Partitioning Clustering**-এর অন্তর্ভুক্ত এবং প্রতিটি Data Point-কে শুধুমাত্র একটি Cluster-এ Assign করে।

K-Means-এর মূল লক্ষ্য হলো এমনভাবে Data-কে বিভিন্ন Cluster-এ ভাগ করা, যাতে একই Cluster-এর Data Point-গুলোর মধ্যে Similarity বেশি এবং ভিন্ন Cluster-এর Data Point-গুলোর মধ্যে Similarity কম হয়।

K-Means-এ প্রতিটি Cluster-এর একটি কেন্দ্র (Center) থাকে, যাকে **Centroid** বলা হয়। প্রতিটি Data Point তার নিকটতম Centroid-এর Cluster-এ যুক্ত হয়। এরপর প্রতিটি Cluster-এর নতুন Centroid গণনা করা হয়। এই প্রক্রিয়া বারবার চলতে থাকে যতক্ষণ না Cluster আর পরিবর্তিত হয়।

K-Means-এর "K" নির্দেশ করে মোট কতটি Cluster তৈরি করা হবে। এই মান Algorithm শুরু করার আগেই নির্ধারণ করতে হয়।

Working Principle of K-Means

K-Means Iterative Process অনুসরণ করে কাজ করে।

প্রথমে Kটি Initial Centroid নির্বাচন করা হয়। এরপর প্রতিটি Data Point-এর সাথে প্রতিটি Centroid-এর Distance গণনা করা হয়। যে Centroid-এর Distance সবচেয়ে কম হয়, সেই Cluster-এ Data Point-টি Assign করা হয়। এরপর প্রতিটি Cluster-এর নতুন Centroid নির্ণয় করা হয়। নতুন Centroid পাওয়ার পরে আবার Distance গণনা করা হয় এবং Cluster পুনরায় Update করা হয়।

এই ধাপগুলো পুনরাবৃত্তি হতে থাকে যতক্ষণ পর্যন্ত Cluster Assignment বা Centroid পরিবর্তন না হয়।

Centroid

Centroid হলো একটি Cluster-এর কেন্দ্রবিন্দু (Center Point)। এটি Cluster-এর সকল Data Point-এর Mean বা Average Position নির্দেশ করে।

দুইটি Feature থাকলে একটি Cluster-এর Centroid-এর Formula হলো,

$$C = \left(\frac{\sum x_i}{n}, \frac{\sum y_i}{n} \right)$$

Terms

- C = Centroid
- x_i = প্রথম Feature-এর মান
- y_i = দ্বিতীয় Feature-এর মান

- n = Cluster-এর মোট Data Point
-

Example

ধরো একটি Cluster-এ তিনটি Point রয়েছে।

$$(2, 3), (4, 5), (6, 7)$$

তাহলে,

$$x = \frac{2 + 4 + 6}{3} = 4$$
$$y = \frac{3 + 5 + 7}{3} = 5$$

অতএব,

$$\text{Centroid} = (4, 5)$$

Distance Calculation

K-Means সাধারণত **Euclidean Distance** ব্যবহার করে Data Point এবং Centroid-এর মধ্যকার দূরত্ব নির্ণয় করে।

Formula,

$$d = \sqrt{\sum_{i=1}^n (x_i - c_i)^2}$$

দুইটি Feature-এর জন্য,

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Terms

- d = Distance
- x_i = Data Point
- c_i = Centroid

- n = মোট Feature-এর সংখ্যা
-

Example

ধরো,

Data Point,

$$P = (3, 5)$$

Centroid,

$$C = (6, 9)$$

তাহলে,

$$\begin{aligned} d &= \sqrt{(6-3)^2 + (9-5)^2} \\ &= \sqrt{9+16} \\ &= \sqrt{25} \\ \boxed{d} &= 5 \end{aligned}$$

Distance যত কম হবে, Data Point সেই Cluster-এর তত কাছাকাছি হবে।

K-Means Algorithm

Input

- Training Dataset
 - Number of Clusters (K)
-

Algorithm

Step 1: K -এর মান নির্বাচন করো।

Step 2: Randomভাবে K টি Initial Centroid নির্বাচন করো।

Step 3: প্রতিটি Data Point-এর সাথে সব Centroid-এর Distance গণনা করো।

Step 4: প্রতিটি Data Point-কে নিকটতম Centroid-এর Cluster-এ Assign করো।

Step 5: প্রতিটি Cluster-এর নতুন Centroid (Mean) গণনা করো।

Step 6: নতুন Centroid ব্যবহার করে আবার Distance গণনা করো এবং Cluster পুনরায় Assign করো।

Step 7: যদি কোনো Data Point Cluster পরিবর্তন না করে অথবা Centroid আর পরিবর্তিত না হয়, তাহলে Algorithm বন্ধ করো। অন্যথায় Step 5-এ ফিরে যাও।

Example (Initial Clustering)

ধরো নিচের Dataset রয়েছে।

Point	X	Y
A	2	2
B	3	2
C	3	3
D	8	8
E	9	8
F	8	9

ধরি,

$$K = 2$$

Randomভাবে দুটি Initial Centroid নির্বাচন করা হলো,

$$\begin{aligned}C_1 &= (2, 2) \\C_2 &= (8, 8)\end{aligned}$$

এখন প্রতিটি Data Point-এর Distance এই দুইটি Centroid-এর সাথে গণনা করা হবে।

যে Centroid-এর Distance কম হবে, সেই Cluster-এ Data Point-টি Assign করা হবে।

Distance Calculation এবং Iteration-এর মাধ্যমে ধীরে ধীরে দুটি Cluster তৈরি হবে।

সম্পূর্ণ Distance Calculation, Cluster Assignment এবং প্রতিটি Iteration-এর Centroid Update Part 2-এ ধাপে ধাপে হাতে গণনা করে দেখানো হবে।

Key Points

- K-Means একটি **Unsupervised Learning Algorithm**।
- এটি **Partitioning Clustering** Technique ব্যবহার করে।
- প্রতিটি Data Point শুধুমাত্র একটি **Cluster**-এর সদস্য হয়।

- প্রতিটি Cluster-এর কেন্দ্রকে **Centroid** বলা হয়।
- সাধারণত **Euclidean Distance** ব্যবহার করে নিকটতম Centroid নির্ধারণ করা হয়।
- Algorithm Iterative Process অনুসরণ করে এবং Centroid স্থির (Converge) না হওয়া পর্যন্ত চলতে থাকে।
- K-এর মান Algorithm শুরু করার আগেই নির্ধারণ করতে হয়।

Complete Numerical Example of K-Means Clustering

K-Means Algorithm ভালোভাবে বোঝার জন্য একটি ছোট Dataset ব্যবহার করে প্রতিটি Iteration ধাপে ধাপে গণনা করা যাক।

Dataset

Point	X	Y
A	2	2
B	3	2
C	3	3
D	8	8
E	9	8
F	8	9

ধরি,

$$K = 2$$

Randomভাবে দুটি Initial Centroid নির্বাচন করা হলো,

$$C_1 = (2, 2)$$

$$C_2 = (8, 8)$$

Iteration 1

Euclidean Distance Formula,

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Distance Calculation

Point A (2,2)

To C_1

$$d = \sqrt{(2 - 2)^2 + (2 - 2)^2} = 0$$

To C_2

$$d = \sqrt{(8-2)^2 + (8-2)^2} = \sqrt{36+36} = \sqrt{72} = 8.49$$

Assign → **Cluster 1**

Point B (3,2)

To C_1

$$d = \sqrt{(3-2)^2 + (2-2)^2} = 1$$

To C_2

$$d = \sqrt{(8-3)^2 + (8-2)^2} = \sqrt{25+36} = \sqrt{61} = 7.81$$

Assign → **Cluster 1**

Point C (3,3)

To C_1

$$d = \sqrt{(3-2)^2 + (3-2)^2} = \sqrt{2} = 1.41$$

To C_2

$$d = \sqrt{(8-3)^2 + (8-3)^2} = \sqrt{50} = 7.07$$

Assign → **Cluster 1**

Point D (8,8)

To C_1

$$d = \sqrt{72} = 8.49$$

To C_2

$$d = 0$$

Assign → **Cluster 2**

Point E (9,8)

To C_1

$$d = \sqrt{85} = 9.22$$

To C_2

$$d = 1$$

Assign → **Cluster 2**

Point F (8,9)

To C_1

$$d = \sqrt{85} = 9.22$$

To C_2

$$d = 1$$

Assign → **Cluster 2**

Cluster After Iteration 1

Cluster 1 Cluster 2

A D

B E

C F

Calculate New Centroid

Cluster 1

Points:

$$(2, 2), (3, 2), (3, 3)$$

$$x = \frac{2 + 3 + 3}{3} = 2.67$$

$$y = \frac{2 + 2 + 3}{3} = 2.33$$

$$C_1 = (2.67, 2.33)$$

Cluster 2

Points:

$$(8, 8), (9, 8), (8, 9)$$

$$x = \frac{8 + 9 + 8}{3} = 8.33$$

$$y = \frac{8 + 8 + 9}{3} = 8.33$$

$$C_2 = (8.33, 8.33)$$

Iteration 2

নতুন Centroid ব্যবহার করে আবার Distance গণনা করা হয়।

Distance গণনার পর দেখা যায়,

Point	Assigned Cluster
A	Cluster 1
B	Cluster 1
C	Cluster 1
D	Cluster 2
E	Cluster 2
F	Cluster 2

কোনো Point Cluster পরিবর্তন করেনি।

তাই নতুন Centroid-ও অপরিবর্তিত থাকে।

Final Clusters

Cluster 1	Cluster 2
A (2,2)	D (8,8)
B (3,2)	E (9,8)
C (3,3)	F (8,9)

Final Centroid,

$$C_1 = (2.67, 2.33)$$

$$C_2 = (8.33, 8.33)$$

Objective (Cost) Function

K-Means-এর লক্ষ্য হলো প্রতিটি Data Point এবং তার নিজস্ব Centroid-এর মধ্যকার Squared Distance-এর যোগফলকে সর্বনিম্ন করা।

Formula,

$$J = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

Terms

- J = Cost Function
- K = Number of Clusters
- x = Data Point
- C_i = i -th Cluster
- μ_i = Centroid of Cluster i
- $\|x - \mu_i\|^2$ = Squared Euclidean Distance

Cost Function যত কম হবে, Clustering তত ভালো হবে।

Convergence

Convergence হলো সেই অবস্থা যখন Algorithm আর Cluster পরিবর্তন করে না।

K-Means সাধারণত নিচের যেকোনো একটি শর্ত পূরণ হলে Stop করে।

- কোনো Data Point আর Cluster পরিবর্তন না করলে।
- Centroid আর পরিবর্তিত না হলে।
- Maximum Iteration-এ পৌঁছে গেলে।

এই উদাহরণে **Iteration 2**-এর পর আর কোনো পরিবর্তন হয়নি। তাই Algorithm Converge করেছে এবং সেখানেই থেমে গেছে।

Choosing the Value of K

K-Means Algorithm ব্যবহার করার আগে **K (Number of Clusters)** নির্ধারণ করতে হয়। কিন্তু বাস্তবে অধিকাংশ ক্ষেত্রে K-এর সঠিক মান আগে থেকে জানা থাকে না। তাই বিভিন্ন পদ্ধতি ব্যবহার করে উপযুক্ত K নির্বাচন করা হয়।

সবচেয়ে জনপ্রিয় পদ্ধতি হলো **Elbow Method**। এছাড়াও Silhouette Score-এর মতো পদ্ধতিও ব্যবহৃত হয়, তবে K-Means শেখার প্রাথমিক পর্যায়ে Elbow Method-ই সবচেয়ে বেশি ব্যবহৃত হয়।

Elbow Method

Elbow Method হলো এমন একটি Technique, যেখানে বিভিন্ন K-এর জন্য K-Means চালিয়ে প্রতিবার **Cost Function (WCSS)** গণনা করা হয়। এরপর K-এর মানের বিপরীতে WCSS-এর একটি Graph আঁকা হয়।

যেখানে Graph-এর বাঁক (Elbow) তৈরি হয়, সেই K-কে সাধারণত সবচেয়ে উপযুক্ত Cluster সংখ্যা হিসেবে নির্বাচন করা হয়।

WCSS (Within-Cluster Sum of Squares)

WCSS হলো প্রতিটি Data Point এবং তার নিজস্ব Centroid-এর মধ্যকার Squared Distance-এর যোগফল।

Formula,

$$WCSS = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

Terms

- $WCSS$ = Within-Cluster Sum of Squares
- K = Number of Clusters
- C_i = i -th Cluster
- x = Data Point
- μ_i = Centroid of Cluster i

WCSS যত কম হবে, Data Point-গুলো Centroid-এর তত কাছাকাছি থাকবে এবং Clustering তত ভালো হবে।

Example of Elbow Method

ধরো বিভিন্ন K-এর জন্য WCSS পাওয়া গেল,

K	WCSS
1	950
2	520
3	240
4	180
5	165
6	155

এখানে দেখা যাচ্ছে,

- $K = 1 \rightarrow 2$ এ WCSS অনেক কমেছে।
- $K = 2 \rightarrow 3$ এ আবার অনেক কমেছে।
- কিন্তু $K = 3$ -এর পরে WCSS খুব ধীরে কমেছে।

অতএব,

$$K = 3$$

হতে পারে সবচেয়ে উপযুক্ত Cluster সংখ্যা।

Graph আঁকলে এটি অনেকটা কনুই (Elbow)-এর মতো দেখায়, তাই এর নাম **Elbow Method**।

Advantages of K-Means

- Algorithm সহজ এবং বোঝা সহজ।
- বড় Dataset-এর উপর দ্রুত কাজ করে।
- Computationally Efficient।
- সহজে Implement করা যায়।
- Cluster-এর Centroid সহজে পাওয়া যায়।
- Customer Segmentation-এর মতো সমস্যা খুব কার্যকর।

Disadvantages of K-Means

- K-এর মান আগে থেকে নির্ধারণ করতে হয়।
- Initial Centroid-এর উপর ফলাফল নির্ভর করে।

- Outlier-এর প্রতি সংবেদনশীল।
 - শুধুমাত্র Hard Clustering সমর্থন করে।
 - Non-Spherical বা Irregular Shape-এর Cluster ভালোভাবে শনাক্ত করতে পারে না।
 - বিভিন্ন Density-এর Cluster-এর ক্ষেত্রে Performance কমে যেতে পারে।
-

Applications of K-Means

- Customer Segmentation
 - Market Segmentation
 - Image Segmentation
 - Document Clustering
 - News Categorization
 - Recommendation Systems
 - Anomaly Detection
 - Pattern Recognition
 - Data Compression
-

Key Points

- K-Means একটি **Unsupervised Learning** Algorithm।
- এটি **Partitioning Clustering** Technique ব্যবহার করে।
- প্রতিটি Data Point শুধুমাত্র একটি Cluster-এর সদস্য হয়।
- Cluster-এর কেন্দ্রকে **Centroid** বলা হয়।
- সাধারণত **Euclidean Distance** ব্যবহার করে Distance গণনা করা হয়।
- Algorithm-এর লক্ষ্য হলো **WCSS (Within-Cluster Sum of Squares)** কমানো।
- উপযুক্ত K নির্বাচন করার জন্য সবচেয়ে জনপ্রিয় পদ্ধতি হলো **Elbow Method**।
- K-Means দ্রুত এবং কার্যকর হলেও Outlier এবং Irregular Shape-এর Cluster-এর ক্ষেত্রে সীমাবদ্ধতা রয়েছে।

Hierarchical Clustering

Hierarchical Clustering হলো একটি **Unsupervised Learning Algorithm**, যেখানে Data-কে ধাপে ধাপে (Hierarchy আকারে) বিভিন্ন Cluster-এ ভাগ করা হয়। এই পদ্ধতিতে K-এর মান শুরুতেই নির্ধারণ করতে হয় না। ফলাফল সাধারণত একটি **Dendrogram (Tree Diagram)** আকারে প্রদর্শিত হয়।

Hierarchical Clustering প্রধানত দুই ধরনের।

- **Agglomerative (Bottom-Up)**: প্রতিটি Data Point প্রথমে আলাদা Cluster হিসেবে শুরু করে, তারপর ধীরে ধীরে কাছাকাছি Cluster-গুলো একত্রিত হয়।
- **Divisive (Top-Down)**: শুরুতে সব Data একটি Cluster-এ থাকে, পরে ধাপে ধাপে ছোট ছোট Cluster-এ ভাগ হয়।

Advantages

- K আগে থেকে নির্ধারণ করতে হয় না।
- Dendrogram-এর মাধ্যমে Cluster সহজে বিশ্লেষণ করা যায়।
- ছোট Dataset-এর জন্য কার্যকর।

Disadvantages

- বড় Dataset-এ ধীরগতির।
- Computational Cost বেশি।
- একবার Cluster Merge বা Split হলে তা পরিবর্তন করা যায় না।

Fuzzy K-Means (Fuzzy C-Means)

Fuzzy K-Means বা **Fuzzy C-Means (FCM)** হলো একটি **Soft Clustering Algorithm**, যেখানে একটি Data Point একাধিক Cluster-এর সদস্য হতে পারে। তবে প্রতিটি Cluster-এর জন্য তার Membership Value (0 থেকে 1-এর মধ্যে) ভিন্ন হয়।

উদাহরণস্বরূপ, একটি Customer-এর Membership হতে পারে,

- Cluster A = 0.75
- Cluster B = 0.20
- Cluster C = 0.05

অর্থাৎ, Customer মূলত Cluster A-এর সদস্য, তবে অন্য Cluster-এর সাথেও কিছুটা সম্পর্ক রয়েছে।

Advantages

- Overlapping Data-এর জন্য উপযুক্ত।

- বাস্তব জীবনের অনেক সমস্যায় ভালো ফলাফল দেয়।

Disadvantages

- K-Means-এর তুলনায় ধীর।
- Computational Cost বেশি।
- Membership গণনা তুলনামূলক জটিল।

Gaussian Mixture Model (GMM)

Gaussian Mixture Model (GMM) হলো একটি **Probabilistic Clustering Algorithm**, যেখানে ধরা হয় যে Dataset একাধিক **Gaussian (Normal) Distribution**-এর সমন্বয়ে গঠিত।

K-Means-এর মতো একটি Data Point-কে নির্দিষ্ট একটি Cluster-এ Assign না করে, GMM প্রতিটি Data Point-এর জন্য প্রতিটি Cluster-এর **Probability** নির্ণয় করে।

উদাহরণ,

Cluster	Probability
Cluster 1	0.80
Cluster 2	0.15
Cluster 3	0.05

এখানে Data Point-এর Cluster 1-এ থাকার সম্ভাবনা 80%।

GMM সাধারণত **Expectation-Maximization (EM) Algorithm** ব্যবহার করে Model Train করে।

Advantages

- Soft Clustering সমর্থন করে।
- বিভিন্ন Shape-এর Cluster শনাক্ত করতে পারে।
- Probability ভিত্তিক ফলাফল প্রদান করে।

Disadvantages

- Training Time বেশি।
- Parameter Tuning প্রয়োজন।
- Outlier-এর প্রতি সংবেদনশীল হতে পারে।

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN হলো একটি **Density-Based Clustering Algorithm**, যা Data-এর ঘনত্ব (Density)-এর ভিত্তিতে Cluster তৈরি করে। এটি এমন অঞ্চলকে Cluster হিসেবে বিবেচনা করে যেখানে Data Point-এর ঘনত্ব বেশি এবং কম ঘনত্বের অঞ্চলকে Noise বা Outlier হিসেবে চিহ্নিত করে।

DBSCAN দুটি গুরুত্বপূর্ণ Parameter ব্যবহার করে।

- **Epsilon (ϵ):** একটি Point-এর Neighborhood Radius।
- **MinPts:** একটি Dense Region গঠনের জন্য প্রয়োজনীয় সর্বনিম্ন Data Point-এর সংখ্যা।

Working Steps

1. একটি Data Point নির্বাচন করা।
2. ϵ Radius-এর মধ্যে Neighbor গণনা করা।
3. Neighbor সংখ্যা $\geq$ MinPts হলে নতুন Cluster তৈরি করা।
4. Dense Region-এর Point-গুলোকে একই Cluster-এ যুক্ত করা।
5. Density না থাকলে Point-টিকে Noise হিসেবে চিহ্নিত করা।

Advantages

- K আগে থেকে নির্ধারণ করতে হয় না।
- Outlier সনাক্ত করতে পারে।
- Irregular Shape-এর Cluster ভালোভাবে শনাক্ত করতে পারে।

Disadvantages

- ϵ এবং MinPts নির্বাচন করা কঠিন।
- বিভিন্ন Density-এর Dataset-এ Performance কমে যেতে পারে।

Association Rule Mining

Association Rule Mining হলো একটি **Unsupervised Machine Learning Technique**, যা Dataset-এর বিভিন্ন Item-এর মধ্যে সম্পর্ক (Association) বা Pattern খুঁজে বের করে। এটি বিশেষ করে **Market Basket Analysis**-এ বেশি ব্যবহৃত হয়, যেখানে জানা যায় কোন কোন পণ্য একসাথে কেনা হয়।

Association Rule সাধারণত **IF → THEN** আকারে প্রকাশ করা হয়।

উদাহরণ,

$$\{Bread\} \rightarrow \{Butter\}$$

অর্থাৎ, যে Customer Bread কিনেছে, তার Butter কেনার সম্ভাবনাও বেশি।

Association Rule Mining-এর তিনটি গুরুত্বপূর্ণ Measure হলো,

- **Support:** একটি Itemset Dataset-এ কতবার এসেছে।
- **Confidence:** IF অংশ সত্য হলে THEN অংশ সত্য হওয়ার সম্ভাবনা।
- **Lift:** দুটি Item-এর মধ্যে সম্পর্কের শক্তি নির্দেশ করে।

Popular Algorithms

- Apriori Algorithm
- FP-Growth Algorithm

Applications

- Market Basket Analysis
- Product Recommendation
- Cross Selling
- Web Usage Analysis

Dimensionality Reduction

Dimensionality Reduction হলো এমন একটি Technique, যেখানে Dataset-এর **Feature (Dimension)** সংখ্যা কমিয়ে গুরুত্বপূর্ণ তথ্য সংরক্ষণ করা হয়।

বাস্তবে অনেক Dataset-এ শত বা হাজার Feature থাকতে পারে। সব Feature সমান গুরুত্বপূর্ণ নয়। অপ্রয়োজনীয় বা অতিরিক্ত Feature বাদ দিলে Model দ্রুত Train হয়, Memory কম লাগে এবং অনেক ক্ষেত্রে Accuracy-ও বৃদ্ধি পায়।

Dimensionality Reduction প্রধানত দুই ধরনের।

- **Feature Selection:** গুরুত্বপূর্ণ Feature নির্বাচন করা।
- **Feature Extraction:** পুরোনো Feature থেকে নতুন Feature তৈরি করা।

Advantages

- Training Time কমায়।
- Memory Usage কমায়।
- Overfitting কমাতে সাহায্য করে।
- Visualization সহজ করে।

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) হলো একটি জনপ্রিয় **Dimensionality Reduction Algorithm**, যা একাধিক Correlated Feature-কে কয়েকটি নতুন **Principal Component**-এ রূপান্তর করে।

PCA-এর লক্ষ্য হলো এমন নতুন Feature তৈরি করা, যাতে Dataset-এর সর্বাধিক Variance সংরক্ষিত থাকে এবং কম সংখ্যক Dimension ব্যবহার করেও অধিকাংশ তথ্য ধরে রাখা যায়।

PCA-তে,

- **First Principal Component (PC1)** সর্বাধিক Variance ধারণ করে।
- **Second Principal Component (PC2)** PC1-এর লম্ব (Orthogonal) হয় এবং দ্বিতীয় সর্বাধিক Variance ধারণ করে।
- একইভাবে PC3, PC4 ইত্যাদি তৈরি হয়।

PCA-এর প্রধান ধাপগুলো হলো,

1. Feature Scaling করা।
2. Covariance Matrix তৈরি করা।
3. Eigenvalue ও Eigenvector নির্ণয় করা।
4. সবচেয়ে বড় Eigenvalue-এর Eigenvector নির্বাচন করা।
5. Data-কে নতুন Principal Component-এ Project করা।

Advantages

- Feature সংখ্যা কমায়।
- Training দ্রুত হয়।
- Noise কমাতে সাহায্য করে।

- Data Visualization সহজ হয়।

Disadvantages

- নতুন Feature-এর অর্থ (Interpretation) বোঝা কঠিন।
- কিছু তথ্য (Information Loss) হারাতে পারে।
- Scaling না করলে ভালো ফলাফল নাও দিতে পারে।

Applications

- Image Compression
- Face Recognition
- Data Visualization
- Noise Reduction
- Feature Extraction

Key Points

- Association Rule Mining Data-এর মধ্যে সম্পর্ক খুঁজে বের করে।
- Dimensionality Reduction Feature সংখ্যা কমিয়ে গুরুত্বপূর্ণ তথ্য সংরক্ষণ করে।
- PCA হলো সবচেয়ে জনপ্রিয় Feature Extraction Technique।
- PCA নতুন Principal Component তৈরি করে এবং সর্বাধিক Variance সংরক্ষণ করার চেষ্টা করে।

Bias

Bias হলো Model-এর এমন একটি ত্রুটি (Error), যা ঘটে যখন Model বাস্তব Data-এর প্রকৃত Pattern সঠিকভাবে শিখতে পারে না। অর্থাৎ, Model খুব বেশি সরল (Too Simple) হওয়ার কারণে গুরুত্বপূর্ণ সম্পর্কগুলো উপেক্ষা করে ফেলে।

উচ্চ Bias থাকলে Model Training Data এবং Test Data উভয়ের উপরই খারাপ Performance করে। কারণ Model Data সম্পর্কে ভুল অনুমান (Assumption) করে।

সহজভাবে,

High Bias = Model খুব সরল, তাই ঠিকমতো শেখে না।

Example

ধরো, নিচের Data রয়েছে।

Study Hours	Marks
1	20
2	35
3	50
4	70
5	90

যদি একটি Model এই Data-এর জন্য শুধু একটি Horizontal Line Prediction করে, তাহলে এটি প্রকৃত Trend ধরতে পারবে না।

ফলে Prediction Error অনেক বেশি হবে।

এটি **High Bias**-এর উদাহরণ।

Characteristics of High Bias

- Model খুব Simple হয়।
 - গুরুত্বপূর্ণ Pattern শিখতে পারে না।
 - Training Error বেশি।
 - Test Error-ও বেশি।
 - সাধারণত Underfitting ঘটে।
-

Variance

Variance হলো Model-এর এমন একটি বৈশিষ্ট্য, যেখানে Training Data-এর ছোট পরিবর্তনের কারণেও Model-এর Prediction অনেক পরিবর্তিত হয়।

উচ্চ Variance থাকলে Model Training Data খুব ভালোভাবে মুখস্থ (Memorize) করে ফেলে। ফলে Training Accuracy অনেক বেশি হলেও নতুন Data-এর উপর Performance কমে যায়।

সহজভাবে,

High Variance = Model Training Data অতিরিক্ত শিখে ফেলে।

Example

ধরো, Student-এর Marks Predict করার জন্য একটি Model ব্যবহার করা হয়েছে।

Model Training Data-এর প্রতিটি Point-এর মধ্য দিয়ে একটি জটিল Curve তৈরি করেছে।

Training Accuracy = 100%

কিন্তু নতুন Student-এর Data দিলে Prediction ভুল হচ্ছে।

এটি **High Variance**-এর উদাহরণ।

Characteristics of High Variance

- Model খুব Complex হয়।
 - Training Data মুখস্থ করে ফেলে।
 - Training Error খুব কম।
 - Test Error বেশি।
 - সাধারণত Overfitting ঘটে।
-

Difference Between Bias and Variance

Bias	Variance
Model খুব Simple	Model খুব Complex
গুরুত্বপূর্ণ Pattern শিখতে পারে না	Training Data মুখস্থ করে ফেলে
Underfitting সৃষ্টি করে	Overfitting সৃষ্টি করে

Training Error বেশি	Training Error কম
Test Error বেশি	Test Error বেশি

Bias-Variance Tradeoff

Bias-Variance Tradeoff হলো এমন একটি ধারণা, যেখানে Model-এর **Bias** এবং **Variance**-এর মধ্যে ভারসাম্য (Balance) বজায় রাখার চেষ্টা করা হয়।

Machine Learning-এ এমন Model তৈরি করাই লক্ষ্য, যা খুব বেশি Simple নয় এবং খুব বেশি Complex-ও নয়।

- Model খুব Simple হলে **Bias বাড়ে** এবং Underfitting হয়।
- Model খুব Complex হলে **Variance বাড়ে** এবং Overfitting হয়।

সঠিক Model হলো এমন Model, যেখানে Bias এবং Variance উভয়ই তুলনামূলকভাবে কম থাকে।

Relationship

Model Complexity	Bias	Variance
Low	High	Low
Medium	Balanced	Balanced
High	Low	High

Example

ধরো, Student-এর Study Hours থেকে Marks Predict করা হচ্ছে।

- যদি একটি সরল Line ব্যবহার করা হয়, তাহলে অনেক Pattern বাদ পড়বে → **High Bias (Underfitting)**।
- যদি খুব জটিল Curve ব্যবহার করা হয়, তাহলে Training Data পুরোপুরি Fit হবে → **High Variance (Overfitting)**।
- যদি উপযুক্ত একটি Curve ব্যবহার করা হয়, তাহলে Training এবং Test উভয় Data-তেই ভালো Prediction পাওয়া যাবে → **Good Tradeoff**।

How to Reduce Bias

- আরও Complex Model ব্যবহার করা।

- আরও Feature যোগ করা।
 - Training Time বৃদ্ধি করা।
 - Underfitting কমানো।
-

How to Reduce Variance

- আরও বেশি Training Data ব্যবহার করা।
 - Regularization ব্যবহার করা।
 - Feature Selection করা।
 - Model Complexity কমানো।
 - Ensemble Learning (যেমন Random Forest) ব্যবহার করা।
-

Key Points

- **Bias** হলো Model-এর ভুল Assumption-এর কারণে হওয়া Error।
- **High Bias** সাধারণত **Underfitting** সৃষ্টি করে।
- **Variance** হলো Training Data-এর প্রতি Model-এর অতিরিক্ত সংবেদনশীলতা।
- **High Variance** সাধারণত **Overfitting** সৃষ্টি করে।
- **Bias-Variance Tradeoff**-এর লক্ষ্য হলো এমন একটি Model নির্বাচন করা, যেখানে Bias এবং Variance-এর মধ্যে সর্বোত্তম ভারসাম্য থাকে এবং Test Data-তে ভালো Generalization পাওয়া যায়।

Overfitting

Overfitting হলো এমন একটি সমস্যা যেখানে Machine Learning Model Training Data-কে অতিরিক্ত ভালোভাবে শিখে ফেলে। Model শুধু মূল Pattern-ই নয়, Training Data-এর Noise এবং Outlier-ও মুখস্থ করে ফেলে। ফলে Training Data-তে খুব ভালো ফলাফল দিলেও নতুন বা অজানা (Unseen) Data-এর উপর ভালো Performance করতে পারে না।

সহজভাবে,

Overfitting = Model Training Data মুখস্থ করে ফেলে, কিন্তু নতুন Data-তে ভালো Generalize করতে পারে না।

Overfitting সাধারণত তখন ঘটে যখন Model খুব বেশি Complex হয় অথবা Training Data-এর তুলনায় Model-এর Parameters অনেক বেশি থাকে।

Example

ধরো, নিচের Dataset রয়েছে।

Study Hours	Marks
1	20
2	35
3	50
4	68
5	90

যদি একটি খুব জটিল Polynomial Curve প্রতিটি Data Point-এর মধ্য দিয়ে যায়, তাহলে Training Data-তে Error প্রায় শূন্য হবে।

কিন্তু নতুন একটি Student যদি 3.5 ঘন্টা পড়ে, তখন Model ভুল Prediction দিতে পারে।

এটি **Overfitting**-এর উদাহরণ।

Characteristics

- Model খুব Complex হয়।
- Training Data মুখস্থ করে ফেলে।
- Training Accuracy খুব বেশি।
- Training Error খুব কম।

- Test Accuracy কম।
 - Test Error বেশি।
 - High Variance সৃষ্টি করে।
-

Causes of Overfitting

- Model অত্যন্ত Complex হওয়া।
 - খুব কম Training Data।
 - অনেক বেশি Feature ব্যবহার করা।
 - দীর্ঘ সময় Training করা।
 - Noise বা Outlier বেশি থাকা।
-

How to Prevent Overfitting

- আরও বেশি Training Data ব্যবহার করা।
 - Feature Selection করা।
 - Regularization (L1/L2) ব্যবহার করা।
 - Model Complexity কমানো।
 - Cross Validation ব্যবহার করা।
 - Ensemble Learning ব্যবহার করা।
 - Early Stopping ব্যবহার করা।
-

Underfitting

Underfitting হলো এমন একটি সমস্যা যেখানে Model Training Data-এর প্রকৃত Pattern-ই শিখতে পারে না। অর্থাৎ Model এতটাই Simple হয় যে Data-এর গুরুত্বপূর্ণ সম্পর্কগুলো ধরতে ব্যর্থ হয়।

ফলে Training Data এবং Test Data উভয়ের উপরই Model খারাপ Performance করে।

সহজভাবে,

Underfitting = Model যথেষ্ট শেখেই না।

Example

আগের একই Dataset,

Study Hours	Marks
1	20
2	35
3	50
4	68
5	90

যদি Model সব Student-এর জন্য একই Marks Predict করে, যেমন 50, তাহলে এটি Data-এর প্রকৃত Trend ধরতে পারবে না।

এটি **Underfitting**-এর উদাহরণ।

Characteristics

- Model খুব Simple হয়।
 - গুরুত্বপূর্ণ Pattern শিখতে পারে না।
 - Training Accuracy কম।
 - Test Accuracy কম।
 - Training Error বেশি।
 - Test Error বেশি।
 - High Bias সৃষ্টি করে।
-

Causes of Underfitting

- Model খুব Simple হওয়া।
 - Feature কম ব্যবহার করা।
 - পর্যাপ্ত Training না হওয়া।
 - Model Capacity কম হওয়া।
-

How to Prevent Underfitting

- আরও Complex Model ব্যবহার করা।
- আরও গুরুত্বপূর্ণ Feature যোগ করা।
- Training Time বৃদ্ধি করা।
- Hyperparameter Tuning করা।
- ভালো Feature Engineering করা।

Difference Between Overfitting and Underfitting

Overfitting	Underfitting
Model খুব Complex	Model খুব Simple
Training Data মুখস্থ করে	Pattern-ই শিখতে পারে না
High Variance	High Bias
Training Accuracy বেশি	Training Accuracy কম
Test Accuracy কম	Test Accuracy কম
Training Error কম	Training Error বেশি
Test Error বেশি	Test Error বেশি

Key Points

- **Overfitting** ঘটে যখন Model Training Data অতিরিক্ত শিখে ফেলে এবং নতুন Data-তে ভালো Performance করতে পারে না।
- **Underfitting** ঘটে যখন Model Training Data-এর মূল Pattern-ই শিখতে ব্যর্থ হয়।
- Overfitting সাধারণত **High Variance**-এর সাথে সম্পর্কিত।
- Underfitting সাধারণত **High Bias**-এর সাথে সম্পর্কিত।
- একটি ভালো Machine Learning Model-এর লক্ষ্য হলো **Overfitting এবং Underfitting উভয়ই এড়িয়ে ভালো Generalization অর্জন করা।**

Regularization

Regularization হলো এমন একটি Technique, যা Machine Learning Model-এ **Overfitting কমানোর জন্য** ব্যবহার করা হয়। এটি Model-এর Cost Function-এর সাথে একটি **Penalty Term** যোগ করে, যাতে Model অপ্রয়োজনীয়ভাবে বড় বড় Weight শিখতে না পারে।

যখন Model-এর Weight (w) খুব বড় হয়ে যায়, তখন Model Training Data-কে অতিরিক্ত ভালোভাবে Fit করে। Regularization এই বড় Weight-গুলোর উপর Penalty আরোপ করে Model-কে সহজ (Simpler) রাখতে সাহায্য করে।

Regularization-এর পরে নতুন Cost Function হয়,

$$J_{reg} = J + \lambda \times \text{Penalty}$$

অথবা,

$$\text{New Cost Function} = \text{Original Cost Function} + \text{Regularization Term}$$

Terms

- J = Original Cost Function
- J_{reg} = Regularized Cost Function
- λ = Regularization Parameter
- Penalty = Weight-এর উপর অতিরিক্ত শাস্তি (Penalty)

λ -এর মান যত বড় হবে, Penalty তত বেশি হবে এবং Model তত সহজ (Simple) হবে।

Regularization-এর সবচেয়ে জনপ্রিয় দুইটি পদ্ধতি হলো,

- Ridge Regression (L2 Regularization)
- Lasso Regression (L1 Regularization)

Ridge Regression (L2 Regularization)

Ridge Regression হলো এমন একটি Regularization Technique যেখানে Cost Function-এর সাথে **Weight-এর Square** যোগ করা হয়।

এর ফলে বড় বড় Weight-এর মান ছোট হয়ে যায়, তবে সাধারণত কোনো Weight সম্পূর্ণ 0 হয় না।

Formula

$$J = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^n w_j^2$$

Terms

- m = Number of Training Samples
- n = Number of Features
- y_i = Actual Value
- $\hat{y}_i$ = Predicted Value
- w_j = Weight
- λ = Regularization Parameter

Characteristics

- Weight-এর মান ছোট করে।
- কোনো Feature সম্পূর্ণ বাদ দেয় না।
- সব Feature Model-এ থাকে।
- Multicollinearity কমাতে কার্যকর।
- Overfitting কমায়ে।

Example

ধরো Model-এর Weight,

$$w = [8, 5, 0.8, 3]$$

Ridge প্রয়োগ করার পরে,

$$w = [5.6, 3.8, 0.6, 2.1]$$

এখানে সব Weight ছোট হয়েছে, কিন্তু কোনোটিই 0 হয়নি।

Lasso Regression (L1 Regularization)

Lasso Regression হলো এমন একটি Regularization Technique যেখানে Cost Function-এর সাথে **Weight-এর Absolute Value** যোগ করা হয়।

Lasso-এর সবচেয়ে বড় বৈশিষ্ট্য হলো এটি **অপ্রয়োজনীয় Feature-এর Weight সম্পূর্ণ 0 করে দিতে পারে**। ফলে এটি Feature Selection-ও করে।

Formula

$$J = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^n |w_j|$$

Terms

- m = Number of Training Samples
- n = Number of Features
- w_j = Weight
- λ = Regularization Parameter

Characteristics

- কিছু Weight সম্পূর্ণ 0 করে দেয়।
- অপ্রয়োজনীয় Feature বাদ দেয়।
- Feature Selection করতে পারে।
- Sparse Model তৈরি করে।
- Overfitting কমায়।

Example

ধরো,

$$w = [8, 5, 0.8, 3]$$

Lasso প্রয়োগ করার পরে,

$$w = [5.4, 3.5, 0, 1.9]$$

এখানে তৃতীয় Weight সম্পূর্ণ 0 হয়ে গেছে। অর্থাৎ, সেই Feature Model থেকে বাদ পড়েছে।

Difference Between Ridge and Lasso Regression

Ridge Regression	Lasso Regression
L2 Regularization ব্যবহার করে	L1 Regularization ব্যবহার করে
Weight-এর Square ব্যবহার করে	Weight-এর Absolute Value ব্যবহার করে
কোনো Weight সম্পূর্ণ 0 হয় না	কিছু Weight সম্পূর্ণ 0 হয়ে যেতে পারে
Feature Selection করতে পারে না	Feature Selection করতে পারে
সব Feature Model-এ থাকে	অপ্রয়োজনীয় Feature বাদ দিতে পারে
Multicollinearity কমাতে কার্যকর	Sparse Model তৈরিতে কার্যকর

When to Use

Ridge Regression

- যখন সব Feature গুরুত্বপূর্ণ।
- যখন Multicollinearity বেশি।
- যখন Feature বাদ দিতে না চাই।

Lasso Regression

- যখন অনেক Feature-এর মধ্যে গুরুত্বপূর্ণ Feature নির্বাচন করতে চাই।
- যখন অপ্রয়োজনীয় Feature বাদ দিতে চাই।
- High-Dimensional Dataset-এর ক্ষেত্রে।

Key Points

- Regularization-এর মূল উদ্দেশ্য হলো **Overfitting কমানো**।
- এটি Cost Function-এর সাথে একটি **Penalty Term** যোগ করে।
- **Ridge Regression (L2)** Weight ছোট করে, কিন্তু সাধারণত **0 করে না**।
- **Lasso Regression (L1)** কিছু Weight **0 করে Feature Selection** করতে পারে।
- λ -এর মান যত বেশি হবে, Regularization-এর প্রভাব তত বেশি হবে।

Min-Max Normalization

Min-Max Normalization (Min-Max Scaling) হলো একটি **Feature Scaling Technique**, যেখানে একটি Feature-এর সব মানকে একটি নির্দিষ্ট Range-এর মধ্যে রূপান্তর করা হয়। সাধারণত এই Range **0 থেকে 1** অথবা **-1 থেকে 1** হয়।

Machine Learning-এ বিভিন্ন Feature-এর মানের পরিসর (Scale) অনেক ভিন্ন হতে পারে। যেমন, একটি Feature-এর মান 1–10 এবং অন্যটির মান 10,000–100,000 হতে পারে। এই পার্থক্যের কারণে অনেক Algorithm (যেমন KNN, K-Means, SVM, Neural Network) বড় মানের Feature-কে বেশি গুরুত্ব দিতে পারে। Min-Max Normalization এই সমস্যা দূর করে সব Feature-কে একই Scale-এ নিয়ে আসে।

Formula

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Terms

- x = Original Value
- x' = Normalized Value
- $x_{\min}$ = Minimum Value of the Feature
- $x_{\max}$ = Maximum Value of the Feature

Normalization-এর পরে,

$$0 \leq x' \leq 1$$

General Formula

যদি Normalized Range $[a, b]$ হয়, তাহলে Formula হবে,

$$x' = a + \frac{(x - x_{\min})(b - a)}{x_{\max} - x_{\min}}$$

Terms

- a = New Minimum Value
 - b = New Maximum Value
-

Example

ধরো একটি Feature-এর মানগুলো হলো,

Original Value
20
40
60
80
100

এখানে,

$$\begin{aligned}x_{\min} &= 20 \\x_{\max} &= 100\end{aligned}$$

এখন Value = 60 কে Normalize করি।

$$\begin{aligned}x' &= \frac{60 - 20}{100 - 20} \\&= \frac{40}{80} \\&= 0.5\end{aligned}$$

অতএব,

$$60 \rightarrow 0.5$$

একইভাবে,

Original Value	Normalized Value
20	0.00
40	0.25
60	0.50
80	0.75
100	1.00

Advantages

- সব Feature একই Scale-এ নিয়ে আসে।
- Distance-Based Algorithm-এর Performance উন্নত করে।
- Gradient Descent দ্রুত Converge করতে সাহায্য করে।
- Training Process আরও Stable হয়।
- Implementation সহজ।

Disadvantages

- Outlier-এর প্রতি সংবেদনশীল।
- নতুন Data-এর জন্য একই Minimum ও Maximum Value ব্যবহার করতে হয়।
- Outlier থাকলে অধিকাংশ Data ছোট Range-এ সংকুচিত হয়ে যেতে পারে।

Applications

- K-Nearest Neighbors (KNN)
- K-Means Clustering
- Support Vector Machine (SVM)
- Neural Networks
- Gradient Descent ভিত্তিক Model

Difference Between Normalization and Standardization

Normalization (Min-Max Scaling)	Standardization (Z-Score Scaling)
নির্দিষ্ট Range-এ Scale করে (সাধারণত 0–1)	Mean = 0 এবং Standard Deviation = 1 করে
Minimum ও Maximum ব্যবহার করে	Mean ও Standard Deviation ব্যবহার করে
Outlier-এর প্রতি বেশি সংবেদনশীল	Outlier-এর প্রতি তুলনামূলক কম সংবেদনশীল
KNN, K-Means, Neural Network-এর জন্য বেশি ব্যবহৃত	Linear Regression, Logistic Regression, PCA ইত্যাদিতে বেশি ব্যবহৃত

Key Points

- Min-Max Normalization একটি **Feature Scaling Technique**।
- এটি Feature-এর মানকে সাধারণত **0 থেকে 1** Range-এর মধ্যে রূপান্তর করে।
- Formula হলো,

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- Distance-Based Algorithm-এ এটি খুবই কার্যকর।
- Outlier থাকলে Min-Max Scaling-এর Performance প্রভাবিত হতে পারে।

Naïve Bayes Algorithm

Naïve Bayes হলো একটি **Supervised Machine Learning Algorithm**, যা মূলত **Classification** সমস্যার সমাধানে ব্যবহৃত হয়। এটি **Bayes' Theorem**-এর উপর ভিত্তি করে কাজ করে এবং ধরে নেয় যে সব Feature একে অপরের থেকে **স্বাধীন (Independent)**। এই "Naïve" বা সরল ধারণা থেকেই এর নাম **Naïve Bayes**।

যদিও বাস্তবে Feature-গুলো অনেক সময় একে অপরের উপর নির্ভরশীল হয়, তবুও Naïve Bayes অনেক বাস্তব সমস্যায় দ্রুত এবং ভালো ফলাফল প্রদান করে।

Bayes' Theorem

Naïve Bayes-এর মূল ভিত্তি হলো **Bayes' Theorem**।

Formula,

$$P(C | X) = \frac{P(X | C) \times P(C)}{P(X)}$$

Terms

- $P(C | X)$ = Posterior Probability (Data X দেওয়া থাকলে Class C -এর Probability)
- $P(X | C)$ = Likelihood (Class C হলে Data X -এর Probability)
- $P(C)$ = Prior Probability (Class-এর পূর্ববর্তী Probability)
- $P(X)$ = Evidence (Data X -এর মোট Probability)

Working Principle

Naïve Bayes প্রতিটি Class-এর জন্য Posterior Probability গণনা করে। যে Class-এর Probability সবচেয়ে বেশি হয়, Model সেই Class-কে Final Prediction হিসেবে নির্বাচন করে।

অর্থাৎ,

$$Prediction = \arg \max P(C | X)$$

Naïve Assumption

Naïve Bayes ধরে নেয় যে সব Feature পরস্পরের থেকে স্বাধীন।

যদি Feature হয়,

- Age
- Salary
- Balance

তাহলে Algorithm ধরে নেয়,

- Age, Salary-এর উপর নির্ভর করে না।
- Salary, Balance-এর উপর নির্ভর করে না।
- Balance, Age-এর উপর নির্ভর করে না।

এই Independence Assumption-ই Naïve Bayes-কে দ্রুত এবং সহজ করে তোলে।

Example

ধরো একটি Email Spam Detection System রয়েছে।

Email	Class
"Win Prize"	Spam
"Meeting Today"	Not Spam
"Free Offer"	Spam

নতুন Email:

"Free Prize"

Naïve Bayes প্রতিটি Class-এর Probability গণনা করবে।

ধরা যাক,

$$P(\text{Spam} | X) = 0.92$$
$$P(\text{Not Spam} | X) = 0.08$$

যেহেতু,

$$0.92 > 0.08$$

অতএব,

$Prediction = \text{Spam}$

Types of Naïve Bayes

Gaussian Naïve Bayes

Continuous Numerical Data-এর জন্য ব্যবহৃত হয় এবং ধরে নেয় যে Data Gaussian (Normal) Distribution অনুসরণ করে।

Example: Height, Weight, Salary

Multinomial Naïve Bayes

Count বা Frequency ভিত্তিক Data-এর জন্য ব্যবহৃত হয়।

Example: Document Classification, Word Count

Bernoulli Naïve Bayes

Binary Feature (0 বা 1)-এর জন্য ব্যবহৃত হয়।

Example: Email-এ কোনো নির্দিষ্ট শব্দ আছে (1) বা নেই (0)

Advantages

- সহজ এবং দ্রুত Train হয়।
 - ছোট Dataset-এও ভালো কাজ করে।
 - High-Dimensional Data-এর জন্য উপযুক্ত।
 - Text Classification-এ খুব কার্যকর।
 - কম Memory প্রয়োজন।
-

Disadvantages

- Feature Independence Assumption সবসময় বাস্তবসম্মত নয়।
 - Correlated Feature থাকলে Performance কমতে পারে।
 - যদি কোনো Feature-এর Probability 0 হয়, তাহলে Prediction প্রভাবিত হতে পারে (এটি Laplace Smoothing দিয়ে সমাধান করা যায়)।
-

Applications

- Email Spam Detection
 - Sentiment Analysis
 - Document Classification
 - News Categorization
 - Medical Diagnosis
 - Language Detection
-

Key Points

- Naïve Bayes একটি Supervised Classification Algorithm।
- এটি Bayes' Theorem ব্যবহার করে Prediction করে।
- এটি ধরে নেয় যে সব Feature Independent।
- যে Class-এর Posterior Probability সবচেয়ে বেশি হয়, সেটিই Final Prediction হয়।
- Gaussian, Multinomial এবং Bernoulli হলো Naïve Bayes-এর প্রধান তিনটি ধরন।
- এটি বিশেষভাবে Text Classification এবং Spam Detection-এ অত্যন্ত জনপ্রিয়।

Logistic Regression

Logistic Regression হলো একটি Supervised Machine Learning Algorithm, যা মূলত Classification Problem সমাধানের জন্য ব্যবহৃত হয়। নামের মধ্যে "Regression" থাকলেও এটি Regression-এর জন্য নয়, বরং একটি Observation কোন Class-এর অন্তর্ভুক্ত হবে তা নির্ধারণ করার জন্য Probability Predict করে।

সবচেয়ে বেশি ব্যবহৃত হয় **Binary Classification**-এ, যেখানে দুটি সম্ভাব্য Class থাকে, যেমন:

- Yes / No
- Spam / Not Spam
- Churn / Not Churn
- Pass / Fail
- Disease / No Disease

Logistic Regression প্রথমে একটি **Linear Equation** ব্যবহার করে একটি Score (z) গণনা করে। এরপর সেই Score-কে **Sigmoid Function**-এর মাধ্যমে 0 থেকে 1-এর মধ্যে একটি Probability-তে রূপান্তর করে।

Why Not Linear Regression?

Classification সমস্যায় Linear Regression ব্যবহার করলে Prediction 0-এর নিচে বা 1-এর উপরে চলে যেতে পারে, যা Probability হিসেবে গ্রহণযোগ্য নয়।

উদাহরণ,

$$\hat{y} = 1.45$$

বা

$$\hat{y} = -0.3$$

Probability কখনোই 1-এর বেশি বা 0-এর কম হতে পারে না।

এই সমস্যার সমাধানের জন্য Logistic Regression **Sigmoid Function** ব্যবহার করে।

Linear Equation

প্রথমে একটি Linear Score গণনা করা হয়।

$$z = w^T x + b$$

অথবা,

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Terms

- z = Linear Score
- w = Weight
- x = Input Feature
- b = Bias

Sigmoid Function

এরপর Linear Score (z)-কে Sigmoid Function-এর মাধ্যমে Probability-তে রূপান্তর করা হয়।

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

অথবা,

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

Terms

- $\hat{y}$ = Predicted Probability
- z = Linear Score
- e = Euler's Number (≈ 2.718)
- σ = Sigmoid Function

Sigmoid Function-এর Output সর্বদা,

$$0 < \hat{y} < 1$$

অর্থাৎ Probability সবসময় 0 এবং 1-এর মধ্যে থাকবে।

Decision Boundary

Probability পাওয়ার পরে একটি **Threshold** ব্যবহার করে Final Class নির্ধারণ করা হয়।

সাধারণত,

- যদি $\text{Probability} \geq 0.5$ হয় $\rightarrow \text{Class} = 1$
- যদি $\text{Probability} < 0.5$ হয় $\rightarrow \text{Class} = 0$

Formula,

$$\hat{y} = \begin{cases} 1, & P(Y = 1) \geq 0.5 \\ 0, & P(Y = 1) < 0.5 \end{cases}$$

Working Steps

1. Training Data সংগ্রহ করা।
2. Linear Score (z) গণনা করা।
3. Sigmoid Function ব্যবহার করে Probability নির্ণয় করা।
4. Cost Function ব্যবহার করে Error গণনা করা।
5. Gradient Descent-এর মাধ্যমে Weight ও Bias Update করা।
6. Training সম্পন্ন হলে নতুন Data-এর Probability Predict করা।
7. Threshold ব্যবহার করে Final Class নির্ধারণ করা।

Example

ধরো একজন Student-এর পরীক্ষায় Pass করার Probability Predict করতে হবে।

Feature:

- Study Hours = 5

ধরি,

$$w = 1.2, b = -4$$

তাহলে,

$$\begin{aligned} z &= 1.2 \times 5 - 4 \\ &= 6 - 4 \\ &= 2 \end{aligned}$$

এখন Sigmoid Function ব্যবহার করি,

$$\begin{aligned}\hat{y} &= \frac{1}{1 + e^{-2}} \\ &= \frac{1}{1 + 0.135} \\ &= 0.881\end{aligned}$$

অর্থাৎ,

$$P(\text{Pass}) = 88.1\%$$

যেহেতু,

$$0.881 > 0.5$$

অতএব,

$$\text{Prediction} = \text{Pass}$$

Cost Function (Log Loss / Binary Cross-Entropy)

Linear Regression-এর মতো MSE সাধারণত Logistic Regression-এর জন্য ব্যবহার করা হয় না। এর পরিবর্তে **Log Loss (Binary Cross-Entropy Loss)** ব্যবহার করা হয়।

$$J = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Terms

- J = Cost Function
- m = Number of Training Samples
- y_i = Actual Label
- $\hat{y}_i$ = Predicted Probability

Advantages

- সহজ এবং দ্রুত Train হয়।
- Probability Predict করতে পারে।
- Binary Classification-এর জন্য খুব কার্যকর।

- তুলনামূলকভাবে কম Computational Cost।
 - ফলাফল সহজে ব্যাখ্যা করা যায়।
-

Disadvantages

- Linear Decision Boundary ধরে নেয়।
 - Complex Non-linear Data-এর ক্ষেত্রে Performance কম হতে পারে।
 - Outlier দ্বারা প্রভাবিত হতে পারে।
 - Feature Engineering-এর উপর Performance নির্ভর করতে পারে।
-

Applications

- Email Spam Detection
 - Customer Churn Prediction
 - Disease Prediction
 - Credit Risk Analysis
 - Fraud Detection
 - Pass/Fail Prediction
 - Sentiment Analysis
-

Key Points

- Logistic Regression একটি Supervised Classification Algorithm।
- এটি প্রথমে Linear Equation ব্যবহার করে z গণনা করে।
- এরপর Sigmoid Function ব্যবহার করে Probability নির্ণয় করে।
- Output সবসময় 0 থেকে 1-এর মধ্যে থাকে।
- সাধারণত 0.5 Threshold ব্যবহার করে Final Class নির্ধারণ করা হয়।
- এটি মূলত Binary Classification সমস্যার জন্য ব্যবহৃত হয়, তবে One-vs-Rest (OvR) পদ্ধতির মাধ্যমে Multiclass Classification-এও ব্যবহার করা যায়।

Lazy Learner vs Eager Learner

Machine Learning Algorithm-কে Training Process-এর ভিত্তিতে প্রধানত দুই ভাগে ভাগ করা যায়।

- **Lazy Learner**
- **Eager Learner**

মূল পার্থক্য হলো, Model **কখন শেখে (Learn করে)**।

Lazy Learner

Lazy Learner এমন Algorithm, যা Training-এর সময় কোনো General Model তৈরি করে না। এটি Training Data সংরক্ষণ করে রাখে এবং নতুন Data (Query Point) এলে তখনই Prediction করার সময় শেখা (Learning) শুরু করে।

অর্থাৎ, Training Phase খুবই দ্রুত হয়, কিন্তু Prediction করতে তুলনামূলক বেশি সময় লাগে।

Characteristics

- Training Data সংরক্ষণ করে।
- Prediction-এর সময় Learning করে।
- Training Time কম।
- Prediction Time বেশি।
- Memory বেশি লাগে।
- Local Pattern ভালোভাবে শিখতে পারে।

Examples

- K-Nearest Neighbors (KNN)
 - Case-Based Reasoning
-

Eager Learner

Eager Learner এমন Algorithm, যা Training-এর সময়ই Dataset থেকে একটি General Model তৈরি করে ফেলে। এরপর নতুন Data এলে সেই তৈরি করা Model ব্যবহার করে দ্রুত Prediction দেয়।

অর্থাৎ, Training করতে সময় বেশি লাগে, কিন্তু Prediction খুব দ্রুত হয়।

Characteristics

- Training-এর সময় Model তৈরি করে।

- Prediction-এর সময় Model ব্যবহার করে।
- Training Time বেশি।
- Prediction Time কম।
- Memory তুলনামূলক কম লাগে।
- General Pattern শিখে।

Examples

- Linear Regression
- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- Naïve Bayes
- Neural Networks

Example

ধরো, ১০,০০০টি Student-এর Data দিয়ে একটি Model তৈরি করা হবে।

Lazy Learner (KNN)

Training-এর সময় শুধু সব Student-এর Data সংরক্ষণ করবে। নতুন Student এলে তখন Distance গণনা করে Prediction করবে।

Eager Learner (Decision Tree)

Training-এর সময়ই পুরো Decision Tree তৈরি করবে। নতুন Student এলে শুধু Tree অনুসরণ করেই দ্রুত Prediction দেবে।

Difference Between Lazy Learner and Eager Learner

Lazy Learner	Eager Learner
Training-এর সময় Model তৈরি করে না	Training-এর সময় Model তৈরি করে
Training Data সংরক্ষণ করে	Training Data থেকে General Model শেখে

Training Time কম	Training Time বেশি
Prediction Time বেশি	Prediction Time কম
Memory বেশি লাগে	Memory তুলনামূলক কম লাগে
Query আসলে Learning করে	Training-এর সময়ই Learning সম্পন্ন করে
Local Pattern-এর উপর নির্ভর করে	General Pattern শেখে
Example: KNN	Example: Linear Regression, Logistic Regression, Decision Tree, Random Forest, SVM, Naïve Bayes

Key Points

- **Lazy Learner** Prediction-এর সময় শেখে এবং সাধারণত Training Data সংরক্ষণ করে রাখে।
- **Eager Learner** Training-এর সময়ই একটি General Model তৈরি করে।
- Lazy Learner-এর **Training দ্রুত** কিন্তু **Prediction ধীর**।
- Eager Learner-এর **Training ধীর** কিন্তু **Prediction দ্রুত**।
- **KNN** হলো সবচেয়ে পরিচিত **Lazy Learner**, আর **Decision Tree, Random Forest, SVM, Naïve Bayes, Logistic Regression** ইত্যাদি হলো **Eager Learner**।

Hamming Distance

Hamming Distance হলো দুটি **সমান দৈর্ঘ্যের (Equal Length)** Binary String বা Categorical Data-এর মধ্যে **কতটি Position-এ মান ভিন্ন** আছে, তার সংখ্যা।

এটি সাধারণত **KNN**, Error Detection, Coding Theory এবং Text Processing-এ ব্যবহৃত হয়।

Formula

$$\text{Hamming Distance} = \sum_{i=1}^n I(x_i \neq y_i)$$

Terms

- x_i = প্রথম String-এর i -তম মান
- y_i = দ্বিতীয় String-এর i -তম মান
- $I(x_i \neq y_i)$ = মান ভিন্ন হলে 1, একই হলে 0
- n = মোট Position

Example

String 1:

101110

String 2:

100100

Position	1	2	3	4	5	6
String 1	1	0	1	1	1	0
String 2	1	0	0	1	0	0
Different?	No	No	Yes	No	Yes	No

মোট ভিন্ন Position = 2

$$\text{Hamming Distance} = 2$$

Key Points

- শুধুমাত্র **সমান দৈর্ঘ্যের Data**-এর জন্য প্রযোজ্য।
- ভিন্ন Position-এর সংখ্যা গণনা করে।
- Binary ও Categorical Data-এর জন্য বেশি ব্যবহৃত হয়।
- Distance যত কম হবে, দুটি Data তত বেশি Similar।

Linear Regression-এর Summation Formula

Simple Linear Regression-এ **Intercept (a)** এবং **Slope (b)** সরাসরি নিচের Summation Formula ব্যবহার করে নির্ণয় করা যায়।

Slope (b)

$$b = \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2}$$

Intercept (a)

$$a = \frac{\sum y \sum x^2 - \sum x \sum xy}{n \sum x^2 - (\sum x)^2}$$

Linear Regression Equation

$$y = a + bx$$

Short Example

x	y	x^2	xy
1	2	1	2
2	3	4	6
3	5	9	15

$$n = 3, \sum x = 6, \sum y = 10, \sum x^2 = 14, \sum xy = 23$$

$$b = \frac{3(23) - 6(10)}{3(14) - 6^2} = \frac{69 - 60}{42 - 36} = \frac{9}{6} = 1.5$$

$$a = \frac{10(14) - 6(23)}{3(14) - 6^2} = \frac{140 - 138}{42 - 36} = \frac{2}{6} = 0.33$$

অতএব,

$$y = 0.33 + 1.5x$$

এটি হলো Summation Formula ব্যবহার করে Linear Regression Equation নির্ণয়ের সংক্ষিপ্ত উদাহরণ।

Difference between K-Means and KNN Algorithm

K-Means	K-Nearest Neighbors (KNN)
Unsupervised Learning Algorithm	Supervised Learning Algorithm
Clustering-এর জন্য ব্যবহৃত হয়	Classification ও Regression-এর জন্য ব্যবহৃত হয়
Label ছাড়া Data নিয়ে কাজ করে	Labeled Data প্রয়োজন
Similar Data-কে বিভিন্ন Cluster-এ ভাগ করে	নতুন Data-এর Class Predict করে
Training-এর সময় Cluster তৈরি করে	Prediction-এর সময় Nearest Neighbor খুঁজে বের করে
Centroid ব্যবহার করে	Nearest Neighbor ব্যবহার করে
সাধারণত Euclidean Distance ব্যবহার করে	Euclidean, Manhattan, Hamming ইত্যাদি Distance ব্যবহার করতে পারে
Output হলো Cluster Number	Output হলো Predicted Class বা Value
Example: Customer Segmentation	Example: Spam Detection, Disease Prediction

Choosing Reason of K-Means and KNN

When to Choose K-Means

K-Means ব্যবহার করা উচিত যখন,

- Dataset-এ কোনো Label নেই।
- Similar Data-কে Group বা Cluster করতে হবে।
- Customer Segmentation করতে হবে।
- Data-এর Hidden Pattern খুঁজে বের করতে হবে।
- Market Segmentation বা Image Segmentation করতে হবে।

When to Choose KNN

KNN ব্যবহার করা উচিত যখন,

- Dataset Labeled।
- নতুন Data-এর Class Predict করতে হবে।
- Classification বা Regression সমস্যা সমাধান করতে হবে।

- Dataset ছোট বা মাঝারি আকারের।
- Distance-এর ভিত্তিতে Prediction করতে হবে।